THE COMPLETE LINUX LEARNING SERIES · LEVELS 1–3

Linux from Beginner to Advanced

by **Ajay Upadhyay**

DevOps Mentor · ajayupadhyaydevops@gmail.com

The complete Ubuntu guide for DevOps engineers, in one book.
24 chapters in simple language, with real terminal output explained line by line, diagrams and flowcharts, practice labs, quizzes, three final projects and three cheat sheets.

```
ajay@ubuntu:~$ echo "Level 1: Beginner Foundations"
ajay@ubuntu:~$ echo "Level 2: Daily Administration"
ajay@ubuntu:~$ echo "Level 3: Advanced Administration"
ajay@ubuntu:~$ sudo learn linux --from=zero --to=production
ajay@ubuntu:~$ _
```

Level 1: Chapters 1–7 · Level 2: Chapters 8–17 · Level 3: Chapters 18–24
Based on Ubuntu 24.04 LTS

About This Book

Hi, I'm **Ajay Upadhyay**, a DevOps mentor. This book brings my three Linux guides together into one complete course, from your very first command to running and troubleshooting production servers. It is written the way I teach my own students: simple words first, everyday comparisons, then real commands with the real output explained.

Level	You will be able to…
Level 1 Beginner Foundations	Understand what Linux is, use the terminal confidently, move around the filesystem, manage files, edit with nano and vim, create users and groups, and control permissions.
Level 2 Daily Administration	Search and analyse logs and text, use pipes and redirection, find files, install software, manage processes and services, read logs, make backups, schedule jobs and manage disks with LVM.
Level 3 Advanced Administration	Work with networks and DNS, use SSH like a professional, secure servers with firewalls, write Bash automation, run nginx, SSL and databases, and troubleshoot production with a proven method.

> **How to read it:** Go in order. Each level starts with its own cover page and a short guide to its boxes and practice files, and ends with a final project and a printable cheat sheet. Type every command yourself in an Ubuntu virtual machine.

Contents

Level 1: Beginner Foundations (Chapters 1–7)		**4**
Ch 1	Introduction to Linux	**6**
Ch 2	The Shell and Terminal	**9**
Ch 3	The Linux Filesystem	**13**
Ch 4	Managing Files and Directories	**17**
Ch 5	Viewing and Editing Files	**21**
Ch 6	Users and Groups	**25**
Ch 7	File Permissions and Ownership	**29**
	Final Project and Cheat Sheet	**34**
Level 2: Daily Administration (Chapters 8–17)		**36**
Ch 8	Text Processing and Filters	**39**
Ch 9	I/O Redirection and Pipes	**48**
Ch 10	Finding Files	**53**
Ch 11	Package Management on Ubuntu	**57**
Ch 12	Process Management	**63**
Ch 13	Services and systemd	**69**
Ch 14	Logs and Monitoring	**75**
Ch 15	Archiving and Compression	**81**
Ch 16	Task Scheduling	**85**
Ch 17	Disk and Storage Management	**90**
	Final Project and Cheat Sheet	**97**
Level 3: Advanced Administration (Chapters 18–24)		**100**
Ch 18	Networking Fundamentals	**102**
Ch 19	SSH and Remote Access	**111**
Ch 20	Firewall and Security	**118**
Ch 21	Bash Shell Scripting	**124**
Ch 22	Environment and Shell Customization	**134**
Ch 23	Web Servers and Common Services	**139**
Ch 24	Performance Monitoring and Troubleshooting	**146**
	Final Project and Cheat Sheet	**152**

LINUX LEARNING SERIES · LEVEL 1

Linux for Absolute Beginners

by **Ajay Upadhyay**

DevOps Mentor · ajayupadhyaydevops@gmail.com

A friendly, step-by-step guide to Ubuntu Linux.
Simple language, real examples, pictures and practice — written so you can learn it, and teach it.

```
student@ubuntu:~$ whoami
student
student@ubuntu:~$ echo "I am learning Linux!"
I am learning Linux!
student@ubuntu:~$ _
```

Chapters 1–7 | Introduction · Shell · Filesystem · Files · Viewing & Editing · Users · Permissions
Based on Ubuntu 24.04 LTS (commands work on most Ubuntu versions)

How to Use This Guide

Hi, I'm **Ajay Upadhyay**, a DevOps mentor. I wrote this guide the way I teach my own students: as if a friendly teacher is sitting next to you. You do **not** need any computer knowledge before starting. Every new word is explained the first time it appears.

Each chapter has the same parts, so you always know what to expect:

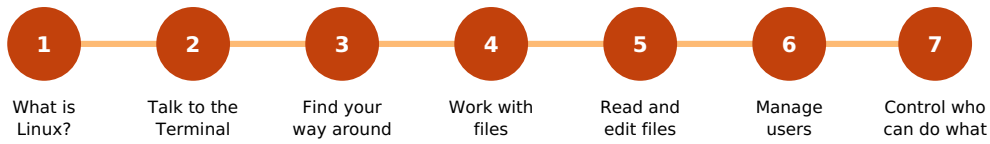
Box	What it means
Real-Life Example (green)	Explains a computer idea using something from everyday life.
Tip (blue)	A helpful shortcut or good habit.
Careful! (red)	A common mistake, or a command that can cause damage.
Teaching Note (purple)	Advice for when you explain this to someone else.
Dark boxes	The terminal. **Green text** is the prompt (do not type it). White text is what you type. Grey text after `#` is a comment explaining the line.
Practice Lab	Hands-on exercises. Do them! Linux is learned by typing, not only reading.
Quick Quiz	Check yourself. Answers are at the bottom of each quiz.

> **Tip: The golden rule.** Type every command yourself. Do not copy and paste in the beginning. Your fingers need to learn it too.

Contents

Ch 1	**Introduction to Linux** — What Linux is, kernel vs distribution, Ubuntu versions, installing Ubuntu
Ch 2	**The Shell and Terminal** — Bash, how a command is built, getting help, history and shortcuts
Ch 3	**The Linux Filesystem** — The folder tree, important folders, absolute vs relative paths
Ch 4	**Managing Files and Directories** — Create, copy, move, delete, wildcards, links
Ch 5	**Viewing and Editing Files** — cat, less, head, tail, nano and vim
Ch 6	**Users and Groups** — Creating users, groups, sudo, and where user data is stored
Ch 7	**File Permissions and Ownership** — rwx, chmod, chown, numbers like 755, special permissions, ACLs
End	**Final Project and Cheat Sheet** — Put it all together

Your Learning Path



Each chapter builds on the one before. Go in order.

CHAPTER 1

Ajay Upadhyay · DevOps Mentor

Introduction to Linux

What Linux is, why everyone in DevOps uses it, and how to get Ubuntu running on your computer.

In this chapter you will learn:

- What an operating system is
- What Linux is and where it came from
- The difference between the *kernel* and a *distribution*
- Ubuntu versions: LTS vs normal, Server vs Desktop
- Three easy ways to install Ubuntu and start practising

1.1 First, What Is an Operating System?

A computer is made of hardware: the processor (CPU), memory (RAM), hard disk, keyboard, screen and so on. Hardware alone cannot do anything useful. It needs a manager that tells each part what to do. That manager is called the **Operating System** (OS).

> **Real-Life Example: The Restaurant**
> Think of a restaurant. The **kitchen, stoves and fridges** are the hardware. The **customers** are the apps you use (Chrome, WhatsApp, games). The **restaurant manager** is the operating system. Customers never walk into the kitchen and switch on the stove themselves. They tell the manager what they want, and the manager makes sure the kitchen does it, fairly and safely.

Popular operating systems you already know: **Windows**, **macOS**, **Android** (which is actually built on Linux!) and **Linux**.

1.2 What Is Linux?

Linux is a free, open-source operating system. "Open source" means anyone can see its code, change it and share it. It was started in **1991** by a Finnish student named **Linus Torvalds** as a hobby project. Today it runs most of the world:

Where Linux runs	How much
Web servers on the internet	The large majority of them
Cloud (AWS, Azure, Google Cloud)	Most virtual machines run Linux
Top 500 supercomputers	All of them
Android phones	Billions of devices (Android uses the Linux kernel)
Docker containers and Kubernetes	Built on Linux features

> **Why this matters for DevOps:** Almost every server, container and CI/CD pipeline you will touch runs Linux. Knowing Linux well is not optional for a DevOps engineer; it is the ground you stand on.

1.3 Kernel vs Distribution (Very Important!)

People say "Linux" for two different things. Let's separate them.

The Kernel is the core, the heart of the operating system. It talks directly to the hardware: it decides which program uses the CPU, how memory is shared, how files are saved on disk. Strictly speaking, "Linux" is only the name of the kernel.

A Distribution (or "distro") is the kernel *plus* everything else you need: tools, commands, a software installer, maybe a desktop with icons. Different groups package these differently, so we get many distros.

Real-Life Example: The Car
The **kernel** is the car **engine**. A **distribution** is the **complete car**: engine + seats + steering + paint + music system. Maruti, Hyundai and Tata can all use similar engines but sell very different cars. In the same way, Ubuntu, Fedora and Debian all use the Linux kernel but feel different to use.

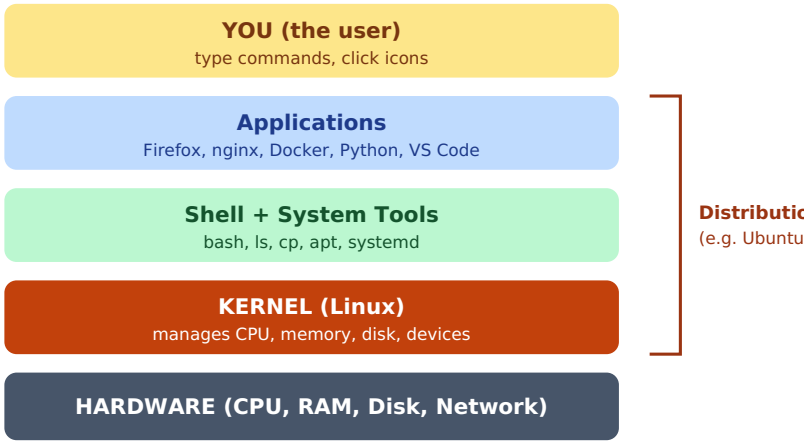


Figure 1.1 — The layers of a Linux system. A distribution bundles the kernel with tools and apps.

Popular Distributions

Distribution	Family	Known for
Ubuntu	Debian	Most popular on cloud servers and for beginners. **This guide uses Ubuntu.**
Debian	Debian	Very stable; Ubuntu is built on top of it
Red Hat (RHEL), Rocky, AlmaLinux	Red Hat	Big companies, banks
Fedora	Red Hat	Newest features first
Alpine	Independent	Very tiny; used in Docker images
Amazon Linux	Red Hat-like	Made for AWS

1.4 Ubuntu Versions Explained

Ubuntu releases a new version every 6 months, in **April** and **October**. The version number is just the **year.month**:

Version	Meaning
`22.04`	Released in 2022, April (04)
`24.04`	Released in 2024, April
`24.10`	Released in 2024, October

Each version also has a funny animal nickname, like "Noble Numbat" for 24.04.

LTS vs Interim

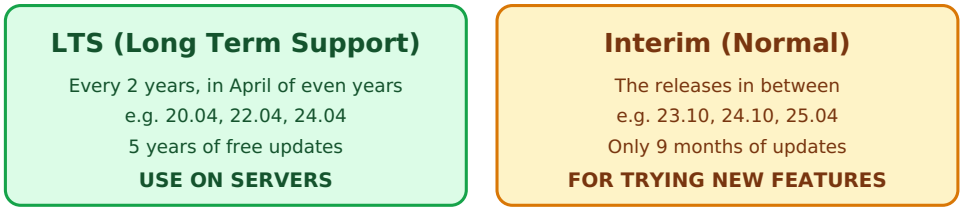


Figure 1.2 — In real jobs, servers almost always run an LTS version.

Desktop vs Server

Ubuntu Desktop has a graphical screen with windows, icons and a mouse, just like Windows. **Ubuntu Server** has *no* graphics; you only get a black screen with a text prompt. Servers skip the graphics to save memory and be more secure. As a DevOps engineer, you will mostly use Server, usually connecting to it from your laptop.

1.5 Installing Ubuntu: Three Easy Ways

Method	How it works	Best for
1. WSL2 (Windows)	Runs Ubuntu inside Windows. Open PowerShell as Admin and run `wsl --install`, then restart.	Fastest start if you use Windows
2. Virtual Machine	Install VirtualBox (free), download the Ubuntu .iso file from ubuntu.com, create a new VM with 2 CPUs, 4 GB RAM, 25 GB disk.	Safe practice; you can break it and rebuild
3. Cloud instance	Launch an Ubuntu 24.04 machine on AWS EC2, Azure or GCP (free tiers exist), then connect with SSH.	Real DevOps experience

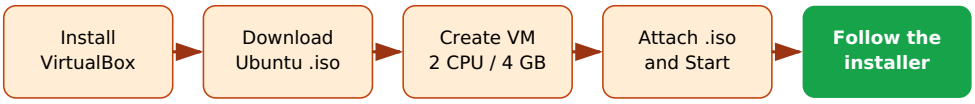


Figure 1.3 — Flow for installing Ubuntu in a Virtual Machine.

Your First Commands

Once Ubuntu is running, open the **Terminal** (press `Ctrl + Alt + T` on Desktop). Try these:

```
student@ubuntu:~$ lsb_release -a          # Which Ubuntu version am I using?
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.1 LTS
Release:        24.04
Codename:       noble

student@ubuntu:~$ uname -r                # Which kernel version?
6.8.0-45-generic

student@ubuntu:~$ sudo apt update         # Refresh the list of available software
student@ubuntu:~$ sudo apt upgrade -y     # Install all available updates
```

Teaching Note: Let the learner run `lsb_release -a` and `uname -r` themselves. Then ask: "Which one tells you about the *engine* and which one about the *car*?" (uname = kernel/engine, lsb_release = distribution/car.) This makes the kernel-vs-distro idea stick.

Chapter 1 Summary

- An operating system manages hardware for the apps you use.
- Linux is free and open-source, and it runs most servers, clouds and containers.
- **Kernel** = the engine. **Distribution** = the full car (Ubuntu, Debian, RHEL...).
- Ubuntu versions are *year.month*. Use **LTS** versions on servers.
- Practise using WSL2, a Virtual Machine or a cloud server.

Quick Quiz

1. Who created Linux, and in which year?
2. Is "Ubuntu" a kernel or a distribution?
3. What does the version number 22.04 mean?
4. Which should a production server use: 24.04 or 24.10? Why?
5. Which command shows the kernel version?

Answers: 1) Linus Torvalds, 1991. 2) A distribution. 3) Released April 2022. 4) 24.04, because it is LTS with 5 years of updates. 5) `uname -r`

CHAPTER 2

Ajay Upadhyay · DevOps Mentor

The Shell and Terminal

How to talk to Linux by typing, and how to get help when you are stuck.

In this chapter you will learn:

- What the terminal, the shell and Bash are
- How to read the prompt
- How every command is built: command, options, arguments
- How to get help: `man` , `--help` , `tldr`
- History and keyboard shortcuts that save hours

2.1 Terminal, Shell and Bash

These three words confuse almost everyone at first. Here is the simple version:

Word	What it is
Terminal	The **window** where you type. Just a screen and keyboard input.
Shell	The **program inside** the terminal that understands your commands and runs them.
Bash	The **name** of the most common shell. Ubuntu uses Bash by default. (Others: zsh, fish, sh.)

Real-Life Example: The Phone Call
The **terminal** is your **phone**. The **shell** is the **translator on the other end** of the call. You speak (type a command), the translator understands it and tells the kernel what to do, then reports the answer back to you. **Bash** is simply the translator's name.

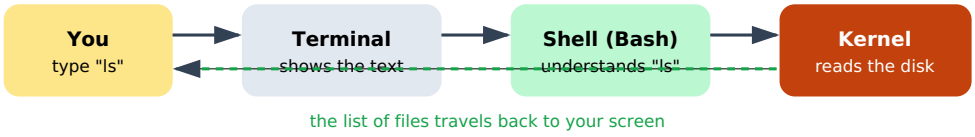


Figure 2.1 — What happens when you type a command.

2.2 Reading the Prompt

When the terminal is ready for you, it shows a **prompt**. It is full of information:



Figure 2.2 — Parts of the Bash prompt.

Careful! If your prompt ends with `#` instead of `$` , you are the **root** user (the all-powerful administrator). Any mistake you make can damage the whole system. Be extra careful.

2.3 How Every Command Is Built

Almost every Linux command follows the same pattern, like a sentence:

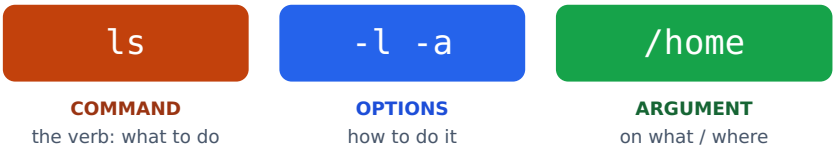


Figure 2.3 — "List, in long format, including hidden files, the folder /home."

- **Command**: the action. Always comes first.
- **Options** (also called flags): change how the command behaves. Short options use one dash and one letter (`-l`). Long options use two dashes and a word (`--all`). Short options can be joined: `-l -a` is the same as `-la` .
- **Arguments**: the thing the command works on, like a file or folder name.

```
student@ubuntu:~$ ls                  # command only
student@ubuntu:~$ ls -l               # command + option
student@ubuntu:~$ ls -la /etc         # command + 2 joined options + argument
student@ubuntu:~$ ls --all --human-readable -l /var/log   # long options
```

> **Careful! Linux is case-sensitive.** `ls` works, but `LS` or `Ls` gives "command not found". Also, `File.txt` and `file.txt` are two *different* files.

Some Simple Commands to Try Right Now

Command	What it does	Example output
`whoami`	Shows your username	`student`
`hostname`	Shows the computer's name	`ubuntu`
`date`	Shows date and time	`Tue Oct 13 10:30:00 IST 2026`
`cal`	Shows a calendar (install with `sudo apt install ncal`)	A month calendar
`uptime`	How long the computer has been on	`up 3 days, 2:14`
`echo "Hi"`	Prints text back to you	`Hi`
`clear`	Cleans the screen	(empty screen)
`exit`	Closes the terminal / logs out	—

2.4 Getting Help (You Will Use This Every Day)

Nobody remembers every option of every command. Professionals look things up all the time. Linux has help built in.

a) `--help` — the quick summary

```
student@ubuntu:~$ ls --help
Usage: ls [OPTION]... [FILE]...
List information about the FILEs (the current directory by default).
  -a, --all                  do not ignore entries starting with .
  -l                         use a long listing format
  ...
```

b) `man` — the full manual

```
student@ubuntu:~$ man ls        # Opens the full manual page for ls
```

Inside a man page: use the **arrow keys** or **Space** to scroll, type `/word` and press Enter to search for "word", press `n` for the next match, and **press `q` to quit**.

c) `tldr` — practical examples (beginner's best friend)

"TL;DR" means "Too Long; Didn't Read". It shows only the most common examples.

```
student@ubuntu:~$ sudo apt install tldr -y   # install once
student@ubuntu:~$ tldr --update              # download the examples
student@ubuntu:~$ tldr tar
  Create an archive from files:
      tar cf target.tar file1 file2 file3
  Extract an archive in the current directory:
      tar xf source.tar ...
```

d) Other helpers

Command	Use
`whatis ls`	One-line description of a command
`apropos copy`	Search all manuals for a word (when you forget the command's name)
`type cd`	Tells you what kind of command it is
`info ls`	Longer, book-style documentation for some tools

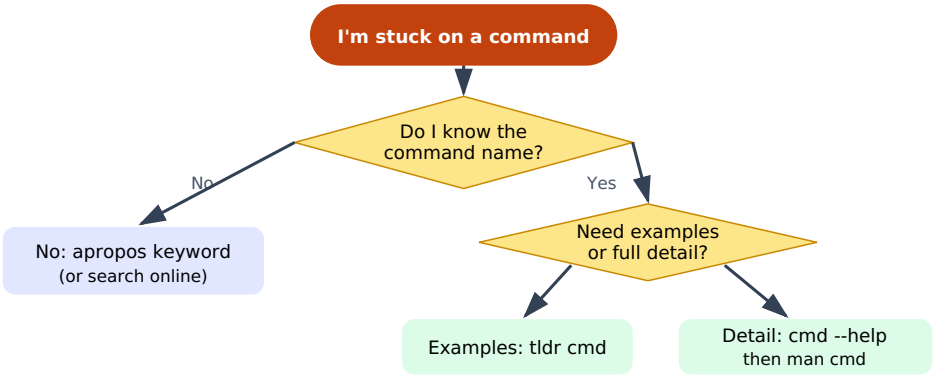


Figure 2.4 — Flowchart: which help tool to use.

2.5 Command History

Bash remembers everything you type. This saves a lot of re-typing.

```
student@ubuntu:~$ history          # show numbered list of past commands
  101  ls -la
  102  cd /etc
  103  cat hostname
student@ubuntu:~$ !103             # run command number 103 again
student@ubuntu:~$ !!               # run the LAST command again
student@ubuntu:~$ sudo !!          # forgot sudo? re-run last command with sudo
student@ubuntu:~$ history | grep apt   # find old commands containing "apt"
```

2.6 Keyboard Shortcuts (Learn These Early!)

Shortcut	What it does
Tab	**Auto-complete** commands, file and folder names. Press Tab twice to see all choices. *The most useful key in Linux.*
↑ / ↓	Move through previous commands
Ctrl + R	Search history: start typing part of an old command
Ctrl + C	**Stop** the running command (your emergency brake)
Ctrl + L	Clear the screen (same as `clear`)
Ctrl + A / **Ctrl + E**	Jump to start / end of the line
Ctrl + U / **Ctrl + K**	Delete everything before / after the cursor
Ctrl + W	Delete the previous word
Ctrl + D	Log out / close the terminal (same as `exit`)
Ctrl + Shift + C / V	Copy / paste inside the terminal (plain Ctrl+C means "stop"!)

Real-Life Example: Tab completion
Tab works like your phone's autocomplete when typing a contact's name. Type `cd /ho` then press Tab, and it becomes `cd /home/`. It saves time *and* stops spelling mistakes. If Tab does nothing, it means no such file exists, which is also useful information!

Teaching Note: Make the learner do one full exercise without touching the mouse and using Tab for every path. Also teach **Ctrl+C** on day one: run `ping google.com` (it never stops by itself) and let them stop it. It removes the fear of "getting stuck".

Practice Lab 2

1. Open the terminal. Run `whoami`, `hostname`, `date` and `uptime`.
2. Run `ls`, then `ls -l`, then `ls -la`. What extra things appear each time?
3. Open `man ls`. Search for `/human`. What does the `-h` option do? Press `q` to leave.
4. Run `ping google.com`. Stop it with **Ctrl + C**.
5. Run `history`. Re-run your `whoami` using `!` and its number.
6. Type `ls /usr/sh` and press **Tab**. Watch it complete.

Chapter 2 Summary

- Terminal = window, Shell = translator, Bash = Ubuntu's default shell.
- The prompt shows *user@host:folder*$. `$` = normal user, `#` = root.
- Commands are built as **command + options + arguments**. Everything is case-sensitive.
- Help: `--help`, `man`, `tldr`, `apropos`.
- **Tab** to complete, **Ctrl+C** to stop, **Ctrl+R** to search history.

Quick Quiz

1. In `ls -l /tmp`, which part is the option and which is the argument?
2. How do you exit a man page?
3. What does `sudo !!` do?
4. A command is running forever. How do you stop it?
5. Your prompt ends with `#`. What does it mean?

Answers: 1) `-l` is the option, `/tmp` is the argument. 2) Press `q`. 3) Runs the last command again with sudo. 4) Ctrl + C. 5) You are logged in as root.

CHAPTER 3

Ajay Upadhyay · DevOps Mentor

The Linux Filesystem

Where everything lives, and how to move around without getting lost.

In this chapter you will learn:

- How Linux arranges files in one big tree
- What each important top-level folder is for
- How to move around: `pwd`, `cd`, `ls`
- Absolute vs relative paths, and the special symbols `~ . .. -`

3.1 One Big Tree

On Windows you have separate drives: `C:\`, `D:\`. Linux is different. **Everything starts from one single folder called root, written as a slash:** `/`. Every file, every folder, even USB drives and extra hard disks, live somewhere under `/`.

Note: In Linux, a "folder" is usually called a **directory**. Same thing.

> **Real-Life Example: An Upside-Down Tree**
> Picture a tree turned upside down. The **root** (`/`) is at the top. **Branches** (directories) come out of it, and branches have smaller branches. The **leaves** are files. To reach any leaf, you start from the root and follow branches down.

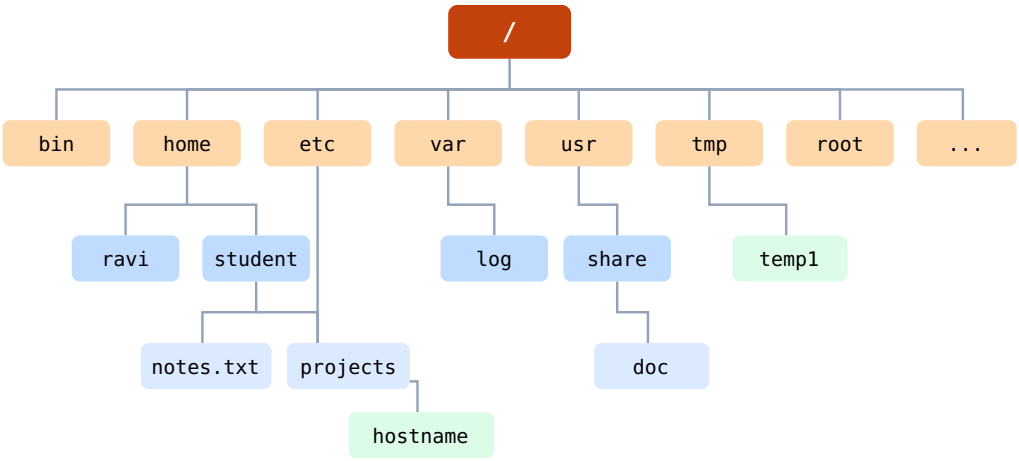


*Figure 3.1 — The Linux directory tree. The full path to notes.txt is **/home/student/notes.txt**.*

3.2 The Important Top-Level Directories

You don't need to memorise all of these today, but you will meet every one of them in DevOps work.

Directory	What is inside	Real-life comparison / example
`/`	Root: the start of everything	The main gate of a building
`/home`	Personal folders for each user: `/home/ravi`, `/home/student`	Each family's private flat in an apartment block
`/root`	Home folder of the root (admin) user	The building owner's office
`/etc`	**Configuration files** for the system and apps	The rule book. e.g. `/etc/nginx/nginx.conf`, `/etc/hosts`
`/var`	**Variable** data that keeps changing: logs, caches, websites	A diary. e.g. logs in `/var/log`, websites in `/var/www`
`/usr`	Installed programs and their files (most software lives here)	The shared library of the building
`/bin`, `/sbin`	Basic commands (`ls`, `cp`). On modern Ubuntu, these are shortcuts to `/usr/bin` and `/usr/sbin`.	The everyday toolbox
`/opt`	Optional, extra software installed by you or a vendor	Extra furniture you bought separately
`/tmp`	Temporary files. Usually **cleared on reboot**.	A whiteboard that gets wiped every night
`/dev`	Devices shown as files: disks (`/dev/sda`), USB, terminals	Labels on every machine in the building
`/proc`	Live information about running programs and the kernel. Not real files on disk.	A live CCTV screen. Try `cat /proc/cpuinfo`
`/sys`	Live information about hardware and drivers	The electrical control panel
`/boot`	Files needed to start the computer, including the kernel	The ignition key
`/mnt`, `/media`	Where extra disks and USB drives get attached	Parking spots for visitors
`/lib`	Shared library files that programs need	Spare parts used by many tools

> **Tip: The "Big Three" for DevOps.** If you remember only three, remember these: `/etc` (configs: where you change settings), `/var/log` (logs: where you find out what went wrong), and `/home` (users' files).

> **Tip: "Everything is a file".** In Linux, almost everything is treated as a file: documents, folders, hard disks (`/dev/sda`), even running programs (`/proc`). This simple idea makes Linux very powerful, because the same tools work on all of them.

3.3 Moving Around: pwd, cd, ls

pwd — "Where am I?"

`pwd` means **P**rint **W**orking **D**irectory. It shows the folder you are currently in.

```
student@ubuntu:~$ pwd
/home/student
```

cd — "Go there"

`cd` means **C**hange **D**irectory.

```
student@ubuntu:~$ cd /etc            # go to /etc
student@ubuntu:/etc$ pwd
/etc
student@ubuntu:/etc$ cd ..             # go UP one level (to /)
student@ubuntu:/$ cd ~               # go to your home folder
student@ubuntu:~$ cd /var/log
student@ubuntu:/var/log$ cd -       # go BACK to the previous folder
/home/student
student@ubuntu:~$ cd                 # cd alone also takes you home
```

ls — "What is here?"

```
student@ubuntu:~$ ls                # simple list
Desktop  Documents  Downloads  notes.txt  projects
student@ubuntu:~$ ls -l             # long list: details
drwxr-xr-x 2 student student 4096 Oct 13 10:00 Documents
-rw-r--r-- 1 student student  220 Oct 13 10:05 notes.txt
student@ubuntu:~$ ls -a             # show hidden files too (names starting with a dot)
.  ..  .bashrc  .profile  Desktop  Documents  notes.txt
student@ubuntu:~$ ls -lh /var/log   # long list with human sizes (K, M, G)
student@ubuntu:~$ ls -lt            # sort by time, newest first
```

In `ls -l`, the first letter tells you the type: `d` = directory, `-` = normal file, `l` = link. (We will read the rest of that line in Chapter 7.)

> **Tip: Hidden files.** Any file whose name starts with a dot (like `.bashrc`) is hidden. They are usually settings files. Use `ls -a` to see them.

3.4 Absolute vs Relative Paths

A **path** is the address of a file. There are two ways to write an address:

Type	Rule	Example
Absolute path	Always **starts with** `/`. The full address from the root. Works from anywhere.	`/home/student/projects/app.py`
Relative path	Does **not** start with `/`. The address starting from *where you are now*.	`projects/app.py` (if you are in `/home/student`)

> **Real-Life Example: Giving Directions**
> **Absolute** is like a full postal address: "Flat 12, Green Tower, MG Road, Noida, UP, India." Anyone anywhere can find it.
> **Relative** is like telling someone who is already standing in your building: "Go up two floors, second door on the left." It only works from where they are standing.

Special Path Symbols

Symbol	Meaning	Example
`/`	The root directory	`cd /`
`~`	Your home directory	`cd ~/Documents` = `/home/student/Documents`
`.`	The current directory (here)	`./script.sh` = "run script.sh from here"
`..`	The parent directory (one level up)	`cd ../..` = go up two levels
`-`	The previous directory you were in	`cd -`

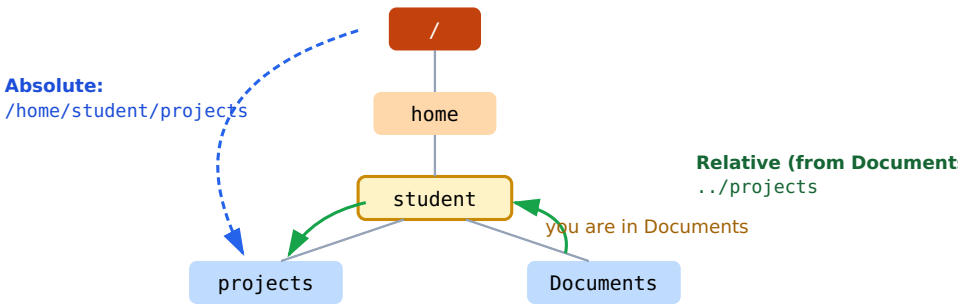


Figure 3.2 — Two ways to reach the same "projects" folder.

> **Teaching Note:** Draw the tree from Figure 3.1 on paper. Put a coin on "Documents" as "you are here". Ask the learner to write both the absolute and the relative path to `/var/log`. (Absolute: `/var/log`; relative from `/home/student/Documents`: `../../../var/log`.) Then let them prove it with `cd` and `pwd`.

Practice Lab 3

1. Run `pwd`. Where are you?
2. Go to `/etc` and list its contents with `ls`. Find the file `hostname`.
3. Go to `/var/log` and run `ls -lh`. Which file is the biggest?
4. Use `cd -` to jump back, then `cd` alone to go home.
5. From your home folder, go to `/tmp` using a *relative* path. (Hint: `../../tmp`)
6. Run `ls -a ~`. How many hidden files do you see?
7. Run `cat /proc/cpuinfo | head` to see your CPU. Remember: this "file" doesn't really exist on disk!

Chapter 3 Summary

- Linux has one tree starting at `/` (root). Folders are called directories.
- Key folders: `/etc` (config), `/var/log` (logs), `/home` (users), `/tmp` (temporary).
- `pwd` = where am I, `cd` = go there, `ls` = what is here.
- Absolute paths start with `/`. Relative paths start from where you are.
- `~` home, `.` here, `..` up one, `-` previous.

Quick Quiz

1. Where would you look for nginx's configuration file?
2. Where would you look for system logs?
3. Is `Documents/report.txt` an absolute or relative path?
4. You are in `/home/student/projects`. Where does `cd ..` take you?
5. How do you see hidden files?

Answers: 1) `/etc/nginx/` 2) `/var/log` 3) Relative (no leading `/`) 4) `/home/student` 5) `ls -a`

CHAPTER 4

Ajay Upadhyay · DevOps Mentor

Managing Files and Directories

Create, copy, move, rename and delete, plus wildcards and links.

In this chapter you will learn:

- Creating folders and files: `mkdir` , `touch`
- Copying, moving, renaming: `cp` , `mv`
- Deleting safely: `rm` , `rmdir`
- Seeing structure: `tree`
- Wildcards: `* ? [ ]`
- Hard links vs soft links

4.1 Creating Directories: mkdir

`mkdir` = **Mak**e **Dir**ectory.

```
student@ubuntu:~$ mkdir practice                # create one folder
student@ubuntu:~$ mkdir photos music videos      # create three at once
student@ubuntu:~$ mkdir -p website/css/images   # -p creates the whole chain of parents
student@ubuntu:~$ mkdir -v test                 # -v (verbose) tells you what it did
mkdir: created directory 'test'
```

> **Tip:** Without `-p` , `mkdir website/css/images` fails if `website` doesn't exist yet. With `-p` , it creates all missing folders, and gives no error if they already exist. That is why scripts almost always use `mkdir -p` .

4.2 Creating Files: touch

```
student@ubuntu:~$ touch notes.txt               # create an empty file
student@ubuntu:~$ touch a.txt b.txt c.txt       # create several
student@ubuntu:~$ echo "Hello Linux" > hello.txt   # create a file WITH text inside
```

If the file already exists, `touch` doesn't erase it; it only updates its "last modified" time.

4.3 Seeing the Structure: tree

```
student@ubuntu:~$ sudo apt install tree -y
student@ubuntu:~$ tree website
website
└── css
    └── images

2 directories, 0 files
student@ubuntu:~$ tree -L 1 /        # only show 1 level deep
```

4.4 Copying: cp

Pattern: `cp SOURCE DESTINATION` (from → to).

```
student@ubuntu:~$ cp notes.txt notes_backup.txt     # copy a file with a new name
student@ubuntu:~$ cp notes.txt practice/            # copy into a folder (same name)
student@ubuntu:~$ cp a.txt b.txt practice/          # copy many files into a folder
student@ubuntu:~$ cp -r website website_backup      # -r copies a FOLDER and everything inside
student@ubuntu:~$ cp -i notes.txt practice/         # -i asks before overwriting
cp: overwrite 'practice/notes.txt'? n
student@ubuntu:~$ cp -p config.yml /tmp/            # -p keeps original date & permissions
```

Careful! To copy a **directory**, you must use `-r` (recursive, meaning "go inside and copy everything"). Without it you'll get: `cp: -r not specified; omitting directory`.

4.5 Moving and Renaming: mv

Linux has no separate "rename" command. `mv` **does both**: if you "move" a file into the same folder with a new name, that's a rename.

```
student@ubuntu:~$ mv notes.txt mynotes.txt         # RENAME
student@ubuntu:~$ mv mynotes.txt practice/          # MOVE into a folder
student@ubuntu:~$ mv practice/mynotes.txt ~/final.txt  # move AND rename together
student@ubuntu:~$ mv website site                   # rename a folder (no -r needed)
student@ubuntu:~$ mv -i a.txt practice/            # ask before overwriting
```

Real-Life Example: cp vs mv
cp is like a **photocopy machine**: the original stays, and you get a second copy.
mv is like **carrying a box to another room**: it is gone from the old place. If you just change the label on the box without moving it, that is a rename.

4.6 Deleting: rm and rmdir

```
student@ubuntu:~$ rm a.txt                # delete a file
student@ubuntu:~$ rm -i b.txt             # ask "are you sure?" first
rm: remove regular empty file 'b.txt'? y
student@ubuntu:~$ rmdir test              # delete an EMPTY folder only
student@ubuntu:~$ rm -r website_backup    # delete a folder and everything inside
student@ubuntu:~$ rm -rf old_folder       # -f = force, no questions, no errors
```

Careful! There is NO Recycle Bin in the terminal. Once you `rm` something, it is gone forever. Before running `rm -rf`, **always** run `pwd` and `ls` to double-check where you are and what you are deleting. Never run `sudo rm -rf /` or `rm -rf *` in a place you are not sure about: it can wipe the entire system.

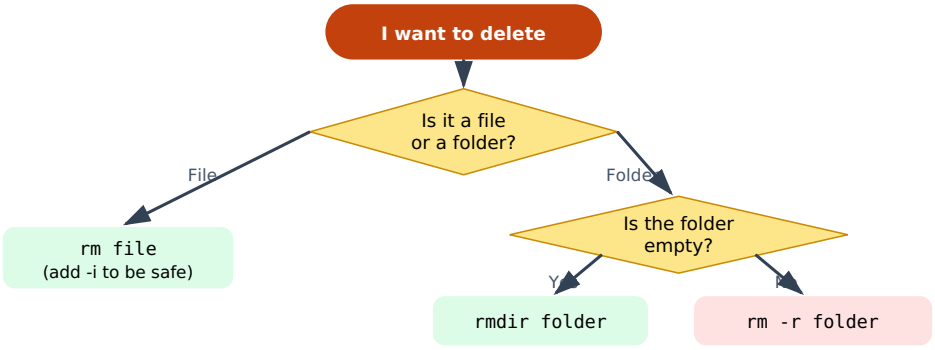


Figure 4.1 — Flowchart: choosing the right delete command. Always check with pwd and ls first!

4.7 Wildcards (Globbing): Work on Many Files at Once

Wildcards are special characters that match many file names. The shell expands them before the command runs.

Wildcard	Matches	Example
`*`	Any number of any characters (even zero)	`ls *.txt` → all files ending in .txt
`?`	Exactly one character	`ls file?.txt` → file1.txt, fileA.txt (not file10.txt)
`[abc]`	One character from the list	`ls file[123].txt` → file1, file2, file3
`[0-9]`	One character in a range	`ls report[0-9].pdf`
`{a,b}`	Each word in the list (brace expansion)	`mkdir {dev,test,prod}` → makes 3 folders

```
student@ubuntu:~$ touch file1.txt file2.txt file3.log photo.jpg
student@ubuntu:~$ ls *.txt
file1.txt  file2.txt
student@ubuntu:~$ ls file*
file1.txt  file2.txt  file3.log
student@ubuntu:~$ mkdir backup && cp *.txt backup/      # copy all .txt files
student@ubuntu:~$ touch day{1..5}.log                    # creates day1.log ... day5.log
student@ubuntu:~$ rm *.log                               # delete every .log here (be careful!)
```

Tip: Test before you delete. Before running `rm *.log`, first run `ls *.log` with the same pattern. You will see exactly what will be deleted.

4.8 Links: Hard Links and Soft Links

A **link** lets one file be reached from more than one place, without making a copy. First, one small idea:

Every file on disk has an **inode**: a number that points to where the real data is stored. A filename is just a **label** attached to an inode.

Soft Link (Symbolic Link / Symlink)

A soft link is a **shortcut** that points to another file's *name*. Exactly like a desktop shortcut on Windows. If the original file is deleted, the shortcut breaks.

Hard Link

A hard link is a **second name** for the *same data* (same inode). Both names are equal. If you delete one, the data is still there through the other name.

Real-Life Example: Contacts in your Phone
Hard link: One person has two names in your phone, "Ravi" and "Ravi Office", both saved with the *same phone number*. Delete "Ravi" and you can still call him through "Ravi Office".
Soft link: A sticky note that says "for Ravi's number, look under 'Ravi' in contacts". If you delete the "Ravi" contact, the sticky note is useless (a broken link).

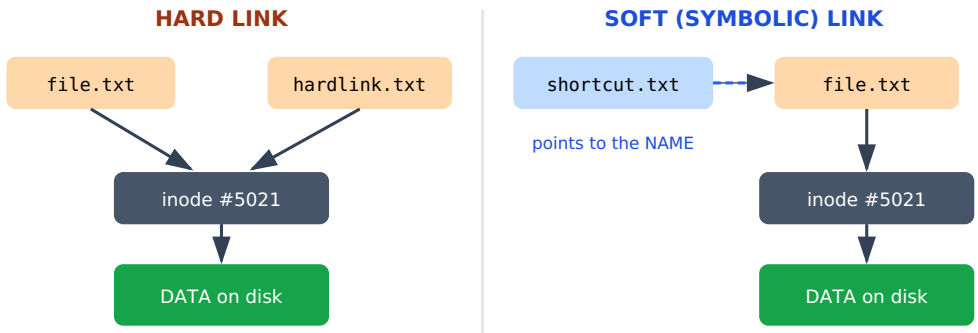


Figure 4.2 — A hard link is a second name for the same inode. A soft link points to a name.

```
student@ubuntu:~$ echo "important data" > original.txt
student@ubuntu:~$ ln original.txt hard.txt          # create a HARD link
student@ubuntu:~$ ln -s original.txt soft.txt       # create a SOFT link (-s = symbolic)
student@ubuntu:~$ ls -li                             # -i shows inode numbers
5021 -rw-r--r-- 2 student student 15 Oct 13 hard.txt
5021 -rw-r--r-- 2 student student 15 Oct 13 original.txt
5099 lrwxrwxrwx 1 student student 12 Oct 13 soft.txt -> original.txt
student@ubuntu:~$ rm original.txt                    # now delete the original...
student@ubuntu:~$ cat hard.txt
important data                                      # still works!
student@ubuntu:~$ cat soft.txt
cat: soft.txt: No such file or directory          # broken shortcut
```

Feature	Hard Link	Soft Link
Command	`ln source link`	`ln -s source link`
Same inode number?	Yes	No (it has its own)
Works on directories?	No	Yes
Works across different disks?	No	Yes
If original is deleted	Data still available	Link breaks
Used most in real life?	Rarely	**Very often**

DevOps Use: Soft links are everywhere. Nginx turns websites on by linking a config from `/etc/nginx/sites-available/` into `/etc/nginx/sites-enabled/`. Deployment tools point a `current` link at the newest release folder, so switching versions is just changing one link: `ln -sfn /app/releases/v2 /app/current`.

Practice Lab 4

1. In your home folder create `lab4/`. Inside it, create `docs`, `images` and `scripts/old` with one `mkdir -p` command.
2. Create `report1.txt`, `report2.txt` and `notes.md` inside `docs`.
3. Copy all the `.txt` files to `images` using a wildcard. Then move them back using `mv`. What error do you get, and why?
4. Rename `notes.md` to `readme.md`.
5. Run `tree lab4` to check your work.
6. Create a soft link called `latest` that points to `docs/report2.txt`. Open it with `cat latest`.
7. Use `ls -li` to compare inode numbers of a hard link and a soft link.
8. Finally, delete `lab4` completely. (Check with `pwd` first!)

Chapter 4 Summary

- `mkdir -p` makes folders, `touch` makes empty files.
- `cp` copies (needs `-r` for folders), `mv` moves *and* renames.
- `rm` deletes forever. Use `-i` while learning; check with `pwd` before `rm -rf`.
- Wildcards: `*` anything, `?` one character, `[ ]` one from a set, `{ }` a list.
- Soft link (`ln -s`) = shortcut. Hard link (`ln`) = second name for the same data.

Quick Quiz

1. How do you rename `old.txt` to `new.txt`?
2. Why does `cp myfolder backup` fail?
3. What does `ls ?.txt` match: `a.txt`, `ab.txt`, or both?
4. You delete a file that has a soft link pointing to it. What happens to the link?
5. Which command makes `dev`, `test` and `prod` folders in one go?

Answers: 1) `mv old.txt new.txt` 2) Folders need `cp -r` 3) Only `a.txt` 4) It becomes broken 5) `mkdir dev test prod` or `mkdir {dev,test,prod}`

CHAPTER 5

Ajay Upadhyay · DevOps Mentor

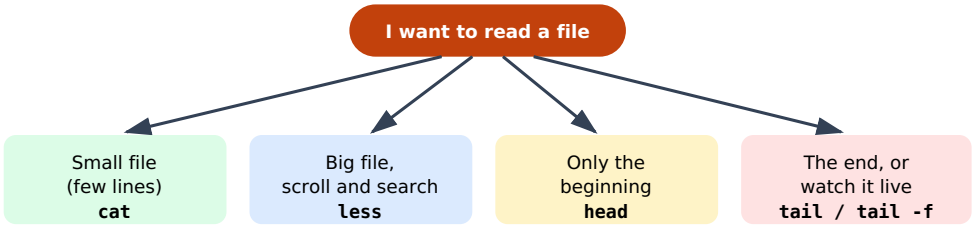
Viewing and Editing Files

Reading files of any size, watching logs live, and editing with nano and vim.

In this chapter you will learn:

- Which viewing command to use for which file: `cat`, `less`, `more`, `head`, `tail`
- How to watch a log file live with `tail -f`
- Editing with **nano** (easy)
- Editing with **vim** (powerful, on every server)

5.1 Viewing Files



Rule of thumb: if you don't know how big it is, use less. It is always safe.

Figure 5.1 — Choosing the right viewing command.

cat — print the whole file

```
student@ubuntu:~$ cat /etc/hostname
ubuntu
student@ubuntu:~$ cat -n /etc/hosts          # -n adds line numbers
     1  127.0.0.1 localhost
     2  127.0.1.1 ubuntu
student@ubuntu:~$ cat a.txt b.txt > all.txt   # join two files into one ("concatenate")
```

Careful! Don't `cat` a huge file (like a 2 GB log). Thousands of lines will fly across your screen. Use `less` instead.

less — scroll through big files

```
student@ubuntu:~$ less /var/log/syslog
```

Key inside less	Action
↑ ↓ or j / k	Move one line
Space / b	Next page / previous page
g / G	Go to the start / end of file
/error	Search forward for "error" (then `n` = next, `N` = previous)
q	**Quit**

`more` is an older, simpler version of `less`. It can only scroll forward. Remember the saying: **"less is more"** (less can do more!).

head and tail — the beginning and the end

```
student@ubuntu:~$ head /etc/passwd          # first 10 lines (default)
student@ubuntu:~$ head -n 3 /etc/passwd     # first 3 lines
student@ubuntu:~$ tail /var/log/syslog      # last 10 lines
student@ubuntu:~$ tail -n 50 /var/log/syslog # last 50 lines
```

tail -f — watch a file live (DevOps superpower!)

`-f` means "follow". The command keeps running and shows new lines the moment they are written. This is how you watch an application while you test it.

```
student@ubuntu:~$ sudo tail -f /var/log/nginx/access.log
192.168.1.5 - - [13/Oct/2026:10:41:02] "GET / HTTP/1.1" 200 612
192.168.1.5 - - [13/Oct/2026:10:41:09] "GET /login HTTP/1.1" 404 153
... new lines appear as visitors open the website. Press Ctrl+C to stop.
```

> **Real-Life Example:** `head` is reading the first page of a newspaper. `tail` is reading the last page. `tail -f` is like watching a live cricket score ticker that updates by itself.

Bonus: a few more useful viewers

Command	What it does
`wc -l file`	Counts lines (also words with `-w`, characters with `-c`)
`file photo.jpg`	Tells you what type a file really is
`grep error app.log`	Shows only lines containing "error" (covered fully at Level 2)
`diff a.txt b.txt`	Shows differences between two files

5.2 Editing with nano (The Easy Editor)

nano works like Notepad. You open it, type, save, quit. The shortcuts are always shown at the bottom of the screen. The `^` symbol means the **Ctrl** key.

```
student@ubuntu:~$ nano shopping.txt            # opens (or creates) the file
student@ubuntu:~$ sudo nano /etc/hosts          # system files need sudo
```

Keys	Action
Ctrl + O then Enter	Save ("Write Out")
Ctrl + X	Exit (asks to save if there are changes: press Y then Enter)
Ctrl + W	Search ("Where is")
Ctrl + K / Ctrl + U	Cut a whole line / paste it
Ctrl + _	Go to a line number

5.3 Editing with vim (The Pro Editor)

vim is installed on almost every Linux server in the world, even tiny ones where nano is missing. It looks scary at first because it has **modes**. Once you understand modes, it becomes simple.

> **Real-Life Example: A Pen with a Cap**
> **Normal mode** is the pen with its **cap on**: you can move it around the page, point at things, but not write. **Insert mode** is the **cap off**: now whatever you do writes on the page. Pressing `i` takes the cap off. Pressing `Esc` puts it back on. **Command mode** (`:`) is like talking to your assistant: "save this", "close the book".

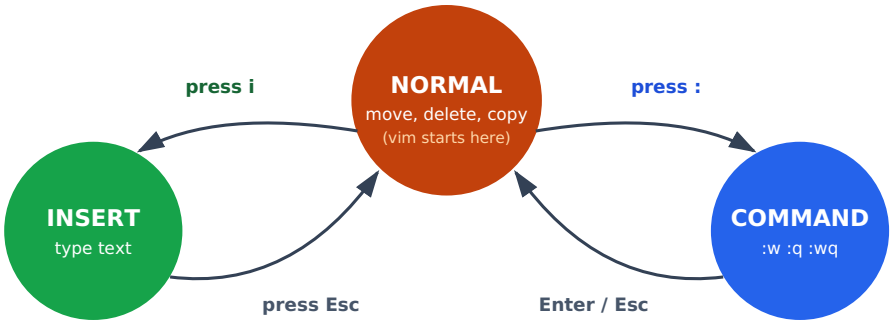


*Figure 5.2 — The three vim modes. When confused, press **Esc** to get back to Normal mode.*

Your First vim Session (step by step)

```
student@ubuntu:~$ vim hello.txt     # 1. open the file (you are in NORMAL mode)
                                    # 2. press  i      -> "-- INSERT --" shows at the bottom
                                    # 3. type:  Hello from vim!
                                    # 4. press  Esc    -> back to NORMAL mode
                                    # 5. type   :wq    and press Enter  -> save and quit
student@ubuntu:~$ cat hello.txt
Hello from vim!
```

The Most Important vim Commands

Save and quit (Command mode)	
`:w`	Save
`:q`	Quit
`:wq` or `:x`	Save and quit
`:q!`	**Quit without saving** (escape hatch!)
`:set number`	Show line numbers
`:25`	Jump to line 25

Entering Insert mode	
`i`	Insert before cursor
`a`	Insert after cursor
`o`	New line below
`O`	New line above

Normal mode: moving	
`h j k l`	Left, down, up, right
`w` / `b`	Next / previous word
`0` / `$`	Start / end of line
`gg` / `G`	Top / bottom of file

Normal mode: editing	
`x`	Delete one character
`dd`	Delete (cut) a whole line
`yy`	Copy ("yank") a line
`p`	Paste below
`u`	**Undo**
`Ctrl + r`	Redo
`/word`	Search for "word" (`n` = next)

> **Tip: Practise vim properly.** Run the built-in tutorial: type `vimtutor` in the terminal. It takes about 30 minutes and is the best way to learn vim.

> **Teaching Note:** Every beginner gets "trapped" in vim once. Teach the escape first: **Esc, then `:q!` , then Enter**. Make them practise escaping three times before they learn anything else. After that they won't be afraid.

Practice Lab 5

1. Run `cat /etc/os-release`. What is the `PRETTY_NAME`?
2. Open `/etc/services` with `less`. Search for `/ssh`. Which port number does SSH use? Quit with `q`.
3. Show the first 5 lines and the last 5 lines of `/etc/passwd`.
4. Count how many lines are in `/etc/passwd` with `wc -l`.
5. Open a second terminal and run `sudo tail -f /var/log/syslog`. In the first terminal run `logger "Hello from my lab"`. Watch your message appear!
6. Using **nano**, create `todo.txt` with 3 tasks. Save and exit.
7. Using **vim**, open `todo.txt`, add a 4th task with `o`, delete the first line with `dd`, then undo with `u`. Save with `:wq`.

Chapter 5 Summary

- `cat` small files, `less` big files, `head` / `tail` start/end.
- `tail -f` watches logs live. Stop with Ctrl + C.
- **nano**: easy. Ctrl+O save, Ctrl+X exit.
- **vim**: modes. `i` to type, `Esc` to stop typing, `:wq` save & quit, `:q!` quit without saving.

Quick Quiz

1. Which command shows the last 20 lines of `app.log` ?
2. You want to watch new errors appear in a log as they happen. Which command?
3. In vim, you are typing text. How do you get back to Normal mode?
4. In vim, how do you quit *without* saving?
5. In nano, what does `^X` mean?

Answers: 1) `tail -n 20 app.log` 2) `tail -f logfile` 3) Press Esc 4) `:q!` 5) Ctrl + X (exit)

CHAPTER 6

Ajay Upadhyay · DevOps Mentor

Users and Groups

Who can log in, how people are organised into teams, and how admin power works with sudo.

In this chapter you will learn:

- Why Linux has users and groups
- The three kinds of users: root, normal, system
- Creating, changing and deleting users and groups
- Where user information is stored: `/etc/passwd`, `/etc/shadow`, `/etc/group`
- Admin power: `sudo`, `su` and `visudo`

6.1 Why Users and Groups?

Linux was built from the start for **many people to share one computer**. Each person gets a **user account** so that their files are private and they cannot break each other's work. A **group** is a team of users who share access to something.

> **Real-Life Example: An Office Building**
> Every employee gets their own **ID card** (a user). Employees are placed in **departments** (groups): Developers, Testers, Finance. The Finance room door opens for anyone in the Finance group. The building **owner** has a master key that opens every door: that's **root**. A trusted manager can **borrow** the master key for a specific job and must sign a register: that's **sudo**.

6.2 Three Kinds of Users

Type	UID number	Description
root	0	The superuser. Can do absolutely anything, including destroying the system. Home is `/root`.
System users	1 – 999	Not real people. Created for services so they run with limited power: `www-data` (web server), `mysql`, `syslog`. They usually cannot log in.
Normal users	1000 and above	Real people like you. The first user you create during install gets UID 1000.

UID = User ID number. **GID** = Group ID number. Linux actually identifies users by number; names are for humans.

```
student@ubuntu:~$ whoami          # your username
student
student@ubuntu:~$ id              # your UID, GID and all your groups
uid=1000(student) gid=1000(student) groups=1000(student),4(adm),27(sudo)
student@ubuntu:~$ groups          # just the group names
student adm sudo
student@ubuntu:~$ who             # who is logged in right now
```

6.3 Where User Information Lives

/etc/passwd — the list of all users

Despite the name, it does **not** contain passwords anymore. Each line is one user, with 7 parts separated by colons:

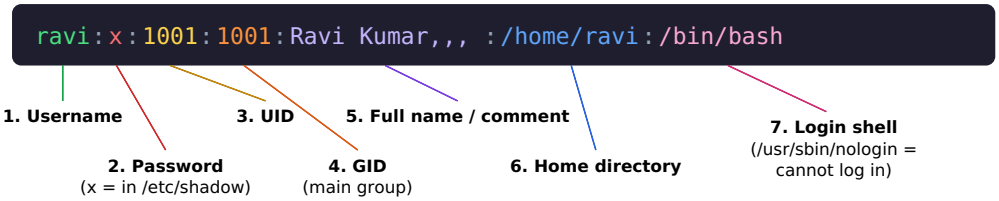


Figure 6.1 — The 7 fields of a line in /etc/passwd.

/etc/shadow — the secret passwords

Stores the **encrypted (hashed)** passwords and password-expiry rules. Only root can read it. You'll never see a real password there, only a scrambled hash like `$y$j9T$...`.

/etc/group — the list of groups

```
student@ubuntu:~$ grep developers /etc/group
developers:x:1005:ravi,priya          # groupname : password : GID : members
```

Primary vs Secondary Groups

Every user has **one primary group** (by default, a group with the same name as the user). New files they create belong to this group. A user can also be in **many secondary groups** to get extra access (like `sudo` or `docker`).

6.4 Creating Users

Ubuntu has two commands. Beginners should prefer `adduser`.

Command	Style
`adduser`	**Friendly.** Asks questions, creates the home folder, sets the password. Ubuntu/Debian only.
`useradd`	**Low-level.** Works on all Linux. You must give options yourself. Used in scripts.

```
student@ubuntu:~$ sudo adduser ravi
Adding user `ravi' ...
Creating home directory `/home/ravi' ...
New password: ********
Retype new password: ********
Full Name []: Ravi Kumar
Is the information correct? [Y/n] Y

# Same result with useradd (for scripts):
student@ubuntu:~$ sudo useradd -m -s /bin/bash -c "Priya Sharma" priya
#   -m  create home folder   -s  login shell   -c  comment / full name
student@ubuntu:~$ sudo passwd priya          # set her password
```

> **Careful!** `useradd priya` *without* `-m` creates a user with **no home folder**, and without `-s /bin/bash` she may get a very basic shell. This is a common beginner mistake.

6.5 Changing and Deleting Users

```
student@ubuntu:~$ sudo passwd ravi                    # change ravi's password
student@ubuntu:~$ passwd                              # change YOUR own password
student@ubuntu:~$ sudo usermod -aG sudo ravi          # give ravi admin (sudo) power
student@ubuntu:~$ sudo usermod -aG docker ravi        # let ravi use Docker
student@ubuntu:~$ sudo usermod -s /bin/zsh ravi       # change his shell
student@ubuntu:~$ sudo usermod -L ravi                # Lock the account (can't log in)
student@ubuntu:~$ sudo usermod -U ravi                # Unlock it
student@ubuntu:~$ sudo chage -l ravi                  # see password expiry info
student@ubuntu:~$ sudo deluser ravi                   # delete user (keeps home folder)
student@ubuntu:~$ sudo deluser --remove-home ravi     # delete user AND home folder
student@ubuntu:~$ sudo userdel -r ravi                # same, low-level version
```

Careful! The famous `-aG` mistake. Always write `usermod -aG group user`. The `-a` means **append** (add to existing groups). If you forget `-a` and write only `-G`, the user is **removed from all other groups**, including sudo. You could lock yourself out of admin access!

Tip: After adding a user to a group, the change only applies at their **next login**. Log out and back in (or run `newgrp docker`) to use it right away.

6.6 Managing Groups

```
student@ubuntu:~$ sudo groupadd developers              # create a group
student@ubuntu:~$ sudo usermod -aG developers ravi      # add a user to it
student@ubuntu:~$ sudo gpasswd -a priya developers      # another way to add
student@ubuntu:~$ sudo gpasswd -d priya developers      # remove priya from the group
student@ubuntu:~$ getent group developers               # see members
developers:x:1005:ravi
student@ubuntu:~$ sudo groupmod -n devteam developers   # rename the group
student@ubuntu:~$ sudo groupdel devteam                 # delete the group
```

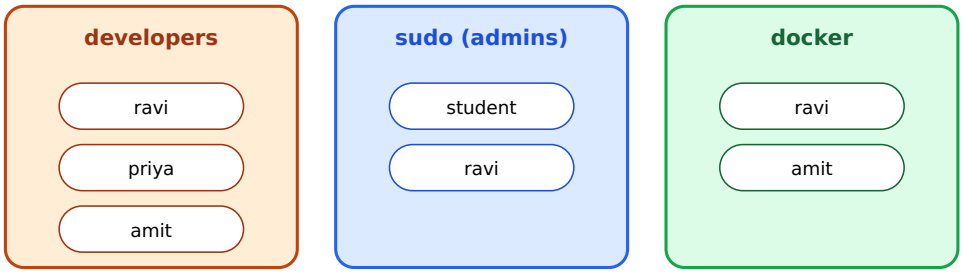


Figure 6.2 — One user can be in many groups. Ravi is a developer, an admin, and can use Docker.

6.7 Becoming an Administrator: sudo and su

On Ubuntu, the root account is **locked by default** for safety. Instead, trusted users run single commands with admin power using `sudo` ("**s**uper**u**ser **do**").

```
student@ubuntu:~$ apt update
E: Could not open lock file ... Permission denied       # normal users can't do this
student@ubuntu:~$ sudo apt update
[sudo] password for student:                            # type YOUR password (nothing shows - that's normal)
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease ...
```

Command	What it does
`sudo command`	Run **one** command as root. Asks for *your* password. Logged in `/var/log/auth.log` .
`sudo -i`	Open a full root shell (prompt changes to `#`). Type `exit` to return.
`su - ravi`	**S**witch **u**ser to ravi. Asks for *ravi's* password.
`sudo -u ravi command`	Run a command as ravi (not root).
`sudo -l`	List what you are allowed to run with sudo.

Who is allowed to use sudo? The sudoers file

The rules live in `/etc/sudoers` . On Ubuntu, any member of the `sudo` group gets full admin rights. To give someone admin power, just add them: `sudo usermod -aG sudo ravi` .

For custom rules, **never edit sudoers with a normal editor**. Always use `visudo` , which checks for mistakes before saving. A broken sudoers file can stop *everyone* from using sudo.

```
student@ubuntu:~$ sudo visudo -f /etc/sudoers.d/deploy
# Allow user 'deploy' to restart nginx only, without a password:
deploy ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart nginx
```

> **DevOps Best Practice:** Never log in directly as root on a server. Use a normal user + sudo. Give people (and automation accounts) only the exact power they need. This is called the **Principle of Least Privilege**.

Practice Lab 6

1. Run `id` . Are you in the `sudo` group?
2. Create two users with `adduser` : `alice` and `bob` .
3. Create a group `webteam` and add both users to it with `usermod -aG` .
4. Check with `getent group webteam` and `id alice` .
5. Look at the last 3 lines of `/etc/passwd` . Find alice's UID and home folder.
6. Switch to alice with `su - alice` . Run `whoami` and `pwd` . Try `sudo apt update` — why does it fail? Type `exit` .
7. Give alice sudo access. Switch to her again and try once more.
8. Run `sudo tail /var/log/auth.log` and find alice's sudo usage recorded there.
9. Clean up: delete bob with his home folder, and delete the `webteam` group.

Chapter 6 Summary

- Users keep work private; groups share access among a team.
- root = UID 0. System users = 1–999. Real people = 1000+.
- Info lives in `/etc/passwd` , `/etc/shadow` (hashed passwords), `/etc/group` .
- `adduser` (friendly) / `useradd -m -s /bin/bash` (scripts). `usermod -aG` to add to groups — never forget `-a` !
- Use `sudo` for admin tasks, edit rules only with `visudo` .

Quick Quiz

1. What is root's UID?
2. Which file stores the hashed passwords?
3. What is wrong with `sudo usermod -G docker ravi` ?
4. Why use `visudo` instead of `nano /etc/sudoers` ?
5. When you type your sudo password, nothing appears on screen. Is something broken?

Answers: 1) 0 2) `/etc/shadow` 3) Missing `-a` ; ravi gets removed from all his other groups 4) visudo checks syntax so you don't break sudo 5) No, it's hidden on purpose for security.

CHAPTER 7

Ajay Upadhyay · DevOps Mentor

File Permissions and Ownership

Controlling exactly who can read, change or run every file. The heart of Linux security.

In this chapter you will learn:

- Reading a permission line like `-rwxr-xr--`
- What r, w, x mean for files and for folders
- Changing permissions with `chmod` (letters and numbers like 755)
- Changing owners with `chown` and `chgrp`
- Default permissions with `umask`
- Special permissions: SUID, SGID, Sticky bit
- ACLs for fine-grained control

7.1 The Three Permissions

Letter	Name	On a FILE it means	On a DIRECTORY it means
r	Read	Can open and read the content	Can list the files inside (`ls`)
w	Write	Can change the content	Can create, delete, rename files inside
x	Execute	Can run it as a program/script	Can enter it (`cd`)
-	None	That permission is not given	

7.2 The Three Kinds of People

Every file has permissions for three groups of people:

Letter	Who	Meaning
u	User (owner)	The person who owns the file
g	Group	Members of the file's group
o	Others	Everyone else on the system
a	All	u + g + o together

Real-Life Example: Your Diary

You (the **owner**) can read it, write in it. Your **family** (the group) may read it but not write in it. **Strangers** (others) can't even open it. That diary would have permissions `rw-r-----`.

7.3 Reading a Permission Line

```
student@ubuntu:~$ ls -l deploy.sh
-rwxr-xr-- 1 ravi developers 2048 Oct 13 11:00 deploy.sh
```

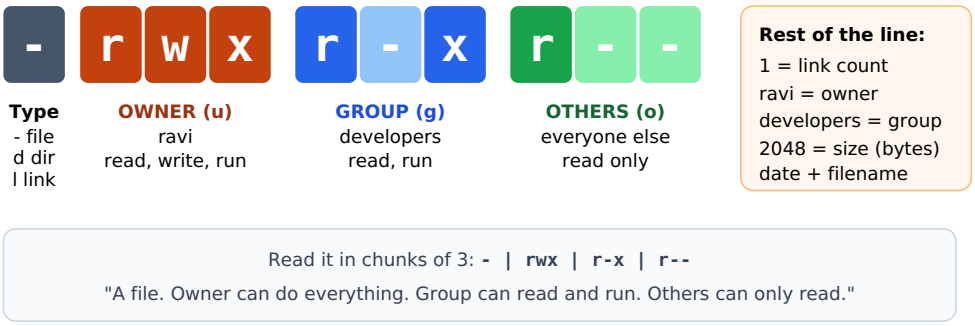


Figure 7.1 — Breaking down the permission string -rwxr-xr--.

7.4 Changing Permissions with chmod (Letter Method)

`chmod` = **ch**ange **mod**e. Formula: **who + action + permission**.

Who	Action	Permission
`u g o a`	`+` add `-` remove `=` set exactly	`r w x`

```
student@ubuntu:~$ chmod u+x deploy.sh        # give the owner permission to run it
student@ubuntu:~$ chmod g+w notes.txt        # let the group edit it
student@ubuntu:~$ chmod o-r secret.txt       # stop others from reading it
student@ubuntu:~$ chmod a+r public.txt       # everyone can read
student@ubuntu:~$ chmod u=rw,g=r,o= diary.txt  # set exactly: rw-r-----
student@ubuntu:~$ chmod -R g+w project/      # -R = apply to folder and everything inside
```

Real example: why you need execute permission

```
student@ubuntu:~$ echo 'echo "Server is healthy"' > check.sh
student@ubuntu:~$ ./check.sh
bash: ./check.sh: Permission denied          # no x permission yet!
student@ubuntu:~$ chmod +x check.sh
student@ubuntu:~$ ./check.sh
Server is healthy
```

7.5 The Number Method (755, 644, 600...)

This is what you'll see most in real jobs. Each permission has a number value. **Add them up** for each person.

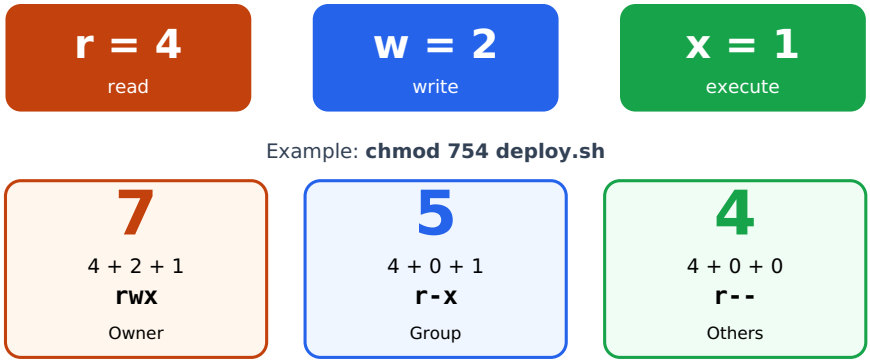


Figure 7.2 — How the number method works. Three digits: owner, group, others.

Number	Letters	Meaning
7	`rwx`	Everything (4+2+1)
6	`rw-`	Read and write (4+2)
5	`r-x`	Read and execute (4+1)
4	`r--`	Read only
0	`---`	Nothing

Common Permission Numbers to Remember

Number	Looks like	Typical use
755	`rwxr-xr-x`	Scripts, programs and **directories**. Everyone can use; only owner can change.
644	`rw-r--r--`	**Normal files**: web pages, config files. Everyone reads; owner edits.
600	`rw-------`	**Private files**: SSH private keys, passwords. Only owner.
700	`rwx------`	Private folders, like `~/.ssh`.
775	`rwxrwxr-x`	Shared team folders.
777	`rwxrwxrwx`	**Everyone can do everything. Almost never use this!**

```
student@ubuntu:~$ chmod 755 backup.sh
student@ubuntu:~$ chmod 644 index.html
student@ubuntu:~$ chmod 600 ~/.ssh/id_rsa       # SSH refuses keys with looser permissions!
student@ubuntu:~$ stat -c "%a %A %n" backup.sh  # show permissions as number AND letters
755 -rwxr-xr-x backup.sh
```

> **Careful! Never fix problems with chmod 777.** When something says "Permission denied", beginners often run `chmod -R 777` to "make it work". This lets *anyone*, including attackers, change your files. Find the correct owner or permission instead.

7.6 Changing Ownership: chown and chgrp

```
student@ubuntu:~$ sudo chown ravi report.txt                # change owner to ravi
student@ubuntu:~$ sudo chown ravi:developers report.txt     # change owner AND group
student@ubuntu:~$ sudo chown :developers report.txt         # change group only
student@ubuntu:~$ sudo chgrp developers report.txt          # same thing, older command
student@ubuntu:~$ sudo chown -R www-data:www-data /var/www/html  # whole website folder for nginx
```

> **DevOps Example:** Nginx runs as the user `www-data`. If your website files are owned by `root` with `600` permission, nginx can't read them and visitors get a **403 Forbidden** error. The fix is `chown -R www-data:www-data /var/www/html` with 755 on folders and 644 on files — *not* 777.

7.7 Default Permissions: umask

When you create a new file, who decides its starting permissions? The **umask**. It is a "mask" that **removes** permissions from the maximum.

- Maximum for new files: **666** (files never start executable). Maximum for new folders: **777**.
- Ubuntu's default umask is usually **002** for normal users (or **022**).

	File	Directory
Maximum	666	777
minus umask 022	– 022	– 022
Result	**644** (rw-r--r--)	**755** (rwxr-xr-x)

```
student@ubuntu:~$ umask
0002
student@ubuntu:~$ umask 077          # make new files private (for this session)
student@ubuntu:~$ touch private.txt && ls -l private.txt
-rw------- 1 student student 0 Oct 13 11:20 private.txt
```

7.8 Special Permissions: SUID, SGID and Sticky Bit

There are three extra permissions for special situations. You will see them as an `s` or `t` in place of an `x`.

Name	Number	What it does	Real example
SUID	4000	On a program: it runs with the **owner's** power, not yours.	`/usr/bin/passwd` is owned by root with SUID, so you can update `/etc/shadow` to change your own password. `-rwsr-xr-x`
SGID	2000	On a folder: new files inside automatically get the **folder's group**.	A shared team folder where every file belongs to `developers`. `drwxrwsr-x`
Sticky bit	1000	On a folder: users can only delete **their own** files, even if everyone can write there.	`/tmp`: everyone writes there, but you can't delete someone else's files. `drwxrwxrwt`

```
student@ubuntu:~$ ls -ld /tmp
drwxrwxrwt 18 root root 4096 Oct 13 11:00 /tmp        # 't' = sticky bit
student@ubuntu:~$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 64152 /usr/bin/passwd          # 's' = SUID

# Setting up a shared team folder:
student@ubuntu:~$ sudo mkdir /srv/team
student@ubuntu:~$ sudo chown root:developers /srv/team
student@ubuntu:~$ sudo chmod 2775 /srv/team      # 2 = SGID, then 775
student@ubuntu:~$ sudo chmod +t /srv/team        # add sticky bit too (now 3775)
```

> **Real-Life Example: Sticky Bit**
> `/tmp` is like a **public notice board** in a college. Anyone can pin a notice (write). But you're only allowed to remove *your own* notices, not tear down other people's. That rule is the sticky bit.

7.9 ACLs: When Owner/Group/Others Is Not Enough

Normal permissions allow only one owner and one group. What if you want to give **just one extra person** access to a file, without adding them to a group? Use an **ACL** (Access Control List).

```
student@ubuntu:~$ sudo apt install acl -y
student@ubuntu:~$ setfacl -m u:priya:rw report.txt       # give priya read+write
student@ubuntu:~$ setfacl -m g:testers:r report.txt     # give group testers read
student@ubuntu:~$ ls -l report.txt
-rw-rw-r--+ 1 student student 120 Oct 13 report.txt    # the '+' means: has an ACL
student@ubuntu:~$ getfacl report.txt                     # see all the rules
# file: report.txt
# owner: student
# group: student
user::rw-
user:priya:rw-
group::r--
group:testers:r--
mask::rw-
other::r--
student@ubuntu:~$ setfacl -x u:priya report.txt         # remove priya's rule
student@ubuntu:~$ setfacl -b report.txt                 # remove ALL ACL rules
```

7.10 How Linux Decides: The Permission Check

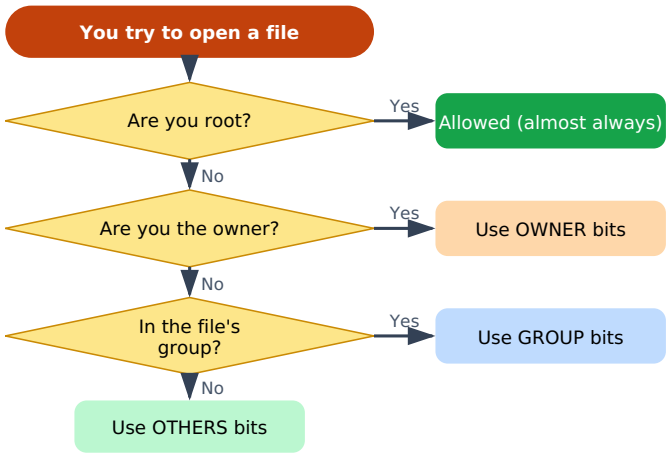


Figure 7.3 — Linux checks only ONE set: the first one that matches you. (ACL rules, if any, are checked between owner and group.)

Surprising fact: Because Linux stops at the first match, if a file is `---rwxrwx`, the **owner** can't open it, even though everyone else can!

Teaching Note: Practise number conversion like a game. Call out "rw-r-----" and they say "640". Call out "750" and they say "rwxr-x---". Ten rounds of this and they'll never forget it.

Practice Lab 7

1. Create `hello.sh` containing `echo "Hello DevOps"`. Try running `./hello.sh`. Fix the error with `chmod`.
2. Convert to numbers: `rwxr-x---`, `rw-rw-r--`, `r--------`.
3. Convert to letters: `750`, `664`, `400`.
4. Create `secret.txt` and make it readable and writable *only* by you. Check with `ls -l`.
5. As `alice` (from Lab 6), try `cat /home/student/secret.txt`. What happens?
6. Create `/srv/shared`, give it group `developers`, and set SGID with `chmod 2775`. Create a file inside and check its group.
7. Check the sticky bit on `/tmp` with `ls -ld /tmp`.
8. Use `setfacl` to give only `alice` read access to `secret.txt`. Test it as alice. Remove the ACL afterwards.
9. Run `umask`. Create a file and a folder and predict their permissions *before* checking with `ls -l`.

Chapter 7 Summary

- Permissions are **r**ead (4), **w**rite (2), e**x**ecute (1), for **u**ser, **g**roup, **o**thers.
- `chmod u+x file` (letters) or `chmod 755 file` (numbers). `-R` for folders.
- Remember: **755** scripts/folders, **644** files, **600** secrets/keys. Avoid 777.
- `chown user:group file` changes owner and group.
- `umask` sets defaults. SUID (4), SGID (2), Sticky (1) are special. ACLs add per-person rules.

Quick Quiz

1. What number is `rwxr-xr-x`?
2. Which permission should an SSH private key have?
3. On a *directory*, what does `x` allow?
4. What does the `t` in `drwxrwxrwt` mean?
5. Your web page shows "403 Forbidden". Should you run `chmod -R 777`? What should you check instead?

Answers: 1) 755 2) 600 3) Entering it with cd 4) Sticky bit: users can only delete their own files 5) No! Check the owner (e.g. `www-data`) and use 755 for folders and 644 for files.

LEVEL 1 FINALE

Ajay Upadhyay · DevOps Mentor

Final Project: Set Up a Team Server

Use everything from Chapters 1–7 together, like a real DevOps task.

Scenario: Your company has a new Ubuntu server. Two developers (`dev1` , `dev2`) and one tester (`qa1`) are joining. Set it up following these requirements. Try it yourself first; the solution is below.

1. Check the Ubuntu and kernel version and update all packages.
2. Create users `dev1` , `dev2` and `qa1` with home folders and bash as the shell.
3. Create groups `developers` and `testers` , and put each user in the right group.
4. Give `dev1` sudo access (team lead).
5. Create `/srv/app/{src,logs,config}` .
6. `/srv/app/src` : developers can read and write; new files must belong to `developers` ; others have no access.
7. `/srv/app/logs` : everybody can write, but nobody can delete others' files.
8. `/srv/app/config/db.conf` : only `dev1` can read and write it, but `qa1` also needs to read it.
9. Create a soft link `/srv/app/current` pointing to `/srv/app/src` .
10. Verify everything with `ls -l` , `getfacl` , `id` and `tree` .

Solution

```
# 1. Versions and updates
admin@server:~$ lsb_release -a; uname -r
admin@server:~$ sudo apt update && sudo apt upgrade -y

# 2. Users
admin@server:~$ for u in dev1 dev2 qa1; do sudo useradd -m -s /bin/bash $u; sudo passwd $u; done

# 3. Groups
admin@server:~$ sudo groupadd developers; sudo groupadd testers
admin@server:~$ sudo usermod -aG developers dev1
admin@server:~$ sudo usermod -aG developers dev2
admin@server:~$ sudo usermod -aG testers qa1

# 4. Sudo for the team lead
admin@server:~$ sudo usermod -aG sudo dev1

# 5. Folders
admin@server:~$ sudo mkdir -p /srv/app/{src,logs,config}

# 6. src: group developers, SGID, no access for others
admin@server:~$ sudo chown root:developers /srv/app/src
admin@server:~$ sudo chmod 2770 /srv/app/src

# 7. logs: everyone writes, sticky bit
admin@server:~$ sudo chmod 1777 /srv/app/logs

# 8. Private config with an ACL for qa1
admin@server:~$ sudo touch /srv/app/config/db.conf
admin@server:~$ sudo chown dev1:dev1 /srv/app/config/db.conf
admin@server:~$ sudo chmod 600 /srv/app/config/db.conf
admin@server:~$ sudo setfacl -m u:qa1:r /srv/app/config/db.conf

# 9. Soft link
admin@server:~$ sudo ln -s /srv/app/src /srv/app/current

# 10. Verify
admin@server:~$ tree /srv/app; ls -l /srv/app; id dev1; getfacl /srv/app/config/db.conf
```

Note: The `for ... do ... done` line in step 2 is a small **loop**: it repeats the commands for each user. You'll learn loops properly in Bash Scripting (Level 3). For now, you can also run the commands three times by hand.

Level 1 Cheat Sheet

Print this page and keep it next to your keyboard.

Info and Help

`lsb_release -a`	Ubuntu version
`uname -r`	Kernel version
`whoami` / `id`	Who am I
`man cmd`	Full manual (q quits)
`cmd --help`	Quick help
`tldr cmd`	Examples
`history`	Past commands

Navigation

`pwd`	Where am I
`cd /path`	Go to path
`cd ..` / `cd ~` / `cd -`	Up / home / back
`ls -lah`	List everything, detailed
`tree -L 2`	Show folder tree

Files and Folders

`mkdir -p a/b/c`	Make folders
`touch f`	Empty file
`cp -r src dst`	Copy
`mv old new`	Move / rename
`rm -i f` / `rm -r dir`	Delete (forever!)
`ln -s target link`	Soft link
`* ? [ ] { }`	Wildcards

Viewing

`cat f`	Small files
`less f`	Big files (q quits)
`head -n 5 f`	First lines
`tail -f f`	Watch live
`wc -l f`	Count lines

Editors

nano: `^O` `^X`	Save / exit
vim: `i` / `Esc`	Type / stop typing
vim: `:wq`	Save and quit
vim: `:q!`	Quit, no save
vim: `dd yy p u`	Cut copy paste undo

Users and Groups

`sudo adduser u`	New user
`sudo passwd u`	Set password
`sudo usermod -aG g u`	Add to group
`sudo deluser --remove-home u`	Delete user
`sudo groupadd g`	New group
`sudo visudo`	Edit sudo rules
`su - u`	Switch user

Permissions

`r=4 w=2 x=1`	Add them up
`chmod 755 f`	Scripts, folders
`chmod 644 f`	Normal files
`chmod 600 f`	Secrets, keys
`chmod u+x f`	Make runnable
`chown u:g f`	Change owner
`setfacl -m u:x:r f`	ACL rule
SUID 4 · SGID 2 · Sticky 1	Special bits

Shortcuts

Tab	Auto-complete
Ctrl + C	Stop command
Ctrl + R	Search history
Ctrl + L	Clear screen

You finished Level 1!

Questions or feedback? Reach me at **ajayupadhyaydevops@gmail.com** — Ajay Upadhyay, DevOps Mentor.

You can now navigate Linux, manage files, edit configs, create users and control permissions. That's the foundation every DevOps skill sits on. **Next up, Level 2:** text processing (grep, sed, awk), pipes and redirection, package management, processes, systemd services, logs, cron and disk management.

LINUX LEARNING SERIES · LEVEL 2

Linux
Daily Administration

by **Ajay Upadhyay**

DevOps Mentor · ajayupadhyaydevops@gmail.com

The intermediate guide to Ubuntu Linux.
Text processing, pipes, packages, processes, services, logs, scheduling and disks — explained deeply, with real terminal output broken down line by line.

```
ajay@ubuntu:~$ awk '{print $9}' access.log | sort | uniq -c
      4 200
      2 404
      2 500
ajay@ubuntu:~$ systemctl is-active nginx
active
ajay@ubuntu:~$ _
```

Chapters 8–17 | Text Processing · Pipes · Finding Files · Packages · Processes · systemd · Logs · Archives · Scheduling · Storage
Based on Ubuntu 24.04 LTS · Continues from Level 1 (Chapters 1–7)

Before You Start

Hi, I'm **Ajay Upadhyay**, a DevOps mentor. In Level 1 you learned to move around Linux, manage files, users and permissions. Level 2 is where Linux turns into a **daily work tool**. These are the exact skills you use every day as a DevOps engineer: searching logs, installing software, finding out why a server is slow, keeping services running, scheduling backups and adding disks.

This level goes **deeper** than Level 1. You will see more examples per command, and a new kind of box:

Box	What it means
Output Explained (dark header)	Real terminal output broken down column by column or line by line, so you know what every part means.
On the Job (grey)	A real situation from DevOps work, and how you would solve it with what you just learned.
Real-Life Example, **Tip**, **Careful!**, **Teaching Note**	Same as Level 1.

Set Up Your Practice Files

Many examples in Chapters 8 and 9 use the same three small files. Create them once, now, so you can type along and get **exactly** the same output as the book.

```
ajay@ubuntu:~$ mkdir -p ~/lab2 && cd ~/lab2

# 1) A small employee list (CSV = Comma Separated Values)
ajay@ubuntu:~/lab2$ cat > employees.csv <<'EOF'
id,name,dept,city,salary
101,Ravi,DevOps,Noida,85000
102,Priya,QA,Pune,60000
103,Amit,DevOps,Bangalore,92000
104,Neha,Dev,Noida,78000
105,Karan,QA,Noida,58000
106,Sara,Dev,Pune,81000
EOF

# 2) A web server access log (the same format nginx uses)
ajay@ubuntu:~/lab2$ cat > access.log <<'EOF'
192.168.1.10 - - [13/Oct/2026:10:00:01 +0530] "GET /index.html HTTP/1.1" 200 612
192.168.1.22 - - [13/Oct/2026:10:00:05 +0530] "GET /login HTTP/1.1" 404 153
192.168.1.10 - - [13/Oct/2026:10:00:09 +0530] "POST /api/order HTTP/1.1" 500 98
10.0.0.5 - - [13/Oct/2026:10:01:12 +0530] "GET /index.html HTTP/1.1" 200 612
192.168.1.22 - - [13/Oct/2026:10:01:30 +0530] "GET /about HTTP/1.1" 200 1024
192.168.1.10 - - [13/Oct/2026:10:02:44 +0530] "GET /api/products HTTP/1.1" 200 2048
10.0.0.5 - - [13/Oct/2026:10:03:02 +0530] "GET /login HTTP/1.1" 404 153
192.168.1.10 - - [13/Oct/2026:10:03:15 +0530] "POST /api/order HTTP/1.1" 500 98
EOF

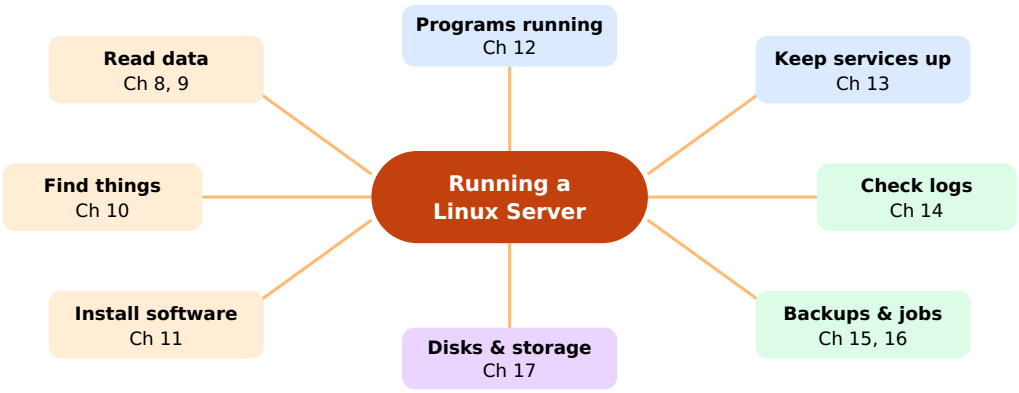
# 3) A small config file
ajay@ubuntu:~/lab2$ cat > app.conf <<'EOF'
# Application settings
server_name=localhost
port=8080
debug=true
# Database
db_host=localhost
db_port=5432
EOF
```

> **Tip: What is `<<'EOF'` ?** It's called a **here-document**. Everything you type until the line `EOF` is fed into the command. Here, `cat > file` saves it into the file. It's a handy way to create a file with several lines in one go.

Contents

Ch 8	**Text Processing and Filters** — grep, sed, awk, cut, sort, uniq, wc, tr, tee, xargs, regular expressions
Ch 9	**I/O Redirection and Pipes** — stdin, stdout, stderr, >, >>, 2>&1, /dev/null, |
Ch 10	**Finding Files** — find, locate, which, whereis
Ch 11	**Package Management** — apt, dpkg, repositories, PPAs, snap, holding versions
Ch 12	**Process Management** — ps, top, htop, signals, kill, jobs, nohup, nice
Ch 13	**Services and systemd** — systemctl, unit files, targets, boot process, timers
Ch 14	**Logs and Monitoring** — journalctl, /var/log, rsyslog, logrotate, dmesg
Ch 15	**Archiving and Compression** — tar, gzip, bzip2, xz, zip
Ch 16	**Task Scheduling** — cron, crontab, at, anacron
Ch 17	**Disk and Storage Management** — df, du, lsblk, partitions, mkfs, mount, fstab, swap, LVM, RAID
End	**Final Project and Cheat Sheet**

The Big Picture of Level 2



Everything in Level 2 connects to one job: keeping a Linux server healthy.

CHAPTER 8

Ajay Upadhyay · DevOps Mentor

Text Processing and Filters

Searching, cutting, counting and transforming text. The most-used skill in day-to-day DevOps.

In this chapter you will learn:

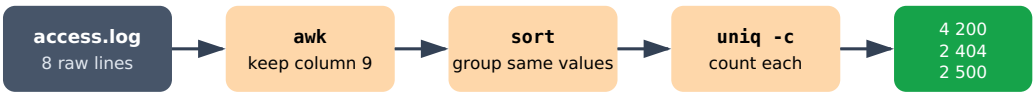
- Why text tools matter so much in Linux
- `grep` to search, with all the important options
- Regular expressions (patterns) in simple language
- `cut` , `sort` , `uniq` , `wc` , `tr` to slice and count
- `sed` to find-and-replace and edit files automatically
- `awk` to work with columns and do calculations
- `tee` and `xargs` to connect commands

8.1 Why Text Tools Are So Important

In Linux, almost everything is text: configuration files, logs, command output, CSV reports. If you can search and reshape text quickly, you can answer almost any question about a server in seconds, without opening a single file by hand.

Real-Life Example: A Water Filter System
Imagine water flowing through pipes. Each **filter** does one small job: one removes sand, one removes salt, one adds minerals. Join them together and you get clean drinking water. Linux text tools work the same way. Each tool does **one small job well**, and you connect them with the **pipe** symbol `|` to build something powerful.

Quick intro to the pipe `|` : You'll use it a lot in this chapter. `command1 | command2` means "send the output of command1 **into** command2". Example: `cat access.log | wc -l` counts the lines of the log. Chapter 9 explains pipes fully.



Each tool is a filter. Together they answer: "how many requests got each status code?"

Figure 8.1 — A pipeline of small filters turns a raw log into an answer.

8.2 grep — Search for Text

`grep` looks through text and prints only the **lines that match** a pattern. Think of it as **Ctrl+F for the terminal**, but it can search thousands of files at once.

Pattern: `grep [options] "pattern" file`

```
ajay@ubuntu:~/lab2$ grep "404" access.log
192.168.1.22 - - [13/Oct/2026:10:00:05 +0530] "GET /login HTTP/1.1" 404 153
10.0.0.5 - - [13/Oct/2026:10:03:02 +0530] "GET /login HTTP/1.1" 404 153
```

Only the two lines containing "404" are printed. The other six lines are filtered out.

The grep Options You'll Use Every Day

Option	Meaning	Example
`-i`	**I**gnore case (ERROR = error = Error)	`grep -i "error" app.log`
`-v`	In**v**ert: show lines that do **NOT** match	`grep -v "^#" app.conf`
`-c`	**C**ount matching lines instead of showing them	`grep -c "500" access.log`
`-n`	Show line **n**umbers	`grep -n "port" app.conf`
`-r`	**R**ecursive: search every file in a folder	`grep -r "password" /etc/`
`-l`	Only **l**ist file names that match	`grep -rl "8080" /etc/`
`-w`	Match whole **w**ords only	`grep -w "port"` won't match "db_port"
`-o`	Print **o**nly the matched part, not the full line	`grep -o "[0-9]*" file`
`-E`	**E**xtended patterns (allows `|` for OR, `+`, etc.)	`grep -E "404|500" access.log`
`-A 3` / `-B 3` / `-C 3`	Show 3 lines **A**fter / **B**efore / around (**C**ontext) each match	`grep -A 5 "Exception" app.log`

```
ajay@ubuntu:~/lab2$ grep -c "500" access.log            # how many server errors?
2
ajay@ubuntu:~/lab2$ grep -n "port" app.conf             # where is "port" mentioned?
3:port=8080
7:db_port=5432
ajay@ubuntu:~/lab2$ grep -v "^#" app.conf              # hide comment lines
server_name=localhost
port=8080
debug=true
db_host=localhost
db_port=5432
ajay@ubuntu:~/lab2$ grep -E "404|500" access.log | wc -l   # total errors (404 OR 500)
4
```

Output Explained: grep -n "port" app.conf	
`3:port=8080`	The number before the colon is the **line number** (line 3). After the colon is the full matching line.
`7:db_port=5432`	Line 7 also matched, because "db_port" *contains* "port". Use `-w` if you only want the exact word.

> **On the Job:** Your app on a server is failing and you need to see the error with some context. Run `grep -i -B 2 -A 10 "exception" /var/log/myapp/app.log`. You'll see 2 lines before every exception (what led to it) and 10 lines after (the full error trace).

8.3 Regular Expressions (Patterns) Made Simple

A **regular expression** (regex) is a pattern that describes text. Instead of searching for one fixed word, you search for a *shape*, like "any line that starts with #" or "anything that looks like an IP address".

> **Real-Life Example:** Telling a friend "find every car number plate that starts with **UP16**" is a pattern. You don't list every plate; you describe the shape. Regex is that description, written in symbols.

Symbol	Meaning	Example and what it matches
`^`	Start of the line	`^#` → lines that begin with #
`$`	End of the line	`\.log$` → text ending in ".log"
`.`	Any single character	`h.t` → hat, hit, hot
`*`	The previous thing, 0 or more times	`ab*c` → ac, abc, abbbc
`+`	1 or more times (needs `-E`)	`[0-9]+` → any number
`?`	0 or 1 time (needs `-E`)	`colou?r` → color, colour
`[abc]`	Any ONE of these characters	`[Ee]rror` → Error, error
`[0-9]` `[a-z]`	A range of characters	`[0-9][0-9][0-9]` → any 3 digits
`[^0-9]`	Anything EXCEPT these	Not a digit
`{n}` `{n,m}`	Exactly n times / n to m times (`-E`)	`[0-9]{3}` → exactly 3 digits
`|`	OR (needs `-E`)	`error|fail`
`\`	Escape: treat the next symbol as normal text	`\.` → a real dot
`^$`	An empty line	`grep -v "^$"` removes blank lines

Example: pull every IP address out of a log and count them

```
ajay@ubuntu:~/lab2$ grep -oE "[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}" access.log | sort | uniq -c | sort -rn
      4 192.168.1.10
      2 192.168.1.22
      2 10.0.0.5
```



Figure 8.2 — Reading the IP-address regex piece by piece.

> **Tip:** Always put your pattern inside **quotes**. Without quotes, the shell may treat symbols like `*` and `|` as its own special characters before grep ever sees them.

8.4 cut — Take Out Columns

`cut` slices each line and keeps only the parts you ask for. Use `-d` to say what separates the columns (the **d**elimiter) and `-f` to pick the **f**ields (column numbers).

```
ajay@ubuntu:~/lab2$ cut -d, -f2,3 employees.csv          # columns 2 and 3, split on commas
name,dept
Ravi,DevOps
Priya,QA
Amit,DevOps
Neha,Dev
Karan,QA
Sara,Dev
ajay@ubuntu:~/lab2$ cut -d: -f1 /etc/passwd | head -3      # all usernames (split on colons)
root
daemon
bin
ajay@ubuntu:~/lab2$ cut -c1-10 access.log | head -2         # -c = characters 1 to 10
192.168.1.
192.168.1.
```

> **Careful!** `cut` treats **every single** separator as a new column. If columns are separated by *several* spaces (like in `ps` or `df` output), `cut` gets confused. Use `awk` for those (section 8.9).

8.5 sort — Put Lines in Order

Option	Meaning
(none)	Alphabetical order (A to Z)
`-r`	**R**everse order
`-n`	**N**umeric sort (so 10 comes after 9, not after 1)
`-h`	**H**uman sizes: understands 2K, 5M, 1G
`-t, -k5`	Separator is a comma (`-t,`); sort by column 5 (`-k5`)
`-u`	**U**nique: remove duplicate lines while sorting

```
ajay@ubuntu:~/lab2$ sort -t, -k5 -n -r employees.csv | head -3    # top 3 salaries
103,Amit,DevOps,Bangalore,92000
101,Ravi,DevOps,Noida,85000
106,Sara,Dev,Pune,81000
ajay@ubuntu:~/lab2$ du -sh /var/* 2>/dev/null | sort -rh | head -3   # biggest folders in /var
2.1G    /var/lib
612M    /var/log
240M    /var/cache
```



Figure 8.3 — Without -n, sort compares text character by character, so "10" comes before "9".

8.6 uniq — Remove or Count Duplicates

`uniq` removes **neighbouring** duplicate lines. With `-c` it **counts** them. Because it only compares a line with the one just above it, you almost always **sort first**.

```
ajay@ubuntu:~/lab2$ cut -d, -f3 employees.csv | tail -n +2 | sort | uniq -c   # people per department
      2 Dev
      2 DevOps
      2 QA
```

Command Explained, step by step	
`cut -d, -f3`	Take only the 3rd column (dept). Output: dept, DevOps, QA, DevOps, Dev, QA, Dev
`tail -n +2`	Start from line 2, so the header "dept" is skipped.
`sort`	Group the same values together: Dev, Dev, DevOps, DevOps, QA, QA
`uniq -c`	Collapse each group into one line, with a count in front.

Option	Meaning
`-c`	Count how many times each line appears
`-d`	Show only lines that appear more than once (duplicates)
`-u`	Show only lines that appear exactly once

> **Remember this famous combo:** `sort | uniq -c | sort -rn` = "count each thing and show the most common first". You'll use it for top IPs, top errors, top URLs, top users... again and again.

8.7 wc — Count Lines, Words, Characters

```
ajay@ubuntu:~/lab2$ wc employees.csv
  7   7 183 employees.csv
ajay@ubuntu:~/lab2$ wc -l access.log              # just the line count
8 access.log
ajay@ubuntu:~/lab2$ ls /etc | wc -l                # how many items are in /etc?
```

Output Explained: wc employees.csv	
7	Lines (`-l`)
7	Words (`-w`). Each line has no spaces, so each line counts as one "word".
183	Bytes/characters (`-c`), including the hidden newline at the end of every line.

8.8 tr — Translate or Delete Characters

`tr` replaces or deletes single characters. It only reads from input (a pipe), not from a file name.

```
ajay@ubuntu:~$ echo "hello world" | tr 'a-z' 'A-Z'         # lowercase to UPPERCASE
HELLO WORLD
ajay@ubuntu:~$ echo "a,b,c" | tr ',' '\n'               # commas into new lines
a
b
c
ajay@ubuntu:~$ echo "too    many   spaces" | tr -s ' '     # -s squeezes repeats into one
too many spaces
ajay@ubuntu:~$ echo "Phone: 98-765-43210" | tr -d '-'     # -d deletes characters
Phone: 9876543210
ajay@ubuntu:~$ echo $PATH | tr ':' '\n'                  # show each PATH folder on its own line
```

8.9 sed — The Stream Editor (Find & Replace)

`sed` edits text **without opening an editor**. Its most common job is find-and-replace, which makes it perfect for changing config files inside scripts and automation.

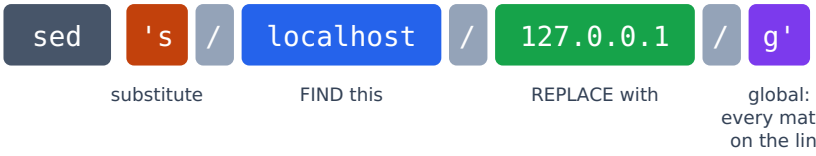


Figure 8.4 — The parts of a sed substitute command.

```
ajay@ubuntu:~/lab2$ sed 's/localhost/127.0.0.1/g' app.conf
# Application settings
server_name=127.0.0.1
port=8080
debug=true
# Database
db_host=127.0.0.1
db_port=5432
```

Careful! sed does not change the file by default. The command above only **prints** the changed text on screen. The file is untouched. To really change the file, add `-i` (in-place). Safer: `-i.bak`, which first saves a backup copy as `app.conf.bak`.

```
ajay@ubuntu:~/lab2$ sed -i.bak 's/debug=true/debug=false/' app.conf   # edit file, keep a backup
ajay@ubuntu:~/lab2$ grep debug app.conf app.conf.bak
app.conf:debug=false
app.conf.bak:debug=true
```

More sed Recipes

Command	What it does
`sed -n '2,4p' file`	Print only lines 2 to 4 (`-n` = silent, `p` = print)
`sed '/^#/d' file`	Delete all comment lines
`sed '/^$/d' file`	Delete all empty lines
`sed '5d' file`	Delete line 5
`sed '3i new text' file`	Insert a line before line 3
`sed '/port=/a timeout=30' file`	Append a line after the line containing "port="
`sed 's/^/# /' file`	Put "# " at the start of every line (comment everything out)
`sed 's|/var/www|/srv/www|g'`	Use `|` as separator when the text contains slashes
`sed -i 's/^#\?Port .*/Port 2222/' /etc/ssh/sshd_config`	Real DevOps use: set the SSH port to 2222, whether the line was commented or not

8.10 awk — The Column Powerhouse

`awk` reads text **line by line**, splits each line into **fields** (columns), and lets you print, filter and calculate. It's almost a small programming language.

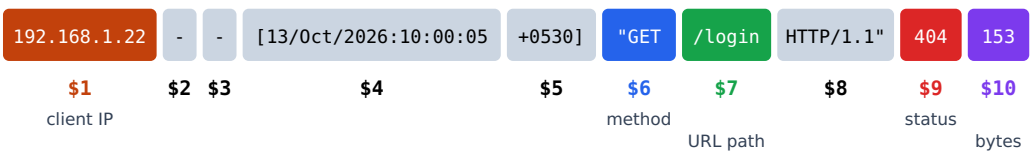


Figure 8.5 — awk splits each line on spaces. Each piece gets a number: $1, $2, $3...

The basic shape of an awk command: `awk 'CONDITION { ACTION }' file`. If the condition is true for a line, the action runs.

Printing columns

```
ajay@ubuntu:~/lab2$ awk '{print $1, $9}' access.log | head -3      # IP and status
192.168.1.10 200
192.168.1.22 404
192.168.1.10 500
ajay@ubuntu:~/lab2$ awk -F, 'NR>1 {print $2, $5}' employees.csv    # -F, = split on commas; NR>1 skips header
Ravi 85000
Priya 60000
Amit 92000
Neha 78000
Karan 58000
Sara 81000
```

Filtering with conditions

```
ajay@ubuntu:~/lab2$ awk -F, '$3=="DevOps" {print $2}' employees.csv     # names in DevOps
Ravi
Amit
ajay@ubuntu:~/lab2$ awk '$9 >= 400 {print $7, $9}' access.log             # all failed requests
/login 404
/api/order 500
/login 404
/api/order 500
ajay@ubuntu:~/lab2$ awk -F, 'NR>1 && $5 > 80000 {print $2" earns "$5}' employees.csv
Ravi earns 85000
Amit earns 92000
Sara earns 81000
```

Calculations: totals and averages

```
ajay@ubuntu:~/lab2$ awk -F, 'NR>1 {sum+=$5} END {print "Total:", sum, "Avg:", sum/(NR-1)}' employees.csv
Total: 454000 Avg: 75666.7
```

Command Explained	
`-F,`	Columns are separated by commas.
`NR>1`	Only for lines after line 1 (skip the header).
`{sum+=$5}`	Add column 5 (salary) to a running total called `sum`.
`END {...}`	Runs **once**, after the last line. Prints the total and the average (sum divided by the 6 data lines).

Grouping: total salary per department

```
ajay@ubuntu:~/lab2$ awk -F, 'NR>1 {d[$3]+=$5} END {for (k in d) print k, d[k]}' employees.csv
QA 118000
Dev 159000
DevOps 177000
```

Here `d` is an **array** (like a small table): one box per department name, adding salaries into the right box.

Real DevOps one-liners with awk

Command	Answers the question
`df -h | awk '$5+0 > 80 {print $6, $5}'`	Which disks are more than 80% full?
`ps aux | awk '{print $1}' | sort | uniq -c`	How many processes does each user run?
`awk -F: '$3 >= 1000 {print $1}' /etc/passwd`	List all real (human) users
`awk '{s+=$10} END {print s/1024 " KB"}' access.log`	Total data sent by the web server
`awk 'NF' file`	Remove empty lines (lines with 0 fields)

8.11 tee — Save AND Show at the Same Time

Normally output goes either to the screen or to a file. `tee` sends it to **both**, like a T-shaped pipe joint.

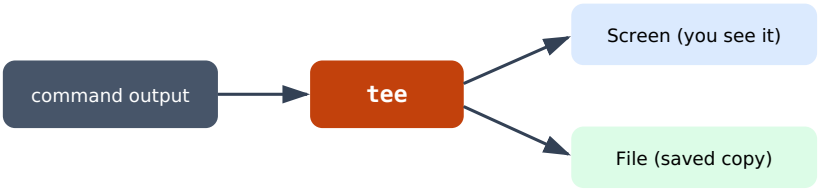


Figure 8.6 — tee splits output into two directions.

```
ajay@ubuntu:~$ sudo apt update | tee update.log          # watch it AND keep a record
ajay@ubuntu:~$ echo "line 2" | tee -a notes.txt            # -a = append, don't overwrite
ajay@ubuntu:~$ echo "10.0.0.9 db01" | sudo tee -a /etc/hosts  # write to a root-owned file
```

Why `sudo tee` ? `sudo echo "x" >> /etc/hosts` **fails**, because the `>>` is done by *your* shell, which isn't root. Piping into `sudo tee -a` makes the root-powered program do the writing. This trick appears in many install guides.

8.12 xargs — Turn Output into Arguments

Some commands (like `rm` , `mkdir`) don't read from a pipe; they need the names as **arguments**. `xargs` takes lines from the pipe and hands them to the command as arguments.

Real-Life Example: A teacher reads out a list of student names (the pipe). `xargs` is the assistant who writes all those names onto one attendance form and gives it to the principal (the command).

```
ajay@ubuntu:~$ echo "dev test prod" | xargs mkdir            # same as: mkdir dev test prod
ajay@ubuntu:~$ find /tmp -name "*.tmp" | xargs rm -f       # delete every .tmp found
ajay@ubuntu:~$ cat servers.txt | xargs -I {} ping -c 1 {} # -I {} = put each line where {} is
ajay@ubuntu:~$ ls *.log | xargs -n 1 gzip                  # -n 1 = one file per gzip call
ajay@ubuntu:~$ find . -name "*.log" -print0 | xargs -0 rm  # safe even if names contain spaces
```

On the Job: "Who is attacking my login page?"
`grep "/login" access.log | grep " 404 " | awk '{print $1}' | sort | uniq -c | sort -rn | head`
Step by step: find login requests → keep only failures → keep only the IP → group → count → biggest first → top 10. Six tiny tools, one powerful answer.

Teaching Note: Build long pipelines **one step at a time**. Run the first command, look at the output, then add `| next` and run again. Students who type the whole thing at once can't tell which part broke.

Practice Lab 8

1. Show all lines in `access.log` that are **not** status 200.
2. Count how many requests each URL received (column 7), most popular first.
3. From `employees.csv` , print the names of everyone who lives in Noida.
4. Print the average salary of the QA department with awk.
5. Use sed to print `app.conf` without comments and without empty lines.
6. Change `port=8080` to `port=9090` in `app.conf` with sed, keeping a backup.
7. List every real user (UID 1000 or more) from `/etc/passwd` .
8. Find every file under `/etc` that mentions `localhost` (show file names only).
9. Write a regex that matches lines ending with `.conf` . Test it on `ls /etc | grep ...` .

Chapter 8 Summary

- `grep` finds lines (`-i -v -c -n -r -E -o -A -B`). Regex describes the shape of text.
- `cut` picks columns with a single separator. `awk` handles columns, conditions and maths.
- `sort` orders (`-n -r -h -k`), `uniq -c` counts (sort first!), `wc -l` counts lines.
- `sed 's/old/new/g'` replaces; add `-i.bak` to edit the file safely. `tr` changes characters.
- `tee` saves and shows; `xargs` turns piped lines into arguments.

Quick Quiz

1. What does `grep -v "^$" file` do?
2. Why does `uniq -c` usually need `sort` before it?
3. In awk, what are `$0` , `NR` and `NF` ?
4. You ran `sed 's/a/b/' file` but the file didn't change. Why?
5. Why does `sudo echo "x" >> /etc/hosts` fail, and what's the fix?

Answers: 1) Shows all lines that are not empty. 2) uniq only compares neighbouring lines. 3) The whole line, the current line number, the number of fields. 4) sed only prints unless you use `-i` . 5) The redirect runs as you, not root; use `echo "x" | sudo tee -a /etc/hosts` .

CHAPTER 9

Ajay Upadhyay · DevOps Mentor

I/O Redirection and Pipes

Controlling where input comes from and where output and errors go.

In this chapter you will learn:

- The three standard streams: stdin, stdout, stderr
- Saving output: `>` and `>>`
- Handling errors: `2>`, `2>&1`, `&>`
- The "black hole": `/dev/null`
- Reading input: `<`, `<<`, `<<<`
- Pipes in depth, and chaining with `&&`, `||`, `;`

9.1 The Three Standard Streams

Every Linux command is born with three "channels" connected to it. Each has a name and a number (called a **file descriptor**):

Number	Name	What flows through it	Default connected to
0	stdin	Standard **input**: data going in	Your keyboard
1	stdout	Standard **output**: normal results	Your screen
2	stderr	Standard **error**: error messages	Your screen



Figure 9.1 — Every command has one way in and two separate ways out.

Real-Life Example: A Post Office Counter
Letters come **in** through the front window (stdin). Delivered letters go out through the **main door** (stdout). Letters that can't be delivered (wrong address) go into a **separate "returned" bin** (stderr). Because they're separate, you can send the good letters one way and the problem letters another way.

See the difference yourself

```
ajay@ubuntu:~$ ls /etc/hostname /nothing-here
ls: cannot access '/nothing-here': No such file or directory     # <- stderr (2)
/etc/hostname                                                   # <- stdout (1)
```

On screen they look the same, but they travel on two different channels. Now watch what happens when we redirect them.

9.2 Redirecting Output: > and >>

Symbol	Meaning
`>`	Send stdout to a file. **Overwrites** (empties the file first). Creates it if missing.
`>>`	Send stdout to a file. **Appends** (adds to the end). Creates it if missing.

```
ajay@ubuntu:~$ echo "first" > demo.txt
ajay@ubuntu:~$ echo "second" > demo.txt        # > replaced the content!
ajay@ubuntu:~$ cat demo.txt
second
ajay@ubuntu:~$ echo "third" >> demo.txt       # >> adds to the end
ajay@ubuntu:~$ cat demo.txt
second
third
ajay@ubuntu:~$ date >> ~/backup-history.log  # build a running log
ajay@ubuntu:~$ > big.log                     # empty a file without deleting it
```

Careful! One `>` instead of `>>` can wipe out an important file in a fraction of a second. You can protect yourself with `set -o noclobber` : after that, `>` refuses to overwrite existing files (use `>|` when you really mean it).

Tip: Emptying a running log. If a log file fills the disk, don't `rm` it while the app is running (the app keeps writing to the deleted file, and the space isn't freed). Instead, empty it with `> /var/log/myapp.log` or `truncate -s 0 /var/log/myapp.log` .

9.3 Redirecting Errors: 2>, 2>&1 and &>

```
ajay@ubuntu:~$ ls /etc/hostname /nothing-here > out.txt 2> err.txt
ajay@ubuntu:~$ cat out.txt
/etc/hostname
ajay@ubuntu:~$ cat err.txt
ls: cannot access '/nothing-here': No such file or directory
```

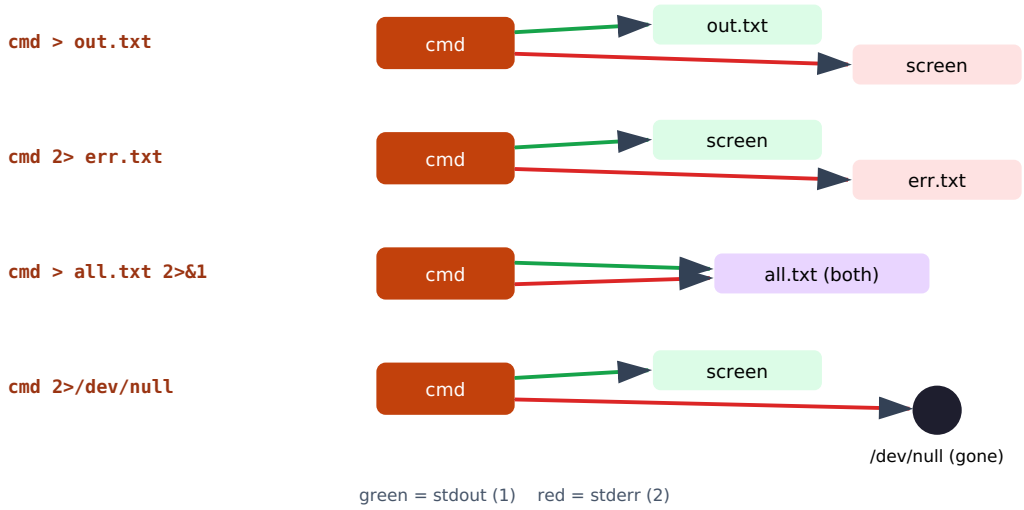


Figure 9.2 — Where output and errors go with each redirection.

Syntax	Meaning
`cmd 2> err.txt`	Errors to a file, normal output to screen
`cmd > out.txt 2> err.txt`	Each to its own file
`cmd > all.txt 2>&1`	**Both** into the same file. Read `2>&1` as "send 2 to wherever 1 is going"
`cmd &> all.txt`	Bash shortcut for the line above
`cmd >> all.log 2>&1`	Both, appended. **The classic line for cron jobs and scripts.**
`cmd 2>&1 | grep error`	Send errors into the pipe too, so grep can search them

Careful! Order matters. `cmd > file 2>&1` puts both in the file. But `cmd 2>&1 > file` does **not**: errors were pointed at the screen *before* stdout was moved to the file. Always write the file redirect first, then `2>&1` .

9.4 /dev/null — The Black Hole

`/dev/null` is a special file that throws away everything written to it. Use it when you don't care about some output.

```
ajay@ubuntu:~$ find / -name "nginx.conf"                  # screen full of "Permission denied"
ajay@ubuntu:~$ find / -name "nginx.conf" 2>/dev/null      # hide errors, see only results
/etc/nginx/nginx.conf
ajay@ubuntu:~$ ping -c1 google.com &>/dev/null && echo "Internet OK"   # silent check
Internet OK
```

9.5 Redirecting Input: <, << and <<<

Symbol	Name	Example
`<`	Read input from a file	`wc -l < access.log` → `8` (no file name printed)
`<< EOF`	Here-document: several lines of input	Writing config files from scripts (see below)
`<<<`	Here-string: one string as input	`tr a-z A-Z <<< "hello"` → `HELLO`

```
# Create an nginx site config from a script, in one go:
ajay@ubuntu:~$ sudo tee /etc/nginx/sites-available/myapp <<'EOF'
server {
    listen 80;
    server_name myapp.example.com;
    root /var/www/myapp;
}
EOF
```

9.6 Pipes In Depth

A pipe `|` connects the **stdout** of the left command to the **stdin** of the right command. No temporary files are needed; data flows directly, and both commands run at the same time.



Note: only stdout goes through a pipe. stderr still goes to the screen (unless you add 2>&1).

Figure 9.3 — A pipe is a direct connection between two commands.

> **Tip: Useless use of cat.** `cat access.log | grep POST` works, but `grep POST access.log` does the same with one less program. Both are fine while learning; just know that most tools can read the file directly.

Pipeline examples you'll actually use

Pipeline	What it tells you
`ps aux | grep nginx`	Is nginx running?
`history | grep docker | tail -5`	The last 5 docker commands you ran
`ls -l /var/log | sort -k5 -n | tail -3`	The 3 biggest log files
`cat /etc/passwd | cut -d: -f7 | sort | uniq -c`	Which shells users have
`journalctl -u nginx | grep -i error | wc -l`	How many nginx errors were logged
`ss -tn | awk 'NR>1{print $5}' | cut -d: -f1 | sort | uniq -c | sort -rn`	Which IPs have the most open connections

9.7 Chaining Commands: ; && ||

Every command finishes with an **exit code**: `0` means success, anything else means failure. You can see it with `echo $?`. Chaining operators use it to decide what runs next.

```
ajay@ubuntu:~$ ls /etc/hostname ; echo $?
/etc/hostname
0                                        # success
ajay@ubuntu:~$ ls /nothing ; echo $?
ls: cannot access '/nothing': No such file or directory
2                                        # failure
```

Operator	Meaning	Example
`;`	Run one after the other, no matter what	`cd /tmp ; ls`
`&&`	Run the next **only if** the previous **succeeded**	`sudo apt update && sudo apt upgrade -y`
`||`	Run the next **only if** the previous **failed**	`ping -c1 db01 || echo "DB is down"`

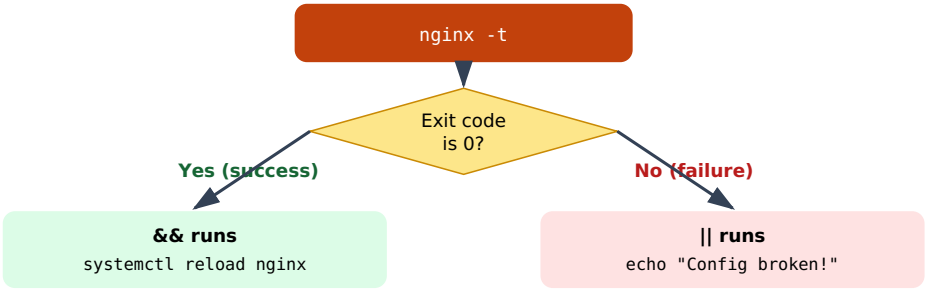


Figure 9.4 — `sudo nginx -t && sudo systemctl reload nginx || echo "Config broken!"` — only reload if the config test passes.

> **On the Job: a safe deploy line**
> `cd /srv/app && git pull && npm install && sudo systemctl restart myapp && echo "Deployed!"`
> If any step fails (for example, `git pull` has a conflict), everything after it stops. You will never restart the app with half-updated code.

Practice Lab 9

1. Run `ls /etc/hosts /fake`. Save the good output in `ok.txt` and the error in `bad.txt`.
2. Run the same command saving both into `both.txt`. Check the file.
3. Try `ls /fake 2>&1 > x.txt`. Where did the error go? Explain why.
4. Use `>>` to append the current date into `dates.log` three times. Check it has 3 lines.
5. Search the whole system for files named `sshd_config` without showing any errors.
6. Write one line that prints "Google reachable" if `ping -c1 google.com` works, and "No internet" if not, with no ping output on screen.
7. Use a here-document to create `team.txt` with 3 names, then count them with `wc -l < team.txt`.

Chapter 9 Summary

- Streams: 0 = stdin, 1 = stdout, 2 = stderr.
- `>` overwrite, `>>` append, `2>` errors, `> f 2>&1` or `&> f` both.
- `/dev/null` throws output away. `<`, `<<EOF`, `<<<` feed input.
- `|` connects stdout to the next stdin.
- Exit code 0 = success. `&&` on success, `||` on failure, `;` always.

Quick Quiz

1. What is the difference between `>` and `>>`?
2. Which number is stderr?
3. Write a command that runs `backup.sh` and appends both output and errors to `backup.log`.
4. What does `echo $?` show?
5. In `A || B`, when does B run?

Answers: 1) > overwrites, >> appends. 2) 2. 3) `./backup.sh >> backup.log 2>&1` 4) The exit code of the last command. 5) Only if A fails.

CHAPTER 10

Ajay Upadhyay · DevOps Mentor

Finding Files

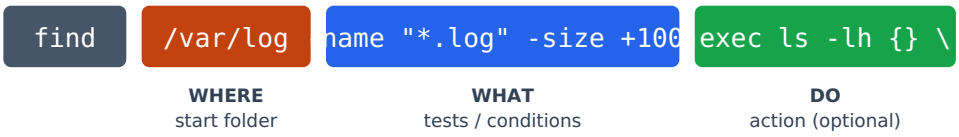
Locating any file on the system by name, size, age, owner or permissions — and acting on what you find.

In this chapter you will learn:

- `find` : the most powerful search tool, with all key tests
- Combining conditions (AND, OR, NOT)
- Taking action on results: `-exec` , `-delete` , `xargs`
- `locate` for instant searches
- `which` , `whereis` and `type` for finding commands

10.1 find — Search Live, By Anything

`find` walks through folders **right now** and checks every file against your conditions. It's slower than `locate` , but always up to date and much more flexible.



WHERE
start folder

WHAT
tests / conditions

DO
action (optional)

Figure 10.1 — find = where + what + do. Default action is just to print the path.

Search by name

```
ajay@ubuntu:~$ find /etc -name "hosts"              # exact name (case-sensitive)
/etc/hosts
ajay@ubuntu:~$ find /etc -iname "*.CONF" 2>/dev/null | head -3  # -iname ignores case
/etc/adduser.conf
/etc/ld.so.conf
/etc/sysctl.conf
ajay@ubuntu:~$ find . -name "*.py"                  # all Python files under current folder
```

Search by type

```
ajay@ubuntu:~$ find /var/log -type f      # f = files only
ajay@ubuntu:~$ find /etc -type d -name "nginx"   # d = directories only
ajay@ubuntu:~$ find /usr/bin -type l | head -3   # l = symbolic links
```

Search by size

Test	Meaning
`-size +100M`	Bigger than 100 megabytes
`-size -10k`	Smaller than 10 kilobytes
`-size 1G`	About exactly 1 gigabyte
`-empty`	Empty files or folders

```
ajay@ubuntu:~$ sudo find / -type f -size +500M 2>/dev/null       # what is eating my disk?
/var/lib/docker/overlay2/3f2a.../layer.tar
/home/ajay/Downloads/ubuntu-24.04.iso
```

Search by time

Time is counted in days (`-mtime`) or minutes (`-mmin`). `+` means "more than", `-` means "less than".

Test	Meaning
`-mtime -1`	Modified in the last 24 hours
`-mtime +30`	Modified more than 30 days ago
`-mmin -15`	Modified in the last 15 minutes
`-newer file.txt`	Modified more recently than file.txt

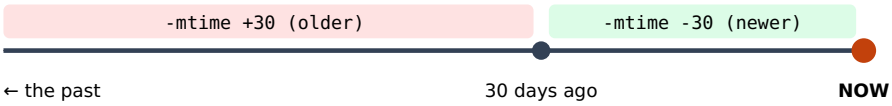


Figure 10.2 — Reading + and - in time tests.

Search by owner and permission

```
ajay@ubuntu:~$ find /home -user ravi                 # files owned by ravi
ajay@ubuntu:~$ find /srv -group developers           # files owned by a group
ajay@ubuntu:~$ sudo find / -nouser 2>/dev/null       # files whose owner was deleted
ajay@ubuntu:~$ find /var/www -perm 777               # exactly 777 (a security problem!)
ajay@ubuntu:~$ sudo find / -perm -4000 2>/dev/null   # all SUID programs (security audit)
```

Limiting depth

```
ajay@ubuntu:~$ find /etc -maxdepth 1 -name "*.conf"    # don't go into sub-folders
```

10.2 Combining Conditions

Operator	Meaning	Example
(two tests side by side)	AND (both must match)	`-name "*.log" -size +10M`
`-o`	OR	`\( -name "*.jpg" -o -name "*.png" \)`
`!` or `-not`	NOT	`! -name "*.gz"`

```
ajay@ubuntu:~$ find ~/Pictures \( -name "*.jpg" -o -name "*.png" \) -size +5M
ajay@ubuntu:~$ find /var/log -name "*.log" ! -name "*.gz" -mtime +7
```

10.3 Doing Something With the Results

Action	What it does
`-print`	Print the path (the default)
`-ls`	Show details like `ls -l`
`-delete`	Delete each match
`-exec cmd {} \;`	Run `cmd` once per file. `{}` is replaced by the file name.
`-exec cmd {} +`	Run `cmd` once with **all** files (faster)
`-ok cmd {} \;`	Like -exec, but asks "yes/no?" for each file

```
ajay@ubuntu:~$ find /tmp -name "*.tmp" -mtime +7 -delete                # clean old temp files
ajay@ubuntu:~$ find /var/www -type d -exec chmod 755 {} \;              # all folders to 755
ajay@ubuntu:~$ find /var/www -type f -exec chmod 644 {} \;              # all files to 644
ajay@ubuntu:~$ find /var/log -name "*.log" -mtime +3 -exec gzip {} +    # compress old logs
ajay@ubuntu:~$ find . -name "*.conf" -exec grep -l "8080" {} +         # which configs use 8080?
```

Careful! Test before you delete. Always run your `find` command **without** `-delete` first and read the list. Also, put `-delete` **at the end**. `find / -delete -name "*.tmp"` would delete *everything* before checking the name!

On the Job: "The disk is 95% full!"
1. `sudo find / -xdev -type f -size +200M -exec ls -lh {} + 2>/dev/null` → find big files on this disk only (`-xdev` stays on one filesystem).
2. `sudo find /var/log -name "*.gz" -mtime +30 -delete` → remove compressed logs older than a month.

10.4 locate — Instant Search from a Database

`locate` doesn't search the disk. It searches a **database** (an index) of all file names, which is updated about once a day. That's why it's almost instant.

Real-Life Example: `find` is walking through every shelf of a library looking at each book. `locate` is checking the library's **catalogue card**. Very fast, but a book that arrived this morning may not be in the catalogue yet.

```
ajay@ubuntu:~$ sudo apt install plocate -y
ajay@ubuntu:~$ sudo updatedb                  # refresh the database now
ajay@ubuntu:~$ locate sshd_config
/etc/ssh/sshd_config
/usr/share/man/man5/sshd_config.5.gz
ajay@ubuntu:~$ locate -i readme | head -3     # -i ignore case
ajay@ubuntu:~$ locate -c ".conf"              # -c just count
```

	find	locate
Speed	Slower (reads the disk)	Instant
Up to date?	Always	Only since the last `updatedb`
Search by	Name, size, time, owner, permissions...	Name only
Can take action?	Yes (`-exec` , `-delete`)	No

10.5 Finding Commands: which, whereis, type

```
ajay@ubuntu:~$ which python3                # which file runs when I type this?
/usr/bin/python3
ajay@ubuntu:~$ which -a python3             # all matches in PATH
ajay@ubuntu:~$ whereis nginx                # program, config and manual locations
nginx: /usr/sbin/nginx /usr/lib/nginx /etc/nginx /usr/share/nginx /usr/share/man/man8/nginx.8.gz
ajay@ubuntu:~$ type ll                      # is it a program, alias or built-in?
ll is aliased to `ls -alF'
ajay@ubuntu:~$ type cd
cd is a shell builtin
```

Tip: "command not found" but you just installed it? Run `which name` . If it shows nothing, the program's folder isn't in your `PATH` , or the package installs it under a different name. `dpkg -L package | grep bin` shows the real file name.

Practice Lab 10

1. Find every `.conf` file directly inside `/etc` (not sub-folders).
2. Find files in your home folder modified in the last 60 minutes.
3. Find all files bigger than 50 MB on the system, hiding errors, and show their sizes.
4. Find all empty files in `/tmp` .
5. In `~/lab2` , create 3 `.bak` files. Use `find ... -ok rm {} \;` to delete them one by one with confirmation.
6. Find all SUID programs on your system. Is `/usr/bin/passwd` among them?
7. Use `which` , `whereis` and `type` on `ls` . Explain each answer.

Chapter 10 Summary

- `find WHERE TESTS ACTION` . Tests: `-name -iname -type -size -mtime -user -perm -maxdepth` .
- Combine with AND (default), `-o` (OR), `!` (NOT).
- Act with `-exec {} \;` , `-exec {} +` , `-delete` . Test without -delete first.
- `locate` is instant but uses a database (`updatedb`).
- `which` , `whereis` , `type` find commands.

Quick Quiz

1. Find all `.log` files in `/var/log` older than 14 days. Write the command.
2. What does `{}` mean in `-exec` ?
3. You created a file 5 minutes ago and `locate` can't find it. Why?
4. Difference between `-exec cmd {} \;` and `-exec cmd {} +` ?
5. Which command tells you if `ll` is an alias?

Answers: 1) `find /var/log -name "*.log" -mtime +14` 2) The current file name. 3) The locate database hasn't been updated; run `sudo updatedb` . 4) `\;` runs once per file, `+` runs once with all files. 5) `type ll`

CHAPTER 11

Ajay Upadhyay · DevOps Mentor

Package Management on Ubuntu

Installing, updating and removing software the safe way.

In this chapter you will learn:

- What packages, repositories and dependencies are
- `apt` : update, upgrade, install, remove, purge, search
- Reading apt's output
- `dpkg` for .deb files and package details
- Repositories, `sources` files, adding vendor repos and PPAs
- Snap packages
- Holding and pinning versions on production servers

11.1 Packages, Repositories and Dependencies

On Windows you download an installer from a website. On Ubuntu, software comes as **packages** (`.deb` files) from trusted online stores called **repositories**. A program that needs other programs to work has **dependencies**; the package manager installs them automatically.

Real-Life Example: The Play Store

The **repository** is like the Google Play Store: a trusted shop with thousands of apps. **apt** is the Play Store app on your phone. **Dependencies** are like a recipe: to make tea you also need milk, sugar and tea leaves. Ask apt for "tea", and it brings all the ingredients too.

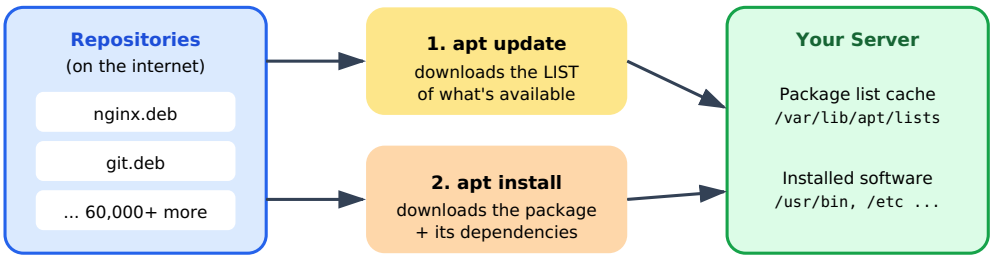


*Figure 11.1 — **update** refreshes the menu, **install** orders the food.*

11.2 The Everyday apt Commands

Command	What it does
`sudo apt update`	Refresh the list of available packages. **Run this first, always.** It installs nothing.
`apt list --upgradable`	Show which installed packages have newer versions
`sudo apt upgrade`	Upgrade all installed packages (never removes anything)
`sudo apt full-upgrade`	Upgrade, and remove packages if needed to solve conflicts
`sudo apt install nginx`	Install a package (and dependencies)
`sudo apt install nginx=1.24.0-2ubuntu7`	Install a specific version
`sudo apt install ./app.deb`	Install a downloaded .deb file and fetch its dependencies
`sudo apt remove nginx`	Uninstall, but **keep** config files
`sudo apt purge nginx`	Uninstall **and delete** config files
`sudo apt autoremove`	Remove dependencies that nothing needs anymore
`sudo apt clean`	Delete downloaded .deb files from the cache (frees disk space)
`apt search keyword`	Search package names and descriptions
`apt show nginx`	Details: version, size, dependencies, description
`apt list --installed`	List everything installed
`apt policy nginx`	Which version is installed, which are available, and from where

Reading apt's output

```
ajay@ubuntu:~$ sudo apt install nginx
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  nginx-common
Suggested packages:
  fcgiwrap nginx-doc ssl-cert
The following NEW packages will be installed:
  nginx nginx-common
0 upgraded, 2 newly installed, 0 to remove and 12 not upgraded.
Need to get 552 kB of archives.
After this operation, 1,596 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
```

Output Explained	
`Reading package lists`	apt reads the list saved by your last `apt update`.
`Building dependency tree`	apt works out what else nginx needs.
`additional packages`	Dependencies apt will install for you automatically (here: nginx-common).
`Suggested packages`	Optional extras. Not installed unless you ask.
`0 upgraded, 2 newly...`	The summary: nothing upgraded, 2 new packages, nothing removed. "12 not upgraded" = 12 updates waiting for `apt upgrade`.
`Need to get 552 kB`	Download size.
`After this operation...`	Disk space the installed files will use.
`[Y/n]`	Capital Y = the default if you just press Enter. Add `-y` to the command to skip this question (used in scripts).

apt vs apt-get: `apt` is the modern, friendly command for humans (progress bar, colours). `apt-get` is the older one with output that never changes, so it's preferred **inside scripts** and Dockerfiles: `apt-get update && apt-get install -y nginx`.

> **Careful! "Could not get lock /var/lib/dpkg/lock-frontend"** means another apt process is already running (often automatic updates in the background). Wait a few minutes. Don't delete the lock file; that can corrupt the package database.

11.3 dpkg — The Low-Level Tool

`apt` is the friendly manager that downloads things and handles dependencies. Underneath, it uses `dpkg`, which actually installs `.deb` files but does **not** fetch dependencies.

Command	What it does
`dpkg -l`	List all installed packages
`dpkg -l | grep nginx`	Is nginx installed?
`dpkg -L nginx`	**L**ist every file a package installed
`dpkg -S /usr/sbin/nginx`	**S**earch: which package owns this file?
`sudo dpkg -i file.deb`	Install a .deb file (no dependency download)
`sudo apt -f install`	**F**ix broken dependencies after a dpkg -i
`dpkg --get-selections`	Export the list of installed packages

```
ajay@ubuntu:~$ dpkg -l | grep nginx
ii  nginx           1.24.0-2ubuntu7   amd64   small, powerful, scalable web/proxy server
ii  nginx-common    1.24.0-2ubuntu7   all     small, powerful, scalable web/proxy server - common files
```

Output Explained: dpkg -l	
`ii`	Status. First `i` = you asked to install it; second `i` = it is installed. `rc` = removed, but config files remain (use `purge` to clean).
`nginx`	Package name
`1.24.0-2ubuntu7`	Version. `1.24.0` = nginx's own version, `-2ubuntu7` = Ubuntu's packaging revision.
`amd64 / all`	Architecture: 64-bit Intel/AMD, or "all" for files that work on any CPU

11.4 Repositories and Where They Are Configured

Ubuntu's own repositories are split into four **components**:

Component	Contents
main	Free software, officially supported by Canonical (Ubuntu's company)
universe	Free software, maintained by the community
restricted	Proprietary drivers (e.g. some graphics cards)
multiverse	Software with copyright or legal restrictions

The list of repositories lives in:

- `/etc/apt/sources.list.d/ubuntu.sources` — Ubuntu 24.04 and newer (new "deb822" format)
- `/etc/apt/sources.list` — older Ubuntu versions (one-line format)
- `/etc/apt/sources.list.d/*.list` or `*.sources` — extra repos you add (Docker, Kubernetes...)

```
ajay@ubuntu:~$ cat /etc/apt/sources.list.d/ubuntu.sources
Types: deb
URIs: http://archive.ubuntu.com/ubuntu/
Suites: noble noble-updates noble-backports
Components: main restricted universe multiverse
Signed-By: /usr/share/keyrings/ubuntu-archive-keyring.gpg
```

Output Explained	
`Types: deb`	Ready-made packages (deb-src would be source code).
`URIs`	The web address of the repository.
`Suites`	`noble` = Ubuntu 24.04's codename. `-updates` = bug fixes, `-backports` = newer versions brought back.
`Signed-By`	The GPG key used to prove packages really come from Ubuntu and weren't tampered with.

Adding a vendor's repository (the modern, safe way)

Many tools (Docker, Kubernetes, HashiCorp, Grafana) publish their own repository. The pattern is always the same: **1)** download their signing key, **2)** add the repo line pointing to that key, **3)** `apt update`, **4)** install.

```
# Example: Docker's official repository
ajay@ubuntu:~$ sudo install -m 0755 -d /etc/apt/keyrings
ajay@ubuntu:~$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
ajay@ubuntu:~$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
  https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME) stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list
ajay@ubuntu:~$ sudo apt update
ajay@ubuntu:~$ sudo apt install docker-ce docker-ce-cli containerd.io -y
```

> **Tip:** Always copy a vendor's repo steps from their **official documentation**, since keys and URLs change over time. And notice how this uses `sudo tee` and `$(...)` (which runs a command and pastes its answer in place) from earlier chapters.

PPAs (Personal Package Archives)

A PPA is a small repository run by a person or project on Launchpad, often used to get a newer version than Ubuntu ships.

```
ajay@ubuntu:~$ sudo add-apt-repository ppa:deadsnakes/ppa      # newer Python versions
ajay@ubuntu:~$ sudo apt update && sudo apt install python3.13
ajay@ubuntu:~$ sudo add-apt-repository --remove ppa:deadsnakes/ppa
```

> **Careful!** A PPA is trusted with root access to your machine every time you update. Only use PPAs from well-known, trusted sources, and avoid them on production servers when possible.

11.5 Snap Packages

Snaps are self-contained packages that bundle all their dependencies inside. They update themselves automatically and run in a sandbox.

```
ajay@ubuntu:~$ snap find kubectl
ajay@ubuntu:~$ sudo snap install kubectl --classic     # --classic = full system access
ajay@ubuntu:~$ snap list                                # installed snaps
ajay@ubuntu:~$ sudo snap refresh                        # update all snaps
ajay@ubuntu:~$ sudo snap remove kubectl
```

	apt (.deb)	snap
Dependencies	Shared with the system	Bundled inside the snap
Size	Smaller	Bigger
Updates	When you run apt upgrade	Automatic, in the background
Typical use	Servers, system tools	Desktop apps, CLIs like kubectl

11.6 Holding and Pinning Versions

On production servers, you often don't want an important package (like your database or Docker) to upgrade by surprise.

```
ajay@ubuntu:~$ sudo apt-mark hold docker-ce          # freeze this package
docker-ce set on hold.
ajay@ubuntu:~$ apt-mark showhold                     # list held packages
docker-ce
ajay@ubuntu:~$ sudo apt-mark unhold docker-ce        # allow upgrades again
```

For finer control, **pinning** sets priorities in `/etc/apt/preferences.d/`. For example, to keep nginx on the 1.24 series:

```
ajay@ubuntu:~$ sudo tee /etc/apt/preferences.d/nginx <<'EOF'
Package: nginx nginx-common
Pin: version 1.24.*
Pin-Priority: 1001
EOF
ajay@ubuntu:~$ apt policy nginx            # check the pin is applied
```

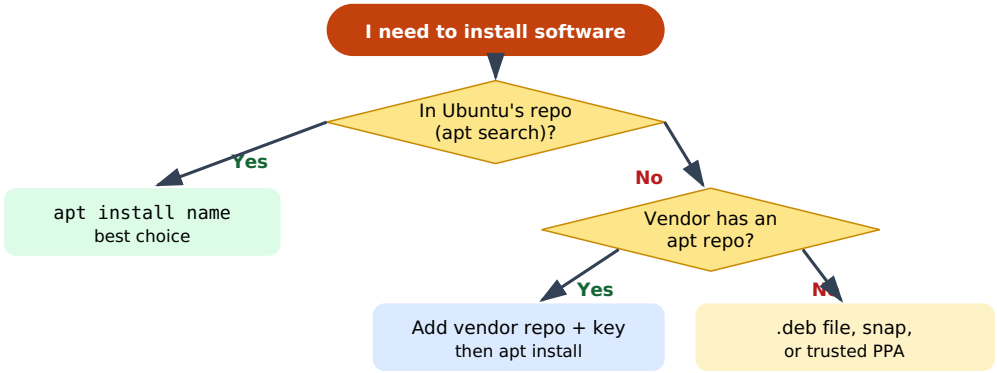


Figure 11.2 — Flowchart: where to get software from, safest first.

> **On the Job: patching a server safely**
> `sudo apt update` → `apt list --upgradable` (review what will change) → `sudo apt upgrade -y` → `sudo apt autoremove -y` → check `ls /var/run/reboot-required`; if that file exists (usually after a kernel update), schedule a reboot.

> **Teaching Note:** The #1 beginner confusion is **update vs upgrade**. Use the restaurant: `update` = get today's menu. `upgrade` = actually order the new dishes. Getting the menu alone never changes what's on your table.

Practice Lab 11

1. Run `sudo apt update`, then `apt list --upgradable`. How many updates are waiting?
2. Search for `htop`, view its details with `apt show`, and install it.
3. Use `dpkg -L htop` to see which files it installed. Where is the program itself?
4. Which package owns `/bin/ls`? (Use `dpkg -S`.)
5. Install `tree`, then `remove` it. Check `dpkg -l tree`. Now `purge` it and check again. What changed?
6. Put `htop` on hold, confirm with `apt-mark showhold`, then unhold it.
7. Run `apt policy openssl` and explain what it shows.

Chapter 11 Summary

- Packages come from repositories. `apt update` refreshes the list; `apt upgrade` installs updates.
- `install`, `remove` (keeps config), `purge` (deletes config), `autoremove`, `search`, `show`, `policy`.
- `dpkg -l / -L / -S / -i` for low-level details.
- Repos live in `/etc/apt/sources.list.d/`. Add vendor repos with a signing key. Be careful with PPAs.
- Snaps bundle dependencies. `apt-mark hold` freezes versions on production.

Quick Quiz

1. You run `apt install nginx` on a brand-new server and get "Unable to locate package". What did you forget?
2. Difference between `remove` and `purge`?
3. In `dpkg -l`, what does status `rc` mean?
4. Why prefer `apt-get` inside scripts?
5. How do you stop `mysql-server` from being upgraded?

Answers: 1) `sudo apt update` 2) purge also deletes config files 3) Removed, but config files remain 4) Its output format is stable 5) `sudo apt-mark hold mysql-server`

CHAPTER 12

Ajay Upadhyay · DevOps Mentor

Process Management

Seeing what's running, finding what's slow, and stopping what's stuck.

In this chapter you will learn:

- What a process, PID and parent process are
- Process states (running, sleeping, zombie...)
- `ps` , `pgrep` , `pidof` — and reading `ps aux` column by column
- `top` and `htop` — reading every line of the header
- Signals: `kill` , `pkill` , `killall`
- Foreground, background, `jobs` , `fg` , `bg` , `nohup`
- Priority: `nice` and `renice`

12.1 What Is a Process?

A **program** is a file sitting on disk (like `/usr/sbin/nginx`). A **process** is that program **running** in memory. One program can run as many processes at once (nginx usually runs one "master" and several "worker" processes).

> **Real-Life Example: A Recipe vs Cooking**
> A **recipe book** on the shelf is the program. When a chef actually **cooks** from it, that's a process. Three chefs cooking the same recipe = three processes from one program. Each chef gets a **token number** from the kitchen manager: that's the **PID** (Process ID).

Key facts:

- Every process has a unique **PID** (number).
- Every process is started by another process, its **parent**. The parent's ID is the **PPID**.
- The very first process, **PID 1**, is `systemd` . It's the ancestor of everything.

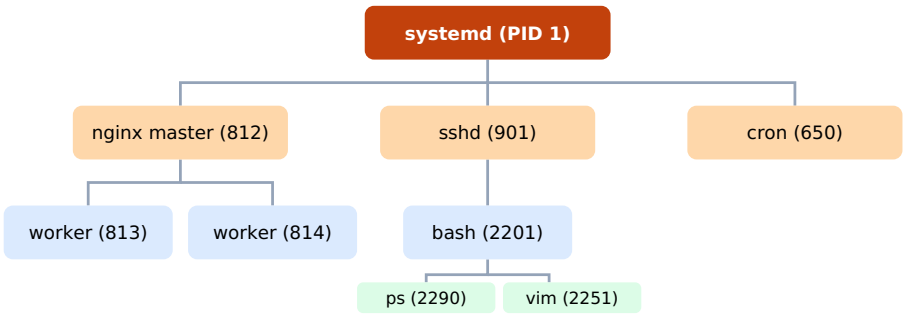


Figure 12.1 — The process tree. See yours with `pstree -p` .

Process states

Code	State	Meaning
R	Running	Using the CPU right now, or ready to
S	Sleeping	Waiting for something (a keypress, a network request). Most processes are here.
D	Uninterruptible sleep	Waiting for disk or network storage. Many D processes = storage problem.
T	Stopped	Paused (e.g. by Ctrl+Z)
Z	Zombie	Finished, but its parent hasn't collected its exit code yet. Takes no CPU or memory; a few are harmless.

12.2 ps — A Snapshot of Processes

```
ajay@ubuntu:~$ ps                    # only processes in THIS terminal
    PID TTY          TIME CMD
   2201 pts/0    00:00:00 bash
   2290 pts/0    00:00:00 ps
ajay@ubuntu:~$ ps aux | head -5       # ALL processes, all users (BSD style)
USER     PID %CPU %MEM    VSZ   RSS TTY   STAT START   TIME COMMAND
root       1  0.0  0.6  22580 13312 ?     Ss   09:12   0:02 /sbin/init
root     650  0.0  0.1   7312  2816 ?     Ss   09:12   0:00 /usr/sbin/cron -f
root     812  0.0  0.1  55228  1664 ?     Ss   09:12   0:00 nginx: master process
www-data 813  0.1  0.3  55860  6400 ?     S    09:12   0:04 nginx: worker process
```

Output Explained: ps aux columns	
USER	Who owns (runs) the process
PID	Process ID number
%CPU	Share of CPU it's using
%MEM	Share of total RAM it's using
VSZ	Virtual memory size in KB (memory it has *reserved*; usually much bigger than real use)
RSS	Resident Set Size in KB: **real RAM actually used**. Look at this one.
TTY	The terminal it belongs to. `?` = no terminal (a background service)
STAT	State (R, S, D, T, Z) plus extras: `s` = session leader, `<` = high priority, `N` = low priority, `+` = in the foreground, `l` = multi-threaded
START	When it started
TIME	Total CPU time used so far (not how long it's been running!)
COMMAND	The command that started it

Useful ps recipes

```
ajay@ubuntu:~$ ps -ef                              # all processes, System V style (shows PPID)
ajay@ubuntu:~$ ps aux --sort=-%mem | head -6       # top 5 memory users
ajay@ubuntu:~$ ps aux --sort=-%cpu | head -6       # top 5 CPU users
ajay@ubuntu:~$ ps -u www-data                      # processes of one user
ajay@ubuntu:~$ ps -o pid,ppid,user,%cpu,%mem,cmd -p 812  # custom columns for one PID
ajay@ubuntu:~$ ps aux | grep [n]ginx               # the [n] trick hides the grep itself
ajay@ubuntu:~$ pstree -p                           # show the tree
```

Finding a PID quickly: pgrep and pidof

```
ajay@ubuntu:~$ pgrep nginx             # PIDs whose name matches
812
813
814
ajay@ubuntu:~$ pgrep -a nginx          # -a shows the full command too
ajay@ubuntu:~$ pgrep -u ajay           # all PIDs of user ajay
ajay@ubuntu:~$ pidof sshd              # exact program name
901
```

12.3 top — The Live Dashboard

`top` refreshes every few seconds and shows the busiest processes at the top. It's the first command to run when "the server is slow".

```
ajay@ubuntu:~$ top
top - 11:42:10 up 3 days,  2:31,  2 users,  load average: 0.52, 0.71, 0.65
Tasks: 142 total,   1 running, 141 sleeping,   0 stopped,   0 zombie
%Cpu(s):  6.2 us,  1.5 sy,  0.0 ni, 91.8 id,  0.3 wa,  0.0 hi,  0.2 si,  0.0 st
MiB Mem :   3915.2 total,    812.4 free,   1430.6 used,   1672.2 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   2246.8 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   1544 mysql     20   0 1781244 402112  35840 S   4.3  10.0  12:04.11 mysqld
    813 www-data  20   0   55860   6400   4352 S   0.7   0.2   0:04.20 nginx
```

Output Explained: the top header, line by line	
`11:42:10 up 3 days`	Current time; the server has been running 3 days without a reboot.
`2 users`	Two login sessions are open.
`load average: 0.52, 0.71, 0.65`	Average number of processes wanting CPU over the last **1, 5 and 15 minutes**. Compare it with your CPU count (`nproc`). On a 2-CPU server, 2.0 = fully busy; above 2.0 = processes are waiting.
`Tasks`	Total processes and how many are in each state. Watch for zombies.
`us / sy`	CPU time on **us**er programs / **sy**stem (kernel) work.
`ni`	CPU used by low-priority ("niced") processes.
`id`	**Idle**. 91.8% idle = the CPU is relaxed.
`wa`	CPU **wa**iting for disk. High wa (e.g. 30%+) = slow disk is the bottleneck, not CPU.
`hi / si`	Time handling hardware / software interrupts.
`st`	**St**olen: on a cloud VM, CPU taken by the host for other customers. High st = noisy neighbours.
`MiB Mem`	RAM. Don't panic at low "free": Linux uses spare RAM as **buff/cache** to speed up the disk, and gives it back when needed. Look at **avail Mem** instead.
`MiB Swap`	Disk used as emergency memory. Heavy swap use = not enough RAM.
`PR / NI`	Priority and nice value (see 12.7).
`VIRT / RES / SHR`	Virtual, **real (resident)**, and shared memory in KB.
`TIME+`	Total CPU time used, to hundredths of a second.

Keys inside top

Key	Action	Key	Action
`P`	Sort by CPU	`M`	Sort by memory
`1`	Show each CPU core separately	`c`	Show full command
`k`	Kill a process (asks for PID)	`u`	Show one user's processes
`r`	Renice (change priority)	`q`	Quit

htop — the friendlier top

```
ajay@ubuntu:~$ sudo apt install htop -y && htop
```

htop shows colour bars for each CPU core and memory, lets you scroll, search (`F3`), show a tree (`F5`), sort (`F6`) and kill (`F9`) using the keys shown at the bottom. Mouse clicks work too.

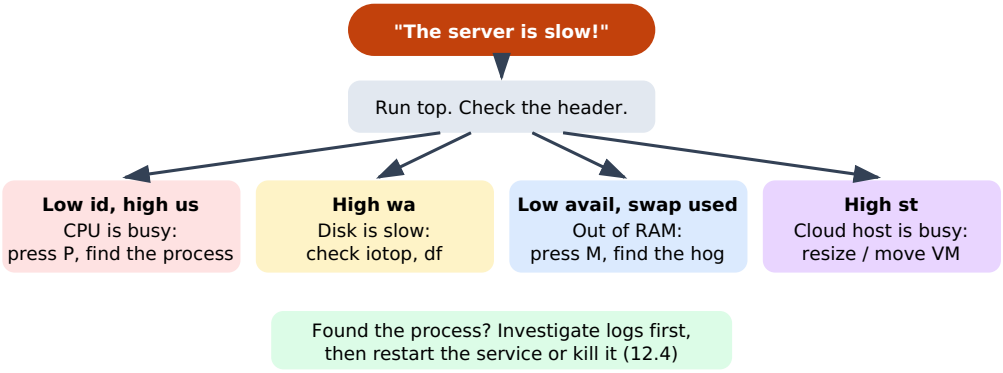


Figure 12.2 — Flowchart: using top to find out why a server is slow.

12.4 Signals and Killing Processes

You control processes by sending them **signals**: short messages like "please stop" or "stop now".

No.	Name	Meaning	When you use it
15	**SIGTERM**	"Please finish up and exit." The program can save its work first.	**Default.** Always try this first.
9	**SIGKILL**	"Die now." The kernel removes it instantly. No clean-up possible.	Only if SIGTERM didn't work
1	SIGHUP	"Hang up." Many services treat it as "reload your config".	Reload nginx, sshd...
2	SIGINT	Interrupt. Same as pressing Ctrl+C.	Stopping a foreground program
19 / 18	SIGSTOP / SIGCONT	Pause / resume	Temporarily freeze a heavy job

Real-Life Example: **SIGTERM** is a manager saying "the office closes in 10 minutes, please save your work and leave". **SIGKILL** is the security guard cutting the electricity. Unsaved work is lost, and files may be left half-written.

```
ajay@ubuntu:~$ kill 2451              # send SIGTERM (15) to PID 2451
ajay@ubuntu:~$ kill -9 2451           # send SIGKILL if it ignored the first one
ajay@ubuntu:~$ kill -HUP 812          # ask nginx master to reload config
ajay@ubuntu:~$ pkill -f "python app.py"   # kill by matching the full command line
ajay@ubuntu:~$ killall node           # kill every process named "node"
ajay@ubuntu:~$ kill -l                # list all signal names and numbers
```

Careful! Don't make `kill -9` your first move. A database killed with -9 can be left in a corrupted state. Use plain `kill`, wait 10–30 seconds, check with `ps`, and only then use `-9`. For services, prefer `systemctl stop` (Chapter 13).

12.5 Foreground, Background and Jobs

A **foreground** job takes over your terminal until it finishes. A **background** job runs while you keep typing.

```
ajay@ubuntu:~$ sleep 300 &              # & starts it in the background
[1] 3012                                 # [job number] PID
ajay@ubuntu:~$ sleep 600                # foreground... terminal is busy
^Z                                       # Ctrl+Z pauses it
[2]+  Stopped                 sleep 600
ajay@ubuntu:~$ jobs                     # list my jobs
[1]-  Running                 sleep 300 &
[2]+  Stopped                 sleep 600
ajay@ubuntu:~$ bg %2                    # resume job 2 in the background
ajay@ubuntu:~$ fg %1                    # bring job 1 to the foreground
ajay@ubuntu:~$ kill %2                  # kill by job number
```

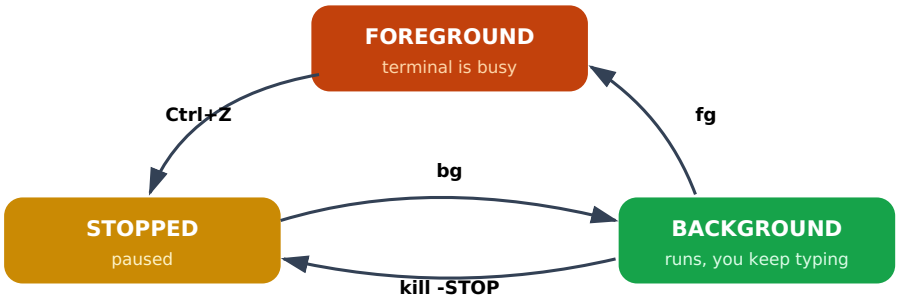


Figure 12.3 — Moving jobs between foreground, stopped and background.

Keeping jobs alive after you log out: nohup

When you close an SSH session, jobs started in it normally die (they receive SIGHUP). `nohup` makes a command ignore that signal.

```
ajay@ubuntu:~$ nohup ./long-backup.sh > backup.out 2>&1 &
[1] 3120
ajay@ubuntu:~$ exit                          # the backup keeps running on the server
```

> **Tip:** For long interactive work over SSH, use `tmux` or `screen`. They keep your whole terminal session alive on the server; reconnect later with `tmux attach`. For anything that must run permanently, make it a **systemd service** (next chapter).

12.6 Priority: nice and renice

When many processes want the CPU, Linux uses a **nice value** to decide who goes first. It ranges from **-20** (highest priority, "not nice to others") to **19** (lowest priority, "very nice"). Default is **0**.

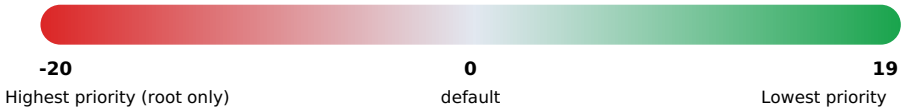


Figure 12.4 — The nice scale. Normal users can only make processes "nicer" (higher number).

```
ajay@ubuntu:~$ nice -n 15 tar czf backup.tar.gz /srv/data   # start a backup at low priority
ajay@ubuntu:~$ sudo renice -n 10 -p 1544                    # lower priority of a running PID
1544 (process ID) old priority 0, new priority 10
ajay@ubuntu:~$ ps -o pid,ni,cmd -p 1544                     # check the NI column
```

> **On the Job:** A nightly report script makes the website slow every evening. Fix: run it with `nice -n 19 ionice -c3 ./report.sh`. `nice` lowers its CPU priority and `ionice -c3` lets it use the disk only when nobody else needs it.

> **Teaching Note:** Have the learner start `yes > /dev/null &` (which uses 100% of one CPU) in one terminal, then watch it in `top`, renice it, and finally kill it. Seeing the CPU number jump and fall makes processes real.

Practice Lab 12

1. Run `ps aux | wc -l`. How many processes are running?
2. Find the 3 processes using the most memory.
3. Run `pstree -p` and find PID 1. What is it called?
4. Start `yes > /dev/null &`. Find it in `top`. What is its %CPU? Renice it to 19. Kill it with SIGTERM.
5. Start `sleep 1000`, pause it with Ctrl+Z, move it to the background with `bg`, list it with `jobs`, bring it back with `fg`, stop with Ctrl+C.
6. Check your load average with `uptime` and your CPU count with `nproc`. Is the server busy?
7. In top, read the header and write down us, id, wa and avail Mem.

Chapter 12 Summary

- A process is a running program with a PID and a parent (PPID). PID 1 = systemd.
- `ps aux`, `ps -ef`, `pgrep`, `pidof`, `pstree`. RSS = real memory.
- `top` / `htop`: watch load average vs CPU count, id, wa, avail Mem.
- `kill` (SIGTERM) first, `kill -9` last. `pkill`, `killall` by name.
- `&`, Ctrl+Z, `jobs`, `bg`, `fg`, `nohup`. `nice` / `renice`: -20 high to 19 low.

Quick Quiz

1. Which signal does plain `kill PID` send?
2. A 4-core server has load average 7.5. What does that mean?
3. top shows 30 MB "free" but 2.5 GB "avail Mem". Is the server out of memory?
4. What does state `Z` mean?
5. How do you start `backup.sh` so it survives logging out?

Answers: 1) SIGTERM (15) 2) Overloaded: about 3.5 processes are always waiting for CPU 3) No, Linux is using spare RAM as cache; 2.5 GB is available 4) Zombie 5) `nohup ./backup.sh > out.log 2>&1 &`

CHAPTER 13

Ajay Upadhyay · DevOps Mentor

Services and systemd

Starting, stopping and auto-starting services — and turning your own app into one.

In this chapter you will learn:

- What a service (daemon) is and what systemd does
- `systemctl` : start, stop, restart, reload, enable, disable, status
- Reading `systemctl status` output line by line
- Writing your own `.service` unit file, step by step
- Targets (the modern runlevels) and the boot process
- systemd timers (the modern cron)

13.1 Services and systemd

A **service** (also called a **daemon**) is a program that runs in the background, usually from boot until shutdown, waiting to do its job: a web server (nginx), a database (mysql), remote login (ssh), job scheduler (cron). Daemon names often end in **d**: `sshd` , `systemd` , `containerd` .

systemd is the program with PID 1. It starts everything at boot, keeps services running, restarts them if they crash, and collects their logs.

> **Real-Life Example: The Hotel Manager**
> systemd is a hotel's **general manager**. Each **service** is a department: reception, kitchen, housekeeping. The manager opens each department in the right order every morning (boot), checks they're working, replaces a worker who walks out (auto-restart), and keeps a logbook of everything that happens (the journal).

Units: the things systemd manages

Unit type	What it is	Example
`.service`	A program/daemon	`nginx.service`
`.timer`	A schedule that starts a service	`logrotate.timer`
`.target`	A group of units = a system "mode"	`multi-user.target`
`.socket`	Starts a service when a connection arrives	`ssh.socket`
`.mount`	A mounted filesystem	`home.mount`

13.2 systemctl — Controlling Services

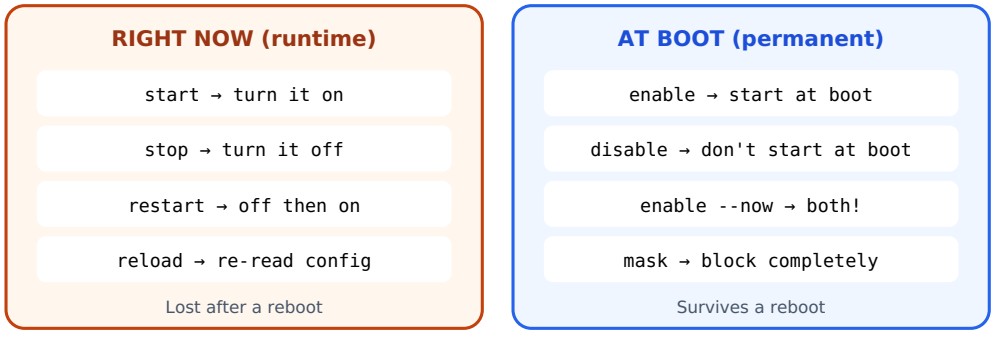


Figure 13.1 — start/stop affect now; enable/disable affect the next boot. They are independent!

```
ajay@ubuntu:~$ sudo systemctl start nginx          # start now
ajay@ubuntu:~$ sudo systemctl stop nginx           # stop now
ajay@ubuntu:~$ sudo systemctl restart nginx        # full restart (brief downtime)
ajay@ubuntu:~$ sudo systemctl reload nginx         # reload config, no downtime
ajay@ubuntu:~$ sudo systemctl enable nginx         # start automatically at boot
ajay@ubuntu:~$ sudo systemctl enable --now nginx   # enable AND start in one go
ajay@ubuntu:~$ sudo systemctl disable --now nginx  # disable AND stop
ajay@ubuntu:~$ systemctl is-active nginx           # quick yes/no answers (great in scripts)
active
ajay@ubuntu:~$ systemctl is-enabled nginx
enabled
ajay@ubuntu:~$ systemctl list-units --type=service --state=running   # what's running?
ajay@ubuntu:~$ systemctl --failed                  # what has crashed?
ajay@ubuntu:~$ systemctl list-unit-files --type=service            # all services + enabled state
```

Careful! Restart vs reload. `restart` kills and re-starts the service, dropping active connections. `reload` keeps it running and just reads the new config. Use reload when the service supports it (nginx, apache, sshd). And always test the config first: `sudo nginx -t && sudo systemctl reload nginx`.

13.3 Reading systemctl status

```
ajay@ubuntu:~$ systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-10-13 09:12:44 IST; 2h 31min ago
       Docs: man:nginx(8)
    Process: 810 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 812 (nginx)
      Tasks: 3 (limit: 4558)
     Memory: 4.1M (peak: 6.2M)
        CPU: 98ms
     CGroup: /system.slice/nginx.service
             ├─812 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             ├─813 "nginx: worker process"
             └─814 "nginx: worker process"

Oct 13 09:12:44 ubuntu systemd[1]: Starting nginx.service - A high performance web server...
Oct 13 09:12:44 ubuntu systemd[1]: Started nginx.service - A high performance web server...
```

Output Explained: systemctl status	
● (dot colour)	**Green** = running, **white** = stopped, **red** = failed.
Loaded: ... ; enabled	Where the unit file is, and whether it starts at boot (enabled/disabled). "preset" is Ubuntu's default choice.
Active: active (running)	**The most important line.** Other values: `inactive (dead)` = stopped; `failed` = crashed; `activating` = still starting.
since ... 2h 31min ago	How long it has been up. A very recent time on a service you didn't touch = it may be crash-looping.
Process: ... ExecStartPre	A check run before starting (here, a config test). `status=0/SUCCESS` = passed.
Main PID: 812	The main process. Use it with `ps` or `kill`.
Tasks / Memory / CPU	Resources used by the whole service.
CGroup	All processes belonging to the service, shown as a tree.
Last lines	The newest log lines for this service. For more: `journalctl -u nginx`.

When a service fails

```
ajay@ubuntu:~$ systemctl status nginx
× nginx.service - A high performance web server and a reverse proxy server
     Active: failed (Result: exit-code) since Tue 2026-10-13 11:50:02 IST; 5s ago
    Process: 4410 ExecStartPre=/usr/sbin/nginx -t -q ... (code=exited, status=1/FAILURE)
Oct 13 11:50:02 ubuntu nginx[4410]: nginx: [emerg] unexpected "}" in /etc/nginx/sites-enabled/myapp:7
```

The log line tells you exactly what's wrong: a typo in line 7 of the `myapp` config. Fix it, run `sudo nginx -t`, then `sudo systemctl start nginx`.

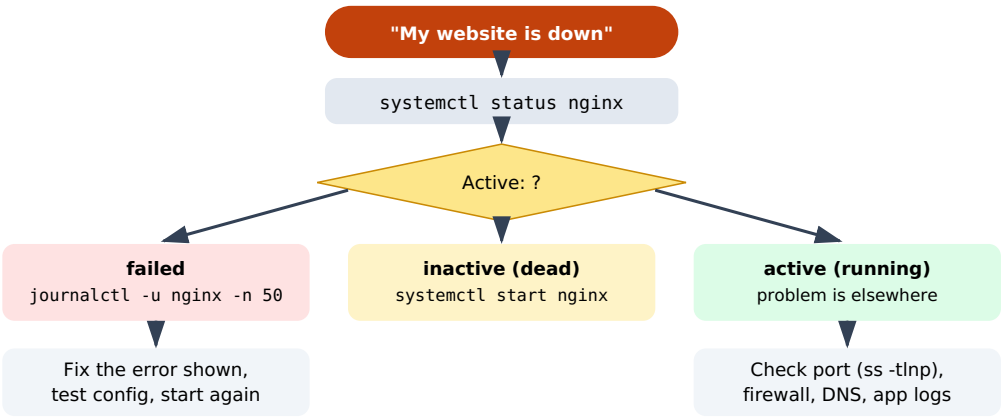


Figure 13.2 — Flowchart: troubleshooting a service that seems down.

13.4 Writing Your Own Service (Unit File)

Say you have a Python web app at `/opt/myapp/app.py`. Running it by hand dies when you log out and never restarts after a crash or reboot. Let's make systemd look after it.

Where unit files live

Folder	Used for
`/usr/lib/systemd/system/`	Files installed by packages. **Don't edit** (updates overwrite them).
`/etc/systemd/system/`	**Your own** unit files and overrides. Wins over the folder above.

Step 1: Create a dedicated user (security)

```
ajay@ubuntu:~$ sudo useradd --system --no-create-home --shell /usr/sbin/nologin myapp
ajay@ubuntu:~$ sudo chown -R myapp:myapp /opt/myapp
```

Step 2: Write the unit file

```
ajay@ubuntu:~$ sudo nano /etc/systemd/system/myapp.service
```

```
[Unit]
Description=My Python Web App
Documentation=https://github.com/ajay/myapp
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=myapp
Group=myapp
WorkingDirectory=/opt/myapp
Environment="PORT=5000"
EnvironmentFile=-/etc/myapp/env
ExecStart=/usr/bin/python3 /opt/myapp/app.py
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Unit File Explained, line by line	
`[Unit]`	General info and ordering.
`Description`	The name shown in `systemctl status`.
`After=`	Start **after** the network is up. (Ordering only.)
`Wants=`	Also try to start network-online.target. (Dependency.)
`[Service]`	How to run the program.
`Type=simple`	The program stays in the foreground; systemd treats it as the main process. (Use `forking` for old-style daemons, `oneshot` for scripts that run and finish.)
`User= / Group=`	Run as this user, **not root**. Much safer.
`WorkingDirectory`	The folder the app starts in.
`Environment=`	Set environment variables.
`EnvironmentFile=-`	Load variables (like secrets) from a file. The `-` means "don't fail if the file is missing".
`ExecStart`	The exact command. **Must be a full path.**
`Restart=on-failure`	If it crashes, start it again. (`always` = restart even after a clean exit.)
`RestartSec=5`	Wait 5 seconds before restarting.
`[Install]`	What `enable` does.
`WantedBy=multi-user.target`	Start it during a normal boot.

Step 3: Load, start, enable, check

```
ajay@ubuntu:~$ sudo systemctl daemon-reload         # REQUIRED after creating/editing any unit file
ajay@ubuntu:~$ sudo systemctl enable --now myapp
Created symlink /etc/systemd/system/multi-user.target.wants/myapp.service → /etc/systemd/system/myapp.service.
ajay@ubuntu:~$ systemctl status myapp
ajay@ubuntu:~$ journalctl -u myapp -f               # follow its logs live
```

> **Notice the symlink!** "enable" simply creates a **soft link** (Chapter 4) inside `multi-user.target.wants/`. At boot, systemd starts everything linked there. "disable" just removes that link.

Changing a package's service safely: overrides

```
ajay@ubuntu:~$ sudo systemctl edit nginx        # opens an empty override file
# add, for example:
[Service]
Restart=always
LimitNOFILE=65535
ajay@ubuntu:~$ systemctl cat nginx              # shows original + your override
```

This saves to `/etc/systemd/system/nginx.service.d/override.conf`, which survives package upgrades.

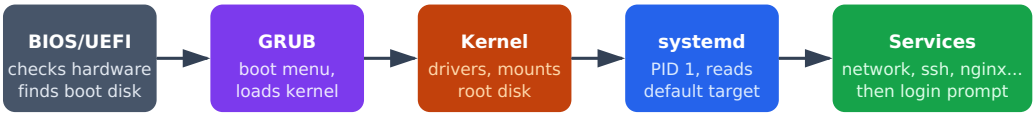
13.5 Targets and the Boot Process

A **target** is a group of units that defines a system "mode". They replace the old numbered **runlevels**.

Target	Old runlevel	Meaning
`poweroff.target`	0	Shut down
`rescue.target`	1	Single-user repair mode
`multi-user.target`	3	Normal server: networking, no graphics
`graphical.target`	5	Desktop with graphics
`reboot.target`	6	Restart

```
ajay@ubuntu:~$ systemctl get-default                        # current default mode
graphical.target
ajay@ubuntu:~$ sudo systemctl set-default multi-user.target  # boot without the desktop
ajay@ubuntu:~$ sudo systemctl reboot                        # or: poweroff, suspend
```

What happens when Ubuntu boots



See how long each step took: `systemd-analyze` and `systemd-analyze blame`

Figure 13.3 — The Linux boot sequence.

```
ajay@ubuntu:~$ systemd-analyze
Startup finished in 3.120s (kernel) + 9.845s (userspace) = 12.966s
ajay@ubuntu:~$ systemd-analyze blame | head -3      # slowest services at boot
5.210s cloud-init.service
2.012s snapd.service
1.140s systemd-networkd-wait-online.service
```

13.6 systemd Timers (Modern Scheduled Jobs)

A timer starts a service on a schedule, like cron (Chapter 16), but with logging in the journal, catch-up for missed runs, and randomised delays built in. You need **two** files: a `.service` (what to do) and a `.timer` (when).

```
# /etc/systemd/system/db-backup.service
[Unit]
Description=Database backup
[Service]
Type=oneshot
ExecStart=/usr/local/bin/db-backup.sh

# /etc/systemd/system/db-backup.timer
[Unit]
Description=Run database backup every night at 2 AM
[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true
RandomizedDelaySec=10m
[Install]
WantedBy=timers.target
```

```
ajay@ubuntu:~$ sudo systemctl daemon-reload
ajay@ubuntu:~$ sudo systemctl enable --now db-backup.timer     # enable the TIMER, not the service
ajay@ubuntu:~$ systemctl list-timers
NEXT                        LEFT     LAST  PASSED  UNIT            ACTIVATES
Wed 2026-10-14 02:04:51 IST 14h left -     -       db-backup.timer  db-backup.service
ajay@ubuntu:~$ systemd-analyze calendar "Mon..Fri 09:00"     # test a schedule expression
```

OnCalendar value	Runs
`hourly` / `daily` / `weekly`	Start of every hour / day (midnight) / Monday
`*-*-* 02:00:00`	Every day at 2:00 AM
`Mon..Fri 09:00`	Weekdays at 9 AM
`*:0/15`	Every 15 minutes
`*-*-01 00:00:00`	The 1st of every month

Persistent=true means: if the server was off at 2 AM, run the job as soon as it boots. Plain cron would just skip it.

Practice Lab 13

1. Install nginx. Check `systemctl status nginx` and write down Loaded, Active and Main PID.
2. Stop it, check status, start it again. Disable it, reboot, and confirm it didn't start. Re-enable it.
3. Break nginx on purpose: add a line `oops;` inside `/etc/nginx/nginx.conf`. Restart and read the error in status and `journalctl -u nginx`. Fix it.
4. Create a script `/usr/local/bin/hello.sh` that loops forever printing the date every 10 seconds. Write `hello.service` for it, enable it, and follow its logs.
5. Kill its main PID with `kill -9`. Does systemd restart it? (Check `Restart=`.)
6. Create a timer that runs `logger "timer works"` every 5 minutes. Find the message with `journalctl -t`.
7. Run `systemd-analyze blame`. Which service slows your boot the most?

Chapter 13 Summary

- systemd (PID 1) starts and supervises services (units).
- `start/stop/restart/reload` = now. `enable/disable` = at boot. `enable --now` = both.
- `systemctl status`: read Loaded and Active first, then the log lines.
- Own units in `/etc/systemd/system/`, then `daemon-reload`. Use `User=`, `Restart=on-failure`.
- Targets = modes (multi-user, graphical). Timers = modern scheduled jobs.

Quick Quiz

1. You started a service but it's gone after a reboot. What did you forget?
2. You edited a unit file but nothing changed. What command is missing?
3. What is the difference between `restart` and `reload`?
4. Which line in a unit file makes a crashed service come back?
5. Which target is a normal server without a desktop?

Answers: 1) `systemctl enable` 2) `sudo systemctl daemon-reload` 3) restart stops and starts (drops connections); reload re-reads config while running 4) `Restart=on-failure` (or always) 5) `multi-user.target`

CHAPTER 14

Ajay Upadhyay · DevOps Mentor

Logs and Monitoring

Where Linux writes down what happened, and how to read it fast.

In this chapter you will learn:

- The two logging systems on Ubuntu: the systemd journal and text files in /var/log
- `journalctl` with all the filters you need
- Important files in `/var/log` and what they contain
- How rsyslog works
- Keeping logs from filling the disk with `logrotate`
- `dmesg` for kernel and hardware messages

14.1 Why Logs Matter

Logs are the server's **diary**. Every login, error, crash, and service start is written down with a timestamp. When something breaks, logs are usually the fastest way to find out **what** happened, **when**, and **why**.

Real-Life Example: An Aeroplane's Black Box
After an incident, investigators don't guess; they read the flight recorder. Logs are your server's black box. A good DevOps engineer's first question is always: **"What do the logs say?"**

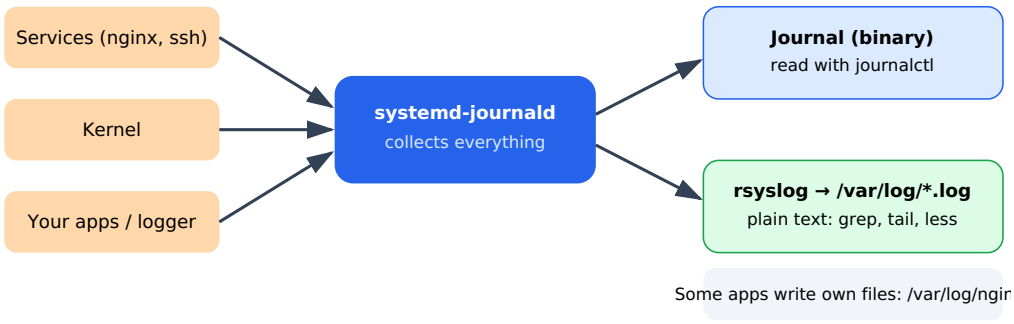


Figure 14.1 — How logs flow on Ubuntu. Most messages end up in both the journal and a text file.

14.2 journalctl — Reading the systemd Journal

Command	Shows
`journalctl`	Everything, oldest first (huge! press q to quit)
`journalctl -e`	Jump to the **end** (newest)
`journalctl -f`	**Follow** live, like `tail -f`
`journalctl -n 50`	Last 50 lines
`journalctl -u nginx`	Only one service (**u**nit) — used most
`journalctl -u nginx -u mysql`	Two services mixed together in time order
`journalctl -p err`	Only errors and worse (priority)
`journalctl -b`	Since the current boot
`journalctl -b -1`	The **previous** boot (why did it crash/reboot?)
`journalctl --since "1 hour ago"`	Time window
`journalctl --since "2026-10-13 09:00" --until "2026-10-13 10:00"`	Exact time range
`journalctl -k`	Kernel messages only
`journalctl -t sshd`	By tag (program name)
`journalctl -o json-pretty -n 1`	Structured output (for tools)
`journalctl --disk-usage`	How much space the journal uses
`sudo journalctl --vacuum-time=7d`	Delete journal entries older than 7 days

```
ajay@ubuntu:~$ journalctl -u ssh --since today -p warning
Oct 13 10:14:02 ubuntu sshd[3301]: Failed password for invalid user admin from 45.83.12.9 port 51122 ssh2
Oct 13 10:14:05 ubuntu sshd[3301]: Failed password for invalid user admin from 45.83.12.9 port 51122 ssh2
```

Output Explained: a journal line	
`Oct 13 10:14:02`	Date and time of the event
`ubuntu`	Hostname (which machine)
`sshd[3301]`	Program name and its PID
`Failed password...`	The message. Here: someone tried to log in as "admin" (a user that doesn't exist) from IP 45.83.12.9. This is a typical internet bot attack.

Log priority levels

No.	Name	Meaning
0	emerg	System is unusable
1	alert	Action needed immediately
2	crit	Critical condition
3	**err**	Error
4	**warning**	Something may go wrong
5	notice	Normal but important
6	info	Information
7	debug	Very detailed, for developers

`-p err` shows levels 0–3 (err and everything more serious).

Tip: Keep the journal after reboots. On most Ubuntu installs the journal is already saved to disk in `/var/log/journal/`. If `journalctl -b -1` says there's no previous boot, create that folder (`sudo mkdir -p /var/log/journal`) and restart `systemd-journald`.

14.3 The Important Files in /var/log

File	Contains	Read it when...
`/var/log/syslog`	General system messages from most services	Anything odd happens
`/var/log/auth.log`	Logins, sudo use, SSH attempts, user changes	Security checks, "who did that?"
`/var/log/kern.log`	Kernel messages	Hardware, driver, disk errors
`/var/log/dpkg.log`	Package installs, upgrades and removals	"What changed on this server?"
`/var/log/apt/history.log`	Every apt command with date	Same, more readable
`/var/log/nginx/access.log`	Every web request	Traffic analysis
`/var/log/nginx/error.log`	Web server errors	500/502 errors
`/var/log/cloud-init.log`	First-boot setup on cloud VMs	A new cloud server isn't configured right
`/var/log/wtmp`, `btmp`	Login records (binary)	Read with `last` and `sudo lastb`

```
ajay@ubuntu:~$ sudo grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -rn | head -3
     48 45.83.12.9
     12 103.21.44.7
      3 192.168.1.50
ajay@ubuntu:~$ sudo grep "sudo:" /var/log/auth.log | tail -3        # recent sudo usage
ajay@ubuntu:~$ last -n 5                                            # last 5 logins
ajay@ubuntu:~$ grep " install " /var/log/dpkg.log | tail -5         # recently installed packages
```

The awk trick `$(NF-3)`: `NF` is the number of fields, so `$NF` is the last column and `$(NF-3)` is the fourth from the end. In "...from 45.83.12.9 port 51122 ssh2", counting back from "ssh2" lands on the IP. Useful when line lengths vary.

14.4 rsyslog — The Text Log Writer

`rsyslog` receives messages and writes them to files based on rules in `/etc/rsyslog.conf` and `/etc/rsyslog.d/*.conf`. A rule has two parts: **which messages** (facility.priority) and **where** to put them.

```
ajay@ubuntu:~$ cat /etc/rsyslog.d/50-default.conf | grep -v "^#" | grep -v "^$"
auth,authpriv.*                 /var/log/auth.log
*.*;auth,authpriv.none          -/var/log/syslog
kern.*                          -/var/log/kern.log
mail.*                          -/var/log/mail.log
```

Output Explained: rsyslog rules	
`auth,authpriv.*`	Messages from the "auth" sources, any priority (*) → auth.log
`*.*;auth,authpriv.none`	Everything, **except** auth → syslog (so passwords info stays out of syslog)
`-/var/log/...`	The `-` means "don't force a disk write after every line" (faster)

```
ajay@ubuntu:~$ logger "Deployment v2.3 started"           # write your own message to the logs
ajay@ubuntu:~$ logger -t deploy -p user.warning "Disk at 85%"
ajay@ubuntu:~$ tail -2 /var/log/syslog
Oct 13 12:01:10 ubuntu ajay: Deployment v2.3 started
Oct 13 12:01:15 ubuntu deploy: Disk at 85%
```

> **On the Job:** In production, logs from many servers are usually shipped to one central place (ELK/OpenSearch, Grafana Loki, Datadog, CloudWatch). rsyslog can forward with one line: `*.* @@logserver.example.com:514` (`@@` = TCP, `@` = UDP). You'll go deeper into this at Level 4.

14.5 logrotate — Stop Logs Filling the Disk

Logs grow forever unless something trims them. **logrotate** runs daily: it renames the current log, starts a fresh one, compresses old ones, and deletes the oldest.



Every day, each file moves one step right. "rotate 7" keeps 7 old copies.

Figure 14.2 — Log rotation, like a conveyor belt.

Rules live in `/etc/logrotate.conf` (defaults) and `/etc/logrotate.d/` (one file per app). Here's one for our app:

```
ajay@ubuntu:~$ sudo nano /etc/logrotate.d/myapp
/var/log/myapp/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 myapp adm
    postrotate
        systemctl reload myapp >/dev/null 2>&1 || true
    endscript
}
```

logrotate Options Explained	
`daily`	Rotate every day (or weekly, monthly, or `size 100M`)
`rotate 14`	Keep 14 old files, delete older ones
`compress`	gzip old logs to save space
`delaycompress`	Wait one cycle before compressing (the app may still be finishing writes to .1)
`missingok`	No error if the log file is missing
`notifempty`	Don't rotate an empty file
`create 0640 myapp adm`	Create the new empty log with these permissions and owner
`postrotate ... endscript`	Command to run after rotating, so the app starts writing to the new file

```
ajay@ubuntu:~$ sudo logrotate -d /etc/logrotate.d/myapp     # -d = dry run, shows what WOULD happen
ajay@ubuntu:~$ sudo logrotate -f /etc/logrotate.d/myapp     # -f = force a rotation now
```

14.6 dmesg — Kernel and Hardware Messages

`dmesg` shows the kernel's ring buffer: messages about hardware, drivers, disks, USB devices, network cards and memory problems.

```
ajay@ubuntu:~$ sudo dmesg -T | tail -5          # -T = human-readable time
ajay@ubuntu:~$ sudo dmesg -T --level=err,warn     # only problems
ajay@ubuntu:~$ sudo dmesg -w                      # watch live (plug in a USB to see it)
ajay@ubuntu:~$ sudo dmesg -T | grep -i "out of memory"
[Tue Oct 13 03:12:44 2026] Out of memory: Killed process 1544 (mysqld) total-vm:1781244kB, anon-rss:3402112kB
```

Output Explained: the OOM Killer	
`Out of memory`	The server ran out of RAM and swap.
`Killed process 1544 (mysqld)`	To save the system, the kernel's **OOM killer** chose the biggest process and killed it. This is why "MySQL just died at 3 AM with no error in its own log".
`anon-rss:3402112kB`	It was using about 3.4 GB of real memory.

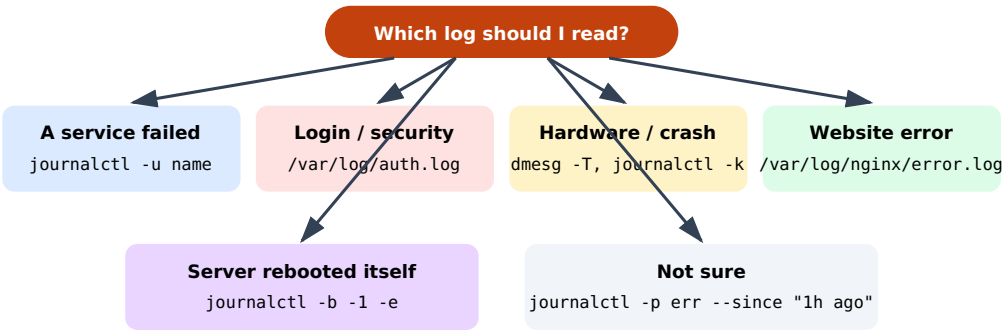


Figure 14.3 — Flowchart: picking the right log for the problem.

14.7 Quick Health Monitoring Commands

Command	Shows
`uptime`	Up time and load average
`free -h`	Memory and swap (look at "available")
`df -h`	Disk space per filesystem (Chapter 17)
`vmstat 2 5`	CPU, memory, swap and I/O every 2 seconds, 5 times
`watch -n 2 'df -h /'`	Re-run any command every 2 seconds
`ss -tulpn`	Which ports are open and which program owns them

```
ajay@ubuntu:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.8Gi       1.4Gi       812Mi        12Mi       1.6Gi       2.2Gi
Swap:          2.0Gi          0B       2.0Gi
```

Output Explained: free -h	
`total`	All installed RAM
`used`	Memory used by programs
`free`	Completely unused memory (often small, and that's fine)
`buff/cache`	Memory Linux borrowed to speed up disk reads. Given back when programs need it.
`available`	**The number that matters:** how much memory new programs can really get.
`Swap used`	0B is ideal. Growing swap usage means memory pressure.

Teaching Note: Run `journalctl -f` in one window and do things in another: `sudo ls`, a wrong sudo password, `systemctl restart cron`, `logger hello`. Seeing each action appear live makes logs click instantly.

Practice Lab 14

1. Show only today's error-level messages from the journal.
2. Show the last 20 log lines of the ssh service.
3. In one terminal run `journalctl -f`; in another type a wrong sudo password. Find the message, then find it again in `/var/log/auth.log`.
4. Use `logger -t mytest "hello logs"` and find it with `journalctl -t mytest`.
5. List all packages installed today from `/var/log/dpkg.log`.
6. Check the journal's disk usage. Clean entries older than 14 days.
7. Write a logrotate rule for `/var/log/lab/*.log`: weekly, keep 4, compress. Test it with `-d`.
8. Run `free -h` and `uptime`. Is your server healthy? Explain using the numbers.

Chapter 14 Summary

- journald collects logs (`journalctl`); rsyslog writes text files in `/var/log`.
- journalctl filters: `-u -f -n -p -b -k --since --until -t`.
- Key files: syslog, auth.log, kern.log, dpkg.log, nginx/*.log.
- logrotate prevents full disks: daily/rotate/compress/postrotate. Test with `-d`.
- `dmesg -T` for kernel and hardware issues, including OOM kills.

Quick Quiz

1. How do you watch nginx's journal live?
2. The server rebooted unexpectedly overnight. Which command shows logs from before the reboot?
3. Which file shows who used sudo?
4. What does the logrotate option `rotate 7` do?
5. MySQL disappeared at 3 AM with nothing in its own log. Where do you look?

Answers: 1) `journalctl -u nginx -f` 2) `journalctl -b -1` 3) `/var/log/auth.log` 4) Keeps 7 old log files 5) `dmesg -T | grep -i "out of memory"` (the OOM killer)

CHAPTER 15

Ajay Upadhyay · DevOps Mentor

Archiving and Compression

Bundling files together and making them smaller — for backups, transfers and releases.

In this chapter you will learn:

- The difference between archiving and compressing
- `tar` : create, list, extract, and its options
- `gzip` , `bzip2` , `xz` and which one to choose
- `zip` and `unzip` for Windows-friendly files
- A real backup script

15.1 Archiving vs Compressing

	Archiving	Compressing
What it does	Joins many files and folders into **one file**	Makes a file **smaller**
Size change	Same size (slightly bigger)	Smaller
Tool	`tar`	`gzip` , `bzip2` , `xz`
Extension	`.tar`	`.gz` , `.bz2` , `.xz`

Real-Life Example: Moving House

Archiving is putting all your things into **one big box** so nothing gets lost. **Compressing** is using a **vacuum bag** to squeeze your clothes so the box takes less space. A `.tar.gz` file is a box with vacuum-packed contents: both at once.



tar -czf does both steps in one command.

Figure 15.1 — From many files to one small file.

15.2 tar — The Tape Archiver

The name comes from "tape archive" (backups used to go to magnetic tape). The letters are easy to remember:

Letter	Meaning	Memory trick
c	Create an archive	**c**reate
x	Extract an archive	e**x**tract
t	List contents	**t**able of contents
f	File name follows	**f**ile (always put it last!)
v	Verbose: show each file	**v**ery talkative
z	Compress with gzip (.tar.gz / .tgz)	**z**ip it (gz)
j	Compress with bzip2 (.tar.bz2)	—
J	Compress with xz (.tar.xz)	capital J = extra squeeze
C dir	Extract into a different folder	**C**hange directory

The three commands you'll use 90% of the time

```
ajay@ubuntu:~$ tar -czvf website.tar.gz /var/www/html        # CREATE a compressed archive
tar: Removing leading `/' from member names
/var/www/html/
/var/www/html/index.html
/var/www/html/css/style.css
ajay@ubuntu:~$ tar -tzvf website.tar.gz                      # LIST what's inside (without extracting)
drwxr-xr-x root/root         0 2026-10-13 09:00 var/www/html/
-rw-r--r-- root/root       612 2026-10-13 09:00 var/www/html/index.html
ajay@ubuntu:~$ tar -xzvf website.tar.gz -C /tmp/restore     # EXTRACT into /tmp/restore
```

Output Explained	
`Removing leading `/'`	Not an error. tar stores `var/www/html` instead of `/var/www/html`, so extracting can never accidentally overwrite your real system folders. Files land wherever you extract.
`drwxr-xr-x root/root`	When listing, tar shows permissions and owner, just like `ls -l`. tar keeps these.

More tar recipes

```
ajay@ubuntu:~$ tar -czf backup-$(date +%F).tar.gz ~/projects        # date in the name: backup-2026-10-13.tar.gz
ajay@ubuntu:~$ tar -czf app.tar.gz --exclude="node_modules" --exclude="*.log" myapp/
ajay@ubuntu:~$ tar -xzf app.tar.gz myapp/config.yml              # extract just ONE file
ajay@ubuntu:~$ tar -cJf logs.tar.xz /var/log/myapp               # xz = smallest size
ajay@ubuntu:~$ tar -xf anything.tar.*                             # modern tar detects compression itself
ajay@ubuntu:~$ sudo tar -cpzf etc-backup.tar.gz /etc              # -p preserves permissions
ajay@ubuntu:~$ tar -czf - /srv/data | ssh backup@nas "cat > data.tar.gz"   # stream straight to another server
```

Careful! The -f order trap. `tar -cfz backup.tar.gz dir` fails or creates a file literally named "z", because whatever follows `f` is taken as the file name. Keep `f` as the **last** letter: `-czf`, not `-cfz`.

Careful! Extracting overwrites. Extracting into a folder replaces files with the same name without asking. Use `-t` to look first, and extract into an empty folder with `-C` when unsure.

15.3 gzip, bzip2 and xz

These compress **one file** each. By default they **replace** the original file with the compressed one.

```
ajay@ubuntu:~$ ls -lh big.log
-rw-r--r-- 1 ajay ajay 120M Oct 13 12:30 big.log
ajay@ubuntu:~$ gzip big.log                 # big.log becomes big.log.gz
ajay@ubuntu:~$ ls -lh big.log.gz
-rw-r--r-- 1 ajay ajay 9.4M Oct 13 12:30 big.log.gz
ajay@ubuntu:~$ gunzip big.log.gz            # back to big.log  (or: gzip -d)
ajay@ubuntu:~$ gzip -k big.log              # -k keeps the original too
ajay@ubuntu:~$ gzip -9 big.log              # -1 fastest ... -9 smallest
ajay@ubuntu:~$ zcat big.log.gz | head       # read without decompressing
ajay@ubuntu:~$ zgrep "ERROR" big.log.gz     # grep inside a .gz
ajay@ubuntu:~$ zless /var/log/syslog.2.gz   # browse old rotated logs
```

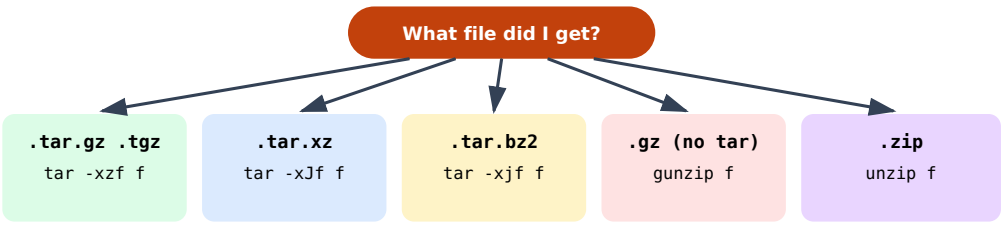
Tool	Ext.	Speed	Size	Decompress	Best for
gzip	.gz	Fast	Good	`gunzip`	Everyday use, logs, web
bzip2	.bz2	Slow	Better	`bunzip2`	Older archives
xz	.xz	Slowest	Best	`unxz`	Releases, long-term storage
zstd	.zst	Very fast	Very good	`unzstd`	Modern backups (`tar --zstd`)

> **Tip:** Text (logs, code, CSV) compresses extremely well, often to 5–10% of its size. Already-compressed files (.jpg, .mp4, .zip) barely shrink, so don't waste CPU compressing them again.

15.4 zip and unzip

Use `zip` when sharing with Windows users. Unlike gzip, zip both archives and compresses, and keeps the originals.

```
ajay@ubuntu:~$ sudo apt install zip unzip -y
ajay@ubuntu:~$ zip -r project.zip myproject/              # -r = include folders
ajay@ubuntu:~$ zip -r project.zip myproject/ -x "*.log"   # exclude logs
ajay@ubuntu:~$ zip -e secret.zip passwords.txt            # -e = password protect
ajay@ubuntu:~$ unzip -l project.zip                       # list contents
ajay@ubuntu:~$ unzip project.zip -d /tmp/project          # extract into a folder
```



Unsure? Run `file name` to see the real type.

Figure 15.2 — Flowchart: how to open any archive.

15.5 Putting It Together: A Real Backup Script

```
ajay@ubuntu:~$ sudo nano /usr/local/bin/backup-www.sh
#!/bin/bash
# Backup /var/www and /etc/nginx, keep 7 days
BACKUP_DIR=/backups
DATE=$(date +%F_%H%M)
mkdir -p "$BACKUP_DIR"

tar -czpf "$BACKUP_DIR/www-$DATE.tar.gz" /var/www /etc/nginx 2>>/var/log/backup-www.log \
  && logger -t backup "OK: www-$DATE.tar.gz" \
  || logger -t backup -p user.err "FAILED: www backup"

find "$BACKUP_DIR" -name "www-*.tar.gz" -mtime +7 -delete
ajay@ubuntu:~$ sudo chmod 755 /usr/local/bin/backup-www.sh
ajay@ubuntu:~$ sudo /usr/local/bin/backup-www.sh && ls -lh /backups
```

Notice how this uses almost everything so far: variables, `$(date)`, `tar`, `2>>`, `&&` / `||`, `logger`, and `find -mtime -delete`. In the next chapter, we'll make it run every night automatically.

Practice Lab 15

1. Create `~/lab15` with a few files and a sub-folder. Archive it to `lab15.tar`. Check the size with `ls -lh`.
2. Make `lab15.tar.gz` and `lab15.tar.xz` too. Compare all three sizes.
3. List the contents of the .tar.gz without extracting.
4. Extract only one file from it into `/tmp/one`.
5. Copy `/var/log/syslog` to your home, compress it with gzip, and compare sizes. Then search it with `zgrep`.
6. Create a zip of `~/lab15` that excludes `.txt` files, and list it with `unzip -l`.
7. Write and run the backup script above. Check the result in `journalctl -t backup`.

Chapter 15 Summary

- tar archives (many → one); gzip/bzip2/xz compress (big → small).
- `tar -czf` create, `tar -tzf` list, `tar -xzf ... -C dir` extract. `f` goes last.
- `z` = gzip, `j` = bzip2, `J` = xz. `--exclude` skips files.
- `zcat`, `zgrep`, `zless` read .gz files directly.
- `zip -r` / `unzip -d` for Windows-friendly archives.

Quick Quiz

1. Write the command to create `logs.tar.gz` from `/var/log/nginx`.
2. How do you see what's in an archive without extracting it?
3. After `gzip report.txt`, does `report.txt` still exist?
4. Which compression gives the smallest files?
5. Why does tar say "Removing leading /"?

Answers: 1) `tar -czf logs.tar.gz /var/log/nginx` 2) `tar -tzf file.tar.gz` 3) No, it's replaced by report.txt.gz (use -k to keep it) 4) xz 5) So extracting can't overwrite system paths; files are stored as relative paths.

CHAPTER 16

Ajay Upadhyay · DevOps Mentor

Task Scheduling

Making the server do jobs by itself: backups, clean-ups, reports and health checks.

In this chapter you will learn:

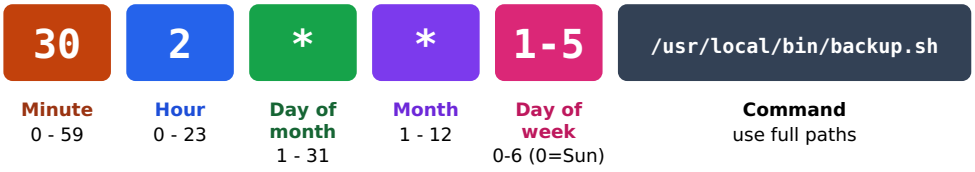
- How cron works and how to read the five time fields
- Editing schedules with `crontab`
- System cron files and the cron.daily folders
- The common cron mistakes (and how to avoid them)
- One-time jobs with `at`
- anacron for computers that aren't always on
- When to choose cron vs systemd timers

16.1 cron — The Repeating Alarm Clock

cron is a service that wakes up every minute, checks its list of jobs, and runs any whose time has come. Each user has their own list called a **crontab** (cron table).

> **Real-Life Example: Phone Alarms**
> You set your phone alarm: "6:30 AM, Monday to Friday". cron is the same, but instead of ringing, it **runs a command**. Each line of a crontab is one alarm.

16.2 The Five Time Fields



Read it as: "At minute 30, hour 2, any day, any month, Monday to Friday"

= Every weekday at 2:30 AM

*Figure 16.1 — The five time fields of a cron line. Memory trick: "**M**y **H**orse **D**rinks **M**ilk **W**eekly".*

Special characters

Symbol	Meaning	Example
`*`	Every value	`*` in hour = every hour
`,`	A list	`0,30` in minute = at :00 and :30
`-`	A range	`9-17` in hour = 9 AM to 5 PM
`/`	Every N steps	`*/15` in minute = every 15 minutes

Examples to learn from

Cron line	Runs
`* * * * *`	Every minute
`*/5 * * * *`	Every 5 minutes
`0 * * * *`	Every hour, on the hour
`0 2 * * *`	Every day at 2:00 AM
`30 18 * * 1-5`	Weekdays at 6:30 PM
`0 9,13,17 * * *`	Daily at 9 AM, 1 PM and 5 PM
`0 0 * * 0`	Every Sunday at midnight
`0 3 1 * *`	1st of every month at 3 AM
`0 0 1 1 *`	Once a year, 1 January at midnight
`*/10 9-18 * * 1-5`	Every 10 minutes during office hours on weekdays
`@reboot`	Once, every time the server starts
`@daily` / `@hourly` / `@weekly`	Shortcuts for common schedules

Tip: Not sure your schedule is right? Paste it into a website like crontab.guru. It explains any cron line in plain English.

16.3 Editing Your Crontab

```
ajay@ubuntu:~$ crontab -e              # edit your own jobs (first time asks which editor)
ajay@ubuntu:~$ crontab -l              # list your jobs
ajay@ubuntu:~$ sudo crontab -e         # edit ROOT's jobs
ajay@ubuntu:~$ sudo crontab -u ravi -l # see another user's jobs
ajay@ubuntu:~$ crontab -r              # DELETES all your jobs! (no confirmation)
```

Careful! `-r` (remove) sits right next to `-e` (edit) on the keyboard, and it deletes everything with no question. Back up first: `crontab -l > ~/crontab-backup.txt` .

Inside the editor, add one job per line:

```
# m  h  dom mon dow  command
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
MAILTO=""

# Nightly website backup at 2:30 AM, log output and errors
30 2 * * * /usr/local/bin/backup-www.sh >> /var/log/backup-www.log 2>&1

# Disk check every 15 minutes
*/15 * * * * /usr/local/bin/check-disk.sh >> /var/log/check-disk.log 2>&1

# Clean temp uploads older than 2 days, every Sunday at 4 AM
0 4 * * 0 find /srv/app/tmp -type f -mtime +2 -delete
```

16.4 System-Wide Cron

Location	Used for
`/etc/crontab`	The main system crontab. Has an extra **user** field after the 5 time fields.
`/etc/cron.d/`	Job files dropped in by packages or by you (same format as /etc/crontab). Great for automation tools.
`/etc/cron.hourly/` `daily/` `weekly/` `monthly/`	Just drop an executable **script** here; no schedule line needed.
`/var/spool/cron/crontabs/`	Where users' personal crontabs are stored. Don't edit directly; use `crontab -e`.

```
# /etc/cron.d/myapp  -- note the extra USER column
*/5 * * * *  myapp  /opt/myapp/bin/sync-cache >> /var/log/myapp/cron.log 2>&1

ajay@ubuntu:~$ sudo cp backup-www.sh /etc/cron.daily/backup-www   # no ".sh"! (see warning)
ajay@ubuntu:~$ sudo chmod 755 /etc/cron.daily/backup-www
ajay@ubuntu:~$ run-parts --test /etc/cron.daily                    # which scripts will run?
```

Careful! Scripts in `/etc/cron.daily/` etc. are run by `run-parts`, which on Ubuntu **silently skips file names containing a dot**. `backup.sh` will never run; name it `backup`.

16.5 Why Cron Jobs Fail (and How to Fix Them)

"It works when I run it by hand but not in cron!" is one of the most common DevOps problems. The reason: cron runs your command in a **very minimal environment**.

Problem	Fix
"command not found"	cron's `PATH` is tiny. Use **full paths** (`/usr/bin/python3`) or set `PATH=` at the top of the crontab.
Relative paths break	cron starts in your home folder, not your project folder. `cd /srv/app && ...` first, or use absolute paths.
No idea what happened	Always redirect output: `>> /var/log/job.log 2>&1`.
`%` in the command	cron treats `%` as a newline. Escape it: `date +\%F`.
Script not executable	`chmod +x script.sh` and start it with `#!/bin/bash`.
Missing environment variables	Your `.bashrc` isn't loaded. Define variables in the script or crontab.
Wrong time	cron uses the server's timezone. Check with `timedatectl`.

```
ajay@ubuntu:~$ grep CRON /var/log/syslog | tail -3          # did cron even start the job?
Oct 13 02:30:01 ubuntu CRON[5120]: (ajay) CMD (/usr/local/bin/backup-www.sh >> /var/log/backup-www.log 2>&1)
ajay@ubuntu:~$ journalctl -u cron --since today
```

Output Explained: a CRON log line	
`CRON[5120]`	cron started a process with PID 5120
`(ajay)`	The job belongs to user ajay
`CMD (...)`	The exact command it ran. If you see this, cron did its job; any problem is inside your script — check its own log.

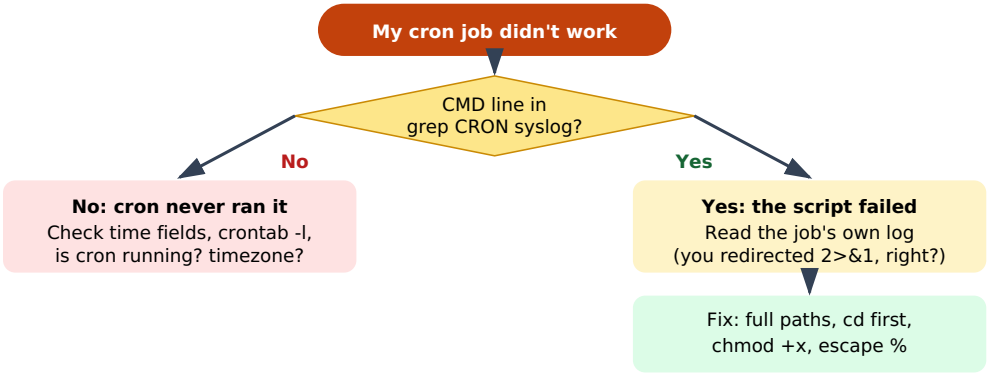


Figure 16.2 — Flowchart: debugging a cron job.

16.6 at — Run Something Once, Later

cron repeats; `at` runs a job **one time** at a future moment.

```
ajay@ubuntu:~$ sudo apt install at -y
ajay@ubuntu:~$ at 23:00
warning: commands will be executed using /bin/sh
at> sudo systemctl restart myapp
at> <Ctrl+D>
job 3 at Tue Oct 13 23:00:00 2026
ajay@ubuntu:~$ echo "/usr/local/bin/backup-www.sh" | at now + 2 hours
ajay@ubuntu:~$ echo "reboot" | sudo at 03:00 tomorrow
ajay@ubuntu:~$ atq                    # list waiting jobs
3       Tue Oct 13 23:00:00 2026 a ajay
ajay@ubuntu:~$ atrm 3                 # cancel job 3
```

> **On the Job: the "safety net" trick.** Before changing firewall rules on a remote server, run `echo "ufw disable" | sudo at now + 10 minutes`. If your change locks you out, the server undoes it in 10 minutes. If everything works, cancel it with `atrm`.

16.7 anacron — For Machines That Sleep

If a laptop is off at 2 AM, cron simply skips that night's job. **anacron** remembers when daily/weekly/monthly jobs last ran and runs any missed ones after the machine starts. On Ubuntu, `/etc/cron.daily`, `weekly` and `monthly` are run through anacron automatically on machines that have it.

```
ajay@ubuntu:~$ cat /etc/anacrontab | grep -v "^#"
1       5       cron.daily      run-parts --report /etc/cron.daily
7       10      cron.weekly     run-parts --report /etc/cron.weekly
@monthly 15     cron.monthly    run-parts --report /etc/cron.monthly
```

Output Explained: /etc/anacrontab	
`1 / 7 / @monthly`	Period in days: run if it hasn't run for this long
`5 / 10 / 15`	Delay in minutes after boot before running (so boot isn't slowed down)
`cron.daily`	Job name; anacron stores its last-run date in `/var/spool/anacron/`
`run-parts ...`	The command: run every script in that folder

16.8 cron vs systemd Timers

	cron	systemd timer (Ch 13)
Setup	One line	Two files (.service + .timer)
Logs	Only if you redirect	Automatic, in journalctl
Missed runs	Skipped (unless anacron)	`Persistent=true` catches up
Resource limits, dependencies	No	Yes (memory limits, After=, User=)
Best for	Quick, simple jobs	Important production jobs

Teaching Note: Start with `* * * * * date >> /tmp/cron-test.log`. Wait two minutes and `cat` the file. Seeing new lines appear every minute proves cron is alive, and then the student can change the schedule and watch what happens.

Practice Lab 16

1. Write cron lines for: every 20 minutes; daily at 11:45 PM; every Monday at 8 AM; 15th of each month at noon; every 2 hours on weekends.
2. Add `* * * * * date >> /tmp/cron-test.log 2>&1`. Wait 3 minutes and check the file. Then remove that line.
3. Find the CRON lines for your test in `/var/log/syslog`.
4. Add a job that uses `date +%F` without escaping `%`. What happens? Fix it.
5. Schedule your backup script from Chapter 15 for 2:30 AM every day, logging to `/var/log/backup-www.log`.
6. Use `at` to write "hello" into `/tmp/at-test.txt` 2 minutes from now. Check `atq`, then the file.
7. Back up your crontab to a file.

Chapter 16 Summary

- Cron line: **minute hour day-of-month month day-of-week command**. Symbols: `* , - /`.
- `crontab -e` edit, `-l` list, `-r` delete all (careful!).
- System cron: `/etc/crontab`, `/etc/cron.d/` (with user field), `/etc/cron.daily/` (no dots in names).
- Cron needs full paths, `>> log 2>&1`, and escaped `\%`.
- `at` runs once; anacron catches missed jobs; timers are the modern alternative.

Quick Quiz

1. What does `0 */6 * * *` mean?
2. Write a cron line for weekdays at 7:15 AM.
3. Your script works by hand but says "command not found" in cron. Why?
4. Why won't `/etc/cron.daily/cleanup.sh` run?
5. Which command lists waiting `at` jobs?

Answers: 1) Every 6 hours, on the hour (00:00, 06:00, 12:00, 18:00) 2) `15 7 * * 1-5 command` 3) cron has a minimal PATH; use full paths 4) run-parts skips names with a dot; rename it to `cleanup` 5) `atq`

CHAPTER 17

Ajay Upadhyay · DevOps Mentor

Disk and Storage Management

Checking space, adding disks, partitions, filesystems, mounting, swap, LVM and RAID.

In this chapter you will learn:

- Checking space: `df` , `du` , `ncdu`
- Seeing disks: `lsblk` , `blkid`
- Disk names and partitions, MBR vs GPT
- Partitioning with `fdisk` and `parted`
- Filesystems (ext4, xfs) and `mkfs`
- Mounting and making it permanent with `/etc/fstab`
- Swap space
- LVM step by step, including growing a volume
- RAID basics

17.1 How Storage Fits Together

Real-Life Example: Land and Buildings

A **disk** is a plot of land. **Partitioning** divides the land into plots. **Formatting** (creating a filesystem) builds a house on a plot with rooms and a filing system. **Mounting** connects that house to the main road (the folder tree), so people can walk in. A disk without a filesystem is empty land; a filesystem that isn't mounted is a house with no road to it.



The same 5 steps every time you add a disk.

Figure 17.1 — The path from a new disk to usable storage.

17.2 Checking Space: df and du

df — free space per filesystem

```
ajay@ubuntu:~$ df -h
Filesystem                         Size  Used Avail Use% Mounted on
tmpfs                              392M  1.3M  391M   1% /run
/dev/mapper/ubuntu--vg-ubuntu--lv   24G  9.1G   14G  41% /
tmpfs                              2.0G     0  2.0G   0% /dev/shm
/dev/sda2                          2.0G  182M  1.7G  10% /boot
/dev/sdb1                           49G   38G  8.5G  82% /data
```

Output Explained: df -h	
`Filesystem`	The device. `/dev/sda2` = disk a, partition 2. `/dev/mapper/...` = an LVM volume (17.8). `tmpfs` = lives in RAM, gone on reboot.
`Size / Used / Avail`	Total, used and available space (`-h` = human units).
`Use%`	**Watch this.** Above 80% = plan action, above 90% = act now. At 100%, apps crash, logs stop and databases can corrupt.
`Mounted on`	The folder where you reach this filesystem.

```
ajay@ubuntu:~$ df -h /var          # which filesystem holds /var, and how full?
ajay@ubuntu:~$ df -hT              # -T adds the filesystem type (ext4, xfs...)
ajay@ubuntu:~$ df -i               # inodes: "No space left" while df -h shows free space?
```

> **Tip: "No space left on device" but df shows space?** Run `df -i`. Every file needs an **inode** (Chapter 4). Millions of tiny files (sessions, cache) can use up all inodes even when gigabytes are free.

du — how big is this folder?

```
ajay@ubuntu:~$ du -sh /var/log                    # -s = total only
612M    /var/log
ajay@ubuntu:~$ sudo du -h --max-depth=1 /var | sort -rh | head -5
3.0G    /var
2.1G    /var/lib
612M    /var/log
240M    /var/cache
12M     /var/backups
ajay@ubuntu:~$ sudo apt install ncdu -y && sudo ncdu /   # interactive explorer: arrow keys to drill down
```

> **On the Job: the "disk full" drill**
> 1. `df -h` → which filesystem is full? 2. `sudo du -h --max-depth=1 /var | sort -rh | head` → follow the biggest folder down. 3. Common culprits: `/var/log` (logs), `/var/lib/docker` (old images: `docker system prune`), `/var/cache/apt` (`apt clean`), journal (`journalctl --vacuum-size=500M`). 4. Check for deleted-but-open files: `sudo lsof +L1`.

17.3 Seeing Disks: lsblk and blkid

```
ajay@ubuntu:~$ lsblk
NAME                      MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                         8:0    0   30G  0 disk
├─sda1                      8:1    0    1M  0 part
├─sda2                      8:2    0    2G  0 part /boot
└─sda3                      8:3    0   28G  0 part
  └─ubuntu--vg-ubuntu--lv 252:0    0   24G  0 lvm  /
sdb                         8:16   0   50G  0 disk
└─sdb1                      8:17   0   50G  0 part /data
sdc                         8:32   0   20G  0 disk
```

Output Explained: lsblk	
`NAME`	Device name, shown as a tree: disk → partitions → LVM volumes.
`RM`	1 = removable (USB stick), 0 = fixed disk.
`SIZE`	Size of the disk or partition.
`RO`	1 = read-only.
`TYPE`	`disk`, `part` (partition), `lvm`, `rom` (CD), `loop` (snap images).
`MOUNTPOINTS`	Where it's mounted. Empty = not in use.
`sdc (no children)`	A brand-new 20 GB disk: no partitions, not mounted. Ready to set up!

Disk naming

Name	Meaning
`/dev/sda`, `sdb`, `sdc`	1st, 2nd, 3rd SATA/SCSI disk (also most cloud disks)
`/dev/sda1`, `sda2`	Partitions 1 and 2 on the first disk
`/dev/nvme0n1`, `nvme0n1p1`	NVMe SSD and its partition 1 (AWS Nitro instances use these)
`/dev/vda`, `xvda`	Virtual disks on KVM / older Xen clouds

```
ajay@ubuntu:~$ sudo blkid
/dev/sda2: UUID="7c1f0a2e-5b1d-4d3a-9b8e-2f4c6a1e9d70" TYPE="ext4" PARTUUID="..."
/dev/sdb1: UUID="e2a4b9c1-3f7d-4e8a-a6b2-91d0c5f3e8a4" TYPE="ext4" PARTUUID="..."
```

A **UUID** is a unique ID for a filesystem. Unlike `/dev/sdb1`, it never changes when disks are added or reordered, which is why `fstab` should use it.

17.4 Partitioning: MBR vs GPT, fdisk and parted

	MBR (old, "dos")	GPT (modern)
Max disk size	2 TB	Practically unlimited
Max partitions	4 primary (or extended + logical)	128
Use with	Old BIOS systems	UEFI and all new setups. **Choose GPT.**

Partitioning a new disk with fdisk (interactive)

```
ajay@ubuntu:~$ sudo fdisk /dev/sdc
Command (m for help): g              # create a new empty GPT partition table
Created a new GPT disklabel.
Command (m for help): n              # new partition
Partition number (1-128, default 1): <Enter>
First sector (2048-41943006, default 2048): <Enter>
Last sector, +/-sectors or +/-size{K,M,G,T,P}: <Enter>      # Enter = use the whole disk (or +10G)
Created a new partition 1 of type 'Linux filesystem' and of size 20 GiB.
Command (m for help): p              # print to check
Command (m for help): w              # WRITE changes and exit
The partition table has been altered.
```

Key	Action	Key	Action
`m`	Help menu	`p`	Print the partition table
`g` / `o`	New GPT / MBR table	`n`	New partition
`d`	Delete a partition	`t`	Change type (e.g. Linux LVM, swap)
`w`	**Write** and quit	`q`	**Quit without saving** (safe exit)

> **Careful! Partitioning destroys data.** Triple-check the device name with `lsblk` before running fdisk. `sudo fdisk /dev/sda` instead of `/dev/sdc` can wipe your operating system. Nothing is changed until you press `w`, so if unsure, press `q`.

The same thing with parted (one line, good for scripts)

```
ajay@ubuntu:~$ sudo parted /dev/sdc --script mklabel gpt mkpart primary ext4 0% 100%
ajay@ubuntu:~$ sudo parted /dev/sdc print
```

17.5 Filesystems and mkfs

Type	Description	Use for
ext4	Ubuntu's default. Mature, reliable, can grow and shrink.	General use, root disk
xfs	Very fast with large files; can grow but **not shrink**.	Big data, databases (RHEL default)
btrfs	Snapshots, checksums, compression built in.	Advanced setups
vfat / exfat	Readable by Windows and macOS.	USB sticks, EFI partition
swap	Not a real filesystem: extra virtual memory.	Swap partitions

```
ajay@ubuntu:~$ sudo mkfs.ext4 -L appdata /dev/sdc1      # format as ext4 with a label
Creating filesystem with 5242624 4k blocks and 1310720 inodes
Filesystem UUID: 9d3e1b72-6a4c-4f1e-8c2d-5b7a0e3f1c64
Writing superblocks and filesystem accounting information: done
ajay@ubuntu:~$ sudo mkfs.xfs /dev/sdd1                  # or xfs
```

17.6 Mounting

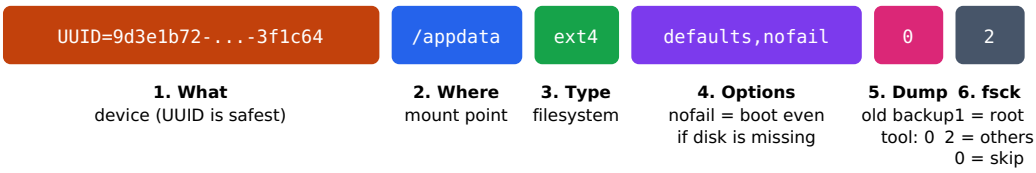
```
ajay@ubuntu:~$ sudo mkdir -p /appdata                   # 1. create an empty folder (the mount point)
ajay@ubuntu:~$ sudo mount /dev/sdc1 /appdata            # 2. connect the filesystem to it
ajay@ubuntu:~$ df -h /appdata                            # 3. check
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdc1        20G   24K   19G   1% /appdata
ajay@ubuntu:~$ findmnt /appdata                          # details: source, type, options
ajay@ubuntu:~$ sudo umount /appdata                      # disconnect (note: umount, no "n")
umount: /appdata: target is busy.
ajay@ubuntu:~$ sudo lsof +D /appdata                     # who is using it? (or: fuser -vm /appdata)
```

> **"target is busy"** usually means a process has a file open there, or **your own terminal is inside that folder**. `cd ~` and try again.

17.7 /etc/fstab — Mount Automatically at Boot

A `mount` command is forgotten after reboot. To make it permanent, add a line to `/etc/fstab` (**f**ile**s**ystem **tab**le).

```
ajay@ubuntu:~$ sudo blkid /dev/sdc1
/dev/sdc1: LABEL="appdata" UUID="9d3e1b72-6a4c-4f1e-8c2d-5b7a0e3f1c64" TYPE="ext4"
ajay@ubuntu:~$ sudo nano /etc/fstab        # add this line:
UUID=9d3e1b72-6a4c-4f1e-8c2d-5b7a0e3f1c64  /appdata  ext4  defaults,nofail  0  2
```



Other useful options: noatime (faster), ro (read-only), noexec (no programs can run from here)

Figure 17.2 — The six fields of an fstab line.

```
ajay@ubuntu:~$ sudo umount /appdata
ajay@ubuntu:~$ sudo systemctl daemon-reload
ajay@ubuntu:~$ sudo mount -a                # mount everything in fstab = TEST before rebooting
ajay@ubuntu:~$ sudo findmnt --verify       # checks fstab for mistakes
ajay@ubuntu:~$ df -h /appdata               # mounted again = fstab line works
```

> **Careful! A broken fstab can stop the server from booting.** A typo can drop you into "emergency mode", and on a cloud server you may have no console to fix it. **Always** run `sudo mount -a` after editing fstab and fix any error before you reboot. Use `nofail` for non-essential disks.

17.8 Swap Space

Swap is disk space used as overflow memory when RAM is full. It's much slower than RAM, but prevents the OOM killer (Chapter 14) from killing processes immediately.

```
ajay@ubuntu:~$ swapon --show                        # current swap
NAME      TYPE SIZE USED PRIO
/swap.img file   2G   0B   -2
# Add a 2 GB swap file (common on small cloud servers that have none):
ajay@ubuntu:~$ sudo fallocate -l 2G /swapfile
ajay@ubuntu:~$ sudo chmod 600 /swapfile             # only root may read it
ajay@ubuntu:~$ sudo mkswap /swapfile
ajay@ubuntu:~$ sudo swapon /swapfile
ajay@ubuntu:~$ echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab   # permanent
ajay@ubuntu:~$ cat /proc/sys/vm/swappiness           # 0-100: how eager Linux is to use swap
60
ajay@ubuntu:~$ sudo sysctl vm.swappiness=10          # prefer RAM (common for servers)
```

17.9 LVM — Flexible Storage

With normal partitions, resizing is painful. **LVM** (Logical Volume Manager) adds a flexible layer: you pool disks together and carve out volumes that can **grow later, even while mounted**. Ubuntu Server uses LVM by default (that's the `ubuntu--vg-ubuntu--lv` you saw in df).

> **Real-Life Example: A Water Tank System**
> **Physical Volumes (PV)** are individual water sources: a well, a tanker, a rain barrel. You pour them all into one big **Volume Group (VG)**: a shared tank. Then you fill **Logical Volumes (LV)**: bottles of whatever size you need. Need a bigger bottle? Pour more from the tank. Tank running low? Connect another water source.

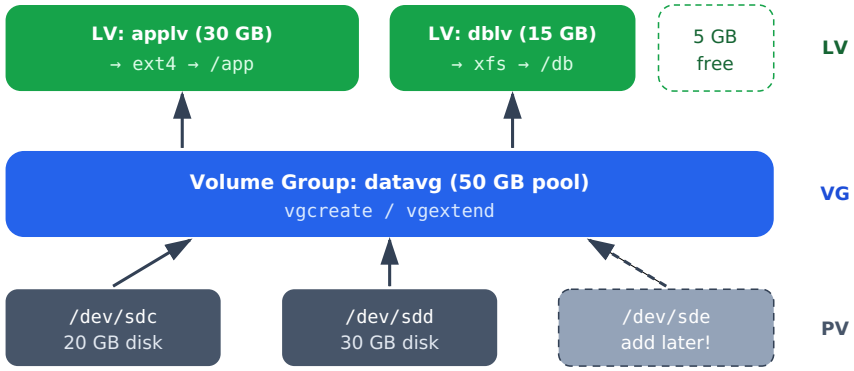


Figure 17.3 — LVM layers: disks (PV) → pool (VG) → flexible volumes (LV).

Building LVM step by step

```
# 1. Turn disks into Physical Volumes
ajay@ubuntu:~$ sudo pvcreate /dev/sdc /dev/sdd
  Physical volume "/dev/sdc" successfully created.
  Physical volume "/dev/sdd" successfully created.
# 2. Pool them into a Volume Group called datavg
ajay@ubuntu:~$ sudo vgcreate datavg /dev/sdc /dev/sdd
# 3. Create Logical Volumes from the pool
ajay@ubuntu:~$ sudo lvcreate -n applv -L 30G datavg     # -L = exact size
ajay@ubuntu:~$ sudo lvcreate -n dblv -l 75%FREE datavg  # -l = percentage of what's free
# 4. Format and mount like any partition
ajay@ubuntu:~$ sudo mkfs.ext4 /dev/datavg/applv
ajay@ubuntu:~$ sudo mkdir -p /app && sudo mount /dev/datavg/applv /app
# 5. Check every layer
ajay@ubuntu:~$ sudo pvs; sudo vgs; sudo lvs
  PV         VG     Fmt  Attr PSize   PFree
  /dev/sdc   datavg lvm2 a--  <20.00g      0
  /dev/sdd   datavg lvm2 a--  <30.00g  <5.00g
  VG     #PV #LV #SN Attr   VSize  VFree
  datavg   2   2   0 wz--n- 49.99g <5.00g
  LV    VG     Attr       LSize
  applv datavg -wi-ao---- 30.00g
  dblv  datavg -wi-a----- 15.00g
```

Output Explained: pvs / vgs / lvs	
`PSize / PFree`	Size of each disk and how much of it is still unallocated.
`#PV #LV`	The VG has 2 disks and 2 volumes.
`VSize / VFree`	Total pool size and free space left in the pool (~5 GB here).
`LSize`	Size of each logical volume.
`Attr -wi-ao----`	`w` writable, `i` inherited allocation, `a` active, `o` open (mounted). dblv has no `o` because we didn't mount it yet.

The big win: growing a volume while it's in use

```
# /app is full. Give it 4 GB more from the pool, and grow the filesystem in the same step (-r):
ajay@ubuntu:~$ sudo lvextend -r -L +4G /dev/datavg/applv
  Size of logical volume datavg/applv changed from 30.00 GiB to 34.00 GiB.
  Logical volume datavg/applv successfully resized.
resize2fs: The filesystem on /dev/mapper/datavg-applv is now 8912896 (4k) blocks long.

# Pool empty? Add a new disk to the VG, then extend:
ajay@ubuntu:~$ sudo pvcreate /dev/sde && sudo vgextend datavg /dev/sde
ajay@ubuntu:~$ sudo lvextend -r -l +100%FREE /dev/datavg/applv
```

> **On the Job: growing the root disk on a cloud VM.** You enlarged the disk in the AWS/Azure console from 30 GB to 60 GB, but `df` still shows the old size. Three layers need growing: **1)** the partition: `sudo growpart /dev/sda 3` **2)** the PV: `sudo pvresize /dev/sda3` **3)** the LV and filesystem: `sudo lvextend -r -l +100%FREE /dev/ubuntu-vg/ubuntu-lv` . No reboot needed.

> **Tip: Ubuntu's installer leaves free space.** On many fresh Ubuntu Server installs, the root LV only uses part of the disk. Check `sudo vgs` : if VFree shows space, one `lvextend -r -l +100%FREE` gives it to `/` .

17.10 RAID Basics

RAID combines several disks for **speed**, **safety** (surviving a failed disk), or both. On Linux, software RAID is managed with `mdadm` .

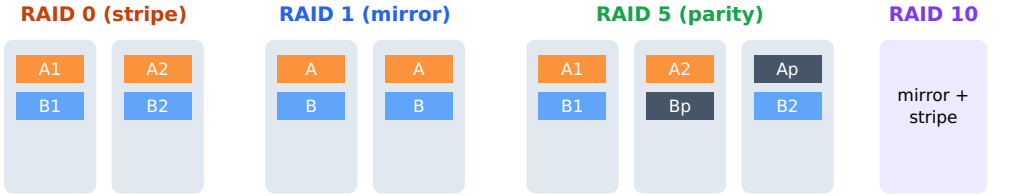


Figure 17.4 — Common RAID levels compared.

```
ajay@ubuntu:~$ sudo apt install mdadm -y
ajay@ubuntu:~$ sudo mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdf /dev/sdg   # RAID 1 mirror
ajay@ubuntu:~$ cat /proc/mdstat               # status and rebuild progress
md0 : active raid1 sdg[1] sdf[0]
      20954112 blocks super 1.2 [2/2] [UU]
ajay@ubuntu:~$ sudo mkfs.ext4 /dev/md0 && sudo mount /dev/md0 /mirror
ajay@ubuntu:~$ sudo mdadm --detail --scan | sudo tee -a /etc/mdadm/mdadm.conf
ajay@ubuntu:~$ sudo update-initramfs -u
```

Output Explained: /proc/mdstat	
`active raid1`	The array is running as a mirror.
`[2/2]`	2 disks expected, 2 working.
`[UU]`	Each U = a disk that is Up. `[U_]` means one disk has failed: replace it soon!

Careful! RAID is not a backup. RAID protects against a **disk dying**. It does **not** protect against deleting a file, ransomware, or a bad script: those mistakes are copied to every disk instantly. You still need backups (Chapter 15 + 16). In the cloud, the provider handles disk redundancy, so you'll mostly meet RAID on physical servers.

Teaching Note: Practise this chapter in a **virtual machine**. In VirtualBox, add 3–4 small virtual disks (5 GB each) to the VM. The learner can partition, format, LVM and RAID them freely, and if something breaks, just delete and re-add the disks.

Practice Lab 17

1. Run `df -hT` and `lsblk`. Which device holds `/`? Is it LVM?
2. Find the 5 biggest folders under `/usr`.
3. Add a 5 GB virtual disk to your VM. Find its name with `lsblk`.
4. Create a GPT partition with fdisk, format it ext4 with label `lab`, mount on `/lab`.
5. Add it to `/etc/fstab` using its UUID and `nofail`. Test with `umount` + `mount -a`, then reboot to confirm.
6. Create a 1 GB swap file, enable it, check with `swapon --show` and `free -h`.
7. Add two more 5 GB disks. Build a VG `labvg`, create a 4 GB LV, format, mount. Then grow it by 3 GB with `lvextend -r` and confirm with `df -h`.
8. (Bonus) Build a RAID 1 from two disks and read `/proc/mdstat`.

Chapter 17 Summary

- `df -h` space per filesystem (and `-i` for inodes); `du -sh` / `ncdu` folder sizes.
- `lsblk` shows disks; `blkid` shows UUIDs. Use GPT.
- Disk → partition (fdisk/parted) → filesystem (mkfs) → mount → fstab (UUID, nofail, test with `mount -a`).
- Swap = emergency memory: fallocate, mkswap, swapon, fstab.
- LVM: PV → VG → LV; grow live with `lvextend -r`. RAID 0/1/5/10 with mdadm — and RAID is not a backup.

Quick Quiz

1. "No space left on device" but `df -h` shows 40% used. What do you check?
2. Why use UUID instead of `/dev/sdb1` in fstab?
3. What must you always do after editing fstab, before rebooting?
4. In LVM, what do PV, VG and LV stand for?
5. What does `[U_]` in `/proc/mdstat` mean?

Answers: 1) `df -i` (inodes) 2) Device names can change; UUIDs don't 3) `sudo mount -a` to test it 4) Physical Volume, Volume Group, Logical Volume 5) One disk in the RAID has failed

LEVEL 2 FINALE

Ajay Upadhyay · DevOps Mentor

Final Project: Production-Ready App Server

Use Chapters 8–17 together, the way you would on a real job.

Scenario: You've been given a fresh Ubuntu 24.04 server with an extra 20 GB disk (`/dev/sdb`). Set it up to host a small web app. Try every step yourself first; the solution follows.

#	Requirement	Chapter
1	Update all packages and install `nginx` , `htop` , `ncdu` . Hold the nginx version.	11
2	Put the extra disk under LVM (VG `appvg`), create a 10 GB LV, format ext4, mount on `/srv/app` permanently.	17
3	Add 1 GB of swap.	17
4	Run a simple app as a systemd service under a non-root user, auto-restarting on failure.	13
5	Rotate the app's logs daily, keeping 7 days.	14
6	Back up `/srv/app` and `/etc/nginx` every night at 1:30 AM, keeping 7 days.	15, 16
7	A health-check that logs a warning if disk use is above 80%, every 10 minutes.	8, 9, 16
8	Produce a report: top 5 IPs and status-code counts from the nginx access log.	8
9	Prove everything works: service status, timers/cron, mounts, logs.	All

Solution

```
# 1. Packages
ajay@server:~$ sudo apt update && sudo apt upgrade -y
ajay@server:~$ sudo apt install -y nginx htop ncdu && sudo apt-mark hold nginx

# 2. LVM storage
ajay@server:~$ sudo pvcreate /dev/sdb && sudo vgcreate appvg /dev/sdb
ajay@server:~$ sudo lvcreate -n datalv -L 10G appvg && sudo mkfs.ext4 /dev/appvg/datalv
ajay@server:~$ sudo mkdir -p /srv/app
ajay@server:~$ echo "UUID=$(sudo blkid -s UUID -o value /dev/appvg/datalv) /srv/app ext4 defaults,nofail 0 2" | sudo tee -
a /etc/fstab
ajay@server:~$ sudo systemctl daemon-reload && sudo mount -a && df -h /srv/app

# 3. Swap
ajay@server:~$ sudo fallocate -l 1G /swapfile && sudo chmod 600 /swapfile
ajay@server:~$ sudo mkswap /swapfile && sudo swapon /swapfile
ajay@server:~$ echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab

# 4. App as a service (a tiny Python web server as the "app")
ajay@server:~$ sudo useradd --system --no-create-home --shell /usr/sbin/nologin webapp
ajay@server:~$ sudo mkdir -p /srv/app/site /var/log/webapp
ajay@server:~$ echo "<h1>Hello from Ajay's server</h1>" | sudo tee /srv/app/site/index.html
ajay@server:~$ sudo chown -R webapp:webapp /srv/app/site /var/log/webapp
ajay@server:~$ sudo tee /etc/systemd/system/webapp.service <<'EOF'
[Unit]
Description=Demo Web App
After=network-online.target
Wants=network-online.target

[Service]
User=webapp
WorkingDirectory=/srv/app/site
ExecStart=/usr/bin/python3 -m http.server 5000
StandardOutput=append:/var/log/webapp/app.log
StandardError=append:/var/log/webapp/app.log
Restart=on-failure
RestartSec=5

[Install]
```

```
WantedBy=multi-user.target
EOF
ajay@server:~$ sudo systemctl daemon-reload && sudo systemctl enable --now webapp
ajay@server:~$ curl -s localhost:5000 | head -1

# 5. Log rotation
ajay@server:~$ sudo tee /etc/logrotate.d/webapp <<'EOF'
/var/log/webapp/*.log {
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty
    copytruncate
}
EOF
ajay@server:~$ sudo logrotate -d /etc/logrotate.d/webapp

# 6. Nightly backup at 1:30 AM
ajay@server:~$ sudo tee /usr/local/bin/backup-app <<'EOF'
#!/bin/bash
DEST=/backups; DATE=$(date +%F)
mkdir -p "$DEST"
tar -czpf "$DEST/app-$DATE.tar.gz" /srv/app /etc/nginx \
  && logger -t backup "OK app-$DATE" || logger -t backup -p user.err "FAILED app-$DATE"
find "$DEST" -name "app-*.tar.gz" -mtime +7 -delete
EOF
ajay@server:~$ sudo chmod 755 /usr/local/bin/backup-app
ajay@server:~$ echo "30 1 * * * root /usr/local/bin/backup-app >> /var/log/backup-app.log 2>&1" | sudo tee
/etc/cron.d/backup-app

# 7. Disk health check every 10 minutes
ajay@server:~$ sudo tee /usr/local/bin/check-disk <<'EOF'
#!/bin/bash
df -h --output=pcent,target | tail -n +2 | while read pct mnt; do
  use=${pct%\%}
  [ "$use" -gt 80 ] && logger -t diskcheck -p user.warning "$mnt is at $pct"
done
exit 0
EOF
ajay@server:~$ sudo chmod 755 /usr/local/bin/check-disk
ajay@server:~$ echo "*/10 * * * * root /usr/local/bin/check-disk" | sudo tee /etc/cron.d/check-disk

# 8. Access-log report
ajay@server:~$ echo "Top IPs:"; awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head -5
ajay@server:~$ echo "Status codes:"; awk '{print $9}' /var/log/nginx/access.log | sort | uniq -c | sort -rn

# 9. Verify
ajay@server:~$ systemctl status webapp nginx --no-pager | grep -E "service|Active"
ajay@server:~$ lsblk; swapon --show; apt-mark showhold; ls /etc/cron.d/
ajay@server:~$ sudo /usr/local/bin/backup-app && ls -lh /backups && journalctl -t backup -n 2
```

New things in this project: `StandardOutput=append:` tells systemd to write the app's output to a file. `copytruncate` lets logrotate rotate a file the app keeps open. `while read ... done` is a loop, and `${pct%\%}` removes the % sign — both are taught properly in Bash Scripting (Level 3).

Level 2 Cheat Sheet

Print this page and keep it next to your keyboard. — *Ajay Upadhyay*

Text Processing (Ch 8)

`grep -inrvc "x" f`	Search lines
`grep -E "a|b"`	OR pattern
`cut -d, -f1,3`	Pick columns
`sort -nrk2 -t,`	Sort numeric, reverse
`sort | uniq -c | sort -rn`	Count & rank
`awk '{print $1}'`	Print column 1
`awk -F: '$3>=1000'`	Filter rows
`sed -i.bak 's/a/b/g'`	Replace in file
`tr 'a-z' 'A-Z'`	Change characters
`| tee -a f`	Show + save
`| xargs cmd`	Lines to arguments

Redirection (Ch 9)

`>` / `>>`	Overwrite / append
`2> err`	Errors to file
`> f 2>&1`	Both to file
`2>/dev/null`	Hide errors
`a && b || c`	On success / on fail
`echo $?`	Last exit code

Finding (Ch 10)

`find / -name "x" 2>/dev/null`	By name
`find . -size +100M`	By size
`find . -mtime +7 -delete`	Old files
`find . -exec cmd {} \;`	Act on results
`locate x` / `which x`	Fast / commands

Packages (Ch 11)

`apt update && apt upgrade`	Patch
`apt install / remove / purge`	Manage
`apt search / show / policy`	Info
`dpkg -l / -L / -S`	Installed / files / owner
`apt-mark hold pkg`	Freeze version

Processes (Ch 12)

`ps aux --sort=-%mem`	Memory hogs
`top` (P, M, 1, k, q)	Live view
`kill PID` / `kill -9`	Stop / force
`pgrep -a` / `pkill -f`	By name
`& Ctrl+Z bg fg jobs`	Job control
`nohup cmd &`	Survive logout
`nice -n 19` / `renice`	Priority

systemd (Ch 13)

`systemctl status x`	Check service
`start stop restart reload`	Control now
`enable --now / disable`	At boot
`systemctl --failed`	Crashed units
`daemon-reload`	After editing units
`systemctl edit x`	Safe override
`systemctl list-timers`	Scheduled timers
`systemd-analyze blame`	Slow boot

Logs (Ch 14)

`journalctl -u x -f`	Follow a service
`journalctl -p err -b`	Errors this boot
`journalctl -b -1`	Previous boot
`--since "1 hour ago"`	Time filter
`/var/log/syslog, auth.log`	Key files
`logrotate -d file`	Test rotation
`dmesg -T`	Kernel / OOM
`free -h`	Memory (available!)

Archives (Ch 15)

`tar -czf a.tar.gz dir`	Create
`tar -tzf a.tar.gz`	List
`tar -xzf a.tar.gz -C dir`	Extract
`gzip -k / gunzip / zgrep`	Compress
`zip -r / unzip -d`	Windows-friendly

Scheduling (Ch 16)

`m h dom mon dow cmd`	Cron format
`crontab -e / -l`	Edit / list
`*/15 * * * *`	Every 15 min
`>> log 2>&1` , `\%`	Cron musts
`at now + 1 hour` / `atq`	One-time

Storage (Ch 17)

`df -hT` / `df -i`	Space / inodes
`du -sh * | sort -rh`	Folder sizes
`lsblk` / `blkid`	Disks / UUIDs
`fdisk` , `mkfs.ext4` , `mount`	Add a disk
`mount -a`	Test fstab!
`pvcreate vgcreate lvcreate`	Build LVM
`lvextend -r -L +5G`	Grow live
`cat /proc/mdstat`	RAID health

You finished Level 2!

You can now search and analyse any log, manage software, investigate slow servers, keep services alive, read logs like a pro, automate backups and jobs, and manage disks with LVM. That's real day-to-day DevOps administration. **Next up, Level 3:** networking, SSH in depth, firewalls and security hardening, Bash scripting, web servers and SSL, and performance troubleshooting.

Questions or feedback? Reach me at **ajayupadhyaydevops@gmail.com** — Ajay Upadhyay, DevOps Mentor.

LINUX LEARNING SERIES · LEVEL 3

Linux
Advanced Administration

by **Ajay Upadhyay**

DevOps Mentor · ajayupadhyaydevops@gmail.com

The advanced guide to Ubuntu Linux.
Networking, SSH, firewalls and security, Bash scripting, shell environment, web servers and SSL, and performance troubleshooting — the skills that make you the person people call when production breaks.

```
ajay@prod:~$ sudo ss -tlnp | grep :443
LISTEN 0 511 0.0.0.0:443 0.0.0.0:* users:(("nginx",pid=812,fd=8))
ajay@prod:~$ sudo ufw status | head -1
Status: active
ajay@prod:~$ ./healthcheck.sh && echo "All green"
All green
ajay@prod:~$ _
```

Chapters 18–24 | Networking · SSH · Firewall & Security · Bash Scripting · Environment · Web Servers & SSL · Troubleshooting
Based on Ubuntu 24.04 LTS · Continues from Level 2 (Chapters 8–17)

Before You Start

Hi, I'm **Ajay Upadhyay**, a DevOps mentor. In Levels 1 and 2 you learned to use Linux and run it day to day. Level 3 is about running Linux **in production**: servers that talk to each other over a network, that strangers on the internet try to break into, that need automation instead of hand-typed commands, and that sometimes slow down at 2 AM for no obvious reason.

The book uses the same boxes as Level 2: **Output Explained**, **On the Job**, **Real-Life Example**, **Tip**, **Careful!** and **Teaching Note**.

What You Need for the Labs

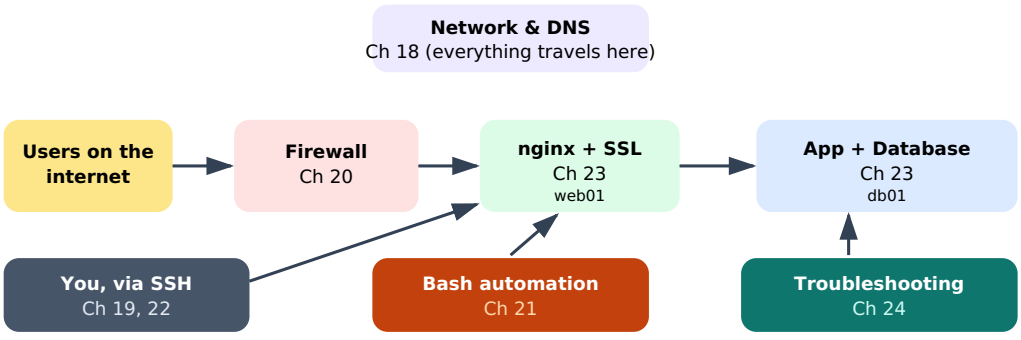
Setup	Why
Two Ubuntu VMs (or cloud instances)	Networking and SSH only make sense between two machines. Call them `web01` and `db01`.
Both on the same network	In VirtualBox use a "NAT Network" or "Bridged Adapter" so they can reach each other.
Console access	When practising firewalls and SSH hardening you *will* lock yourself out at least once. Keep the VirtualBox window or cloud "serial console" available.

> **Careful!** Practise firewall and SSH changes on lab machines first, never directly on a production server. Several labs in this level can cut off your own connection on purpose so you learn how to recover.

Contents

Ch 18	**Networking Fundamentals** — IP, subnets, CIDR, ports, DNS, ip, ss, ping, traceroute, dig, curl, Netplan
Ch 19	**SSH and Remote Access** — keys, ssh config, hardening, scp, rsync, tunnels, jump hosts
Ch 20	**Firewall and Security** — UFW, iptables, nftables, fail2ban, AppArmor, auto-updates, hardening checklist
Ch 21	**Bash Shell Scripting** — variables, arguments, if, loops, functions, arrays, exit codes, safe scripts, real automation
Ch 22	**Environment and Shell Customization** — environment variables, PATH, .bashrc vs .profile, aliases
Ch 23	**Web Servers and Common Services** — nginx, Apache, reverse proxy, load balancing, Let's Encrypt SSL, MySQL/PostgreSQL
Ch 24	**Performance Monitoring and Troubleshooting** — vmstat, iostat, sar, lsof, strace, tcpdump, and a step-by-step method
End	**Final Project and Cheat Sheet**

How Level 3 Fits Together



A typical production setup. Every chapter in Level 3 covers one piece of it.

CHAPTER 18

Ajay Upadhyay · DevOps Mentor

Networking Fundamentals

How computers find and talk to each other, and the commands to see and fix it.

In this chapter you will learn:

- IP addresses, private vs public, and what a gateway is
- Subnets and CIDR (/24, /16) explained with simple maths
- Ports, TCP vs UDP, and the common port numbers
- How DNS turns names into IP addresses
- `ip` , `ss` , `ping` , `traceroute` , `dig` , `nslookup` , `curl` , `wget`
- Configuring the network with Netplan
- `/etc/hosts` , `/etc/resolv.conf` and systemd-resolved
- A step-by-step network troubleshooting method

18.1 IP Addresses

Every device on a network needs an address so data can reach it, like every house needs a postal address. That is the **IP address**. The common version, **IPv4**, is four numbers from 0 to 255 separated by dots: `192.168.1.10` .

> **Real-Life Example: Addresses in a City**
> An **IP address** is a house address. A **subnet** is a street or colony: houses on the same street can visit each other directly. The **gateway** (router) is the colony's main gate: to reach any address *outside* your colony, you must go through the gate. **DNS** is the phone directory that tells you "Sharma ji lives at house 42".

Private vs public addresses

Range	Type	Where you see it
`10.0.0.0 – 10.255.255.255`	Private	Cloud VPCs (AWS often uses 10.x or 172.31.x)
`172.16.0.0 – 172.31.255.255`	Private	Docker networks, AWS default VPC
`192.168.0.0 – 192.168.255.255`	Private	Home and office Wi-Fi
`127.0.0.1` (localhost)	Loopback	"This same machine". Never leaves the computer.
Everything else	Public	Reachable on the internet, e.g. `142.250.183.14`

Private addresses can be reused in millions of networks; they're not reachable from the internet directly. To go out, the router translates them into its one public address. This is called **NAT** (Network Address Translation).

18.2 Subnets and CIDR

An IP address has two parts: the **network part** (which street) and the **host part** (which house). **CIDR notation** like `/24` tells you how many bits belong to the network part. An IPv4 address has 32 bits in total.

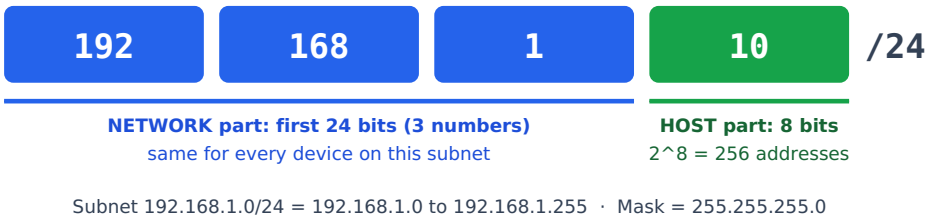


Figure 18.1 — CIDR /24: the first three numbers are the network, the last is the host.

The simple maths

Number of addresses = **2 to the power of (32 – prefix)**. Two addresses are reserved (the first = network address, the last = broadcast), so usable hosts = total − 2. (Cloud providers often reserve a few more.)

CIDR	Subnet mask	Addresses	Usable hosts	Typical use
`/32`	255.255.255.255	1	1	A single IP (firewall rules)
`/28`	255.255.255.240	16	14	Tiny subnet
`/24`	255.255.255.0	256	254	Typical office or cloud subnet
`/20`	255.255.240.0	4,096	4,094	AWS default subnet
`/16`	255.255.0.0	65,536	65,534	A whole cloud VPC
`/0`	0.0.0.0	All	—	`0.0.0.0/0` = "anywhere on the internet"

> **Tip: the quick rule.** Every step **down** by 1 in the prefix **doubles** the size. /24 = 256, /23 = 512, /22 = 1,024. And every 8 steps is a whole number in the address: /8, /16, /24.

> **On the Job:** You'll write CIDR all the time: in cloud security groups (`allow 22 from 203.0.113.5/32`), Kubernetes pod networks (`10.244.0.0/16`), Docker (`172.17.0.0/16`) and firewalls (`ufw allow from 10.0.1.0/24`).

18.3 Ports, TCP and UDP

An IP address gets data to the right **machine**. A **port** gets it to the right **program** on that machine. Ports are numbers from 0 to 65535.

> **Real-Life Example: An Office Building**
> The IP address is the building's address. The port is the **room number**: room 22 is security (SSH), room 80 is the web reception (HTTP), room 443 is the secure web reception (HTTPS), room 5432 is the database office.

Port	Service	Port	Service
22	SSH	3306	MySQL / MariaDB
53	DNS	5432	PostgreSQL
80	HTTP	6379	Redis
443	HTTPS	8080 / 3000 / 5000	App servers (common choices)
25 / 587	Email (SMTP)	6443	Kubernetes API
123	NTP (time)	9090 / 9100	Prometheus / node_exporter

Ports below 1024 are "privileged": only root can open them (that's why nginx starts as root and then drops to `www-data`).

	TCP	UDP
Connection	Handshake first, then data	Just sends
Reliable?	Yes: lost data is re-sent, order is kept	No: lost is lost
Speed	Slower	Faster
Like	A phone call (you confirm the other side is there)	Shouting across a room
Used by	SSH, HTTP/S, databases	DNS lookups, video calls, NTP

18.4 DNS — Names to Numbers

Humans remember `google.com` ; computers need `142.250.183.14` . **DNS** (Domain Name System) is the internet's phone directory that converts one to the other.

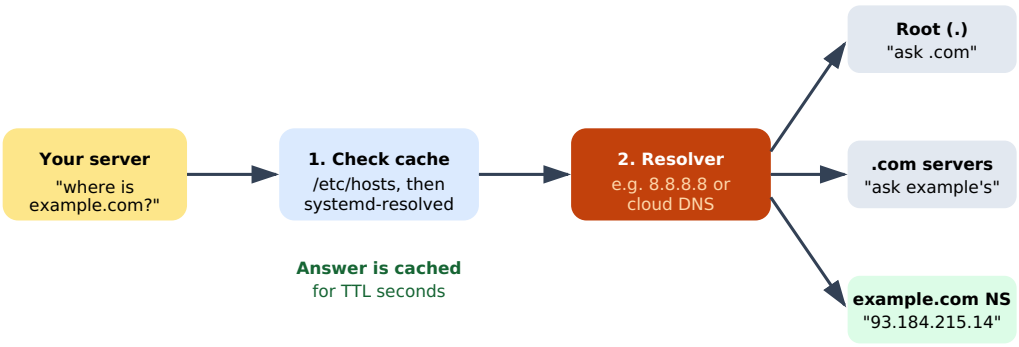


Figure 18.2 — How a DNS lookup travels. The resolver does the walking; your server just asks once.

DNS record types

Type	Meaning	Example
A	Name → IPv4 address	`app.example.com → 203.0.113.10`
AAAA	Name → IPv6 address	`→ 2001:db8::10`
CNAME	Name → another name (an alias)	`www → app.example.com`
MX	Mail server for the domain	`→ mail.example.com`
TXT	Free text (verification, SPF)	`"v=spf1 include:_spf.google.com ~all"`
NS	Which servers are in charge of this domain	`→ ns1.cloudflare.com`

18.5 Seeing Your Network: ip

`ip` is the modern tool (the older `ifconfig` and `route` are deprecated, though you'll still see them in old guides).

```
ajay@web01:~$ ip a                      # short for: ip address show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
    inet 127.0.0.1/8 scope host lo
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
    link/ether 08:00:27:3a:9c:51 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.10/24 metric 100 brd 192.168.1.255 scope global dynamic enp0s3
       valid_lft 85912sec preferred_lft 85912sec
    inet6 fe80::a00:27ff:fe3a:9c51/64 scope link
```

Output Explained: ip a	
`lo`	The loopback interface (127.0.0.1). Always there; used to talk to yourself.
`enp0s3`	The real network card. Names vary: `eth0`, `ens5` (AWS), `enp0s3` (VirtualBox).
`UP,LOWER_UP`	UP = turned on; LOWER_UP = cable/link connected. `state DOWN` = no connection.
`mtu 1500`	Largest packet size in bytes. Normal Ethernet is 1500.
`link/ether 08:00:...`	The MAC address: the network card's hardware ID.
`inet 192.168.1.10/24`	**The IPv4 address and subnet.** The line you'll look at most.
`dynamic`	Address came from DHCP (automatic). Static addresses don't say "dynamic".
`inet6 fe80::...`	IPv6 link-local address (only works on the local network).

```
ajay@web01:~$ ip -br a                  # brief: one line per interface
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s3           UP             192.168.1.10/24 fe80::a00:27ff:fe3a:9c51/64
ajay@web01:~$ ip route                  # routing table: where does traffic go?
default via 192.168.1.1 dev enp0s3 proto dhcp src 192.168.1.10 metric 100
192.168.1.0/24 dev enp0s3 proto kernel scope link src 192.168.1.10 metric 100
ajay@web01:~$ ip route get 8.8.8.8         # which route will THIS address use?
ajay@web01:~$ ip neigh                    # neighbours (IP to MAC) on the local network
ajay@web01:~$ ip link set enp0s3 down    # turn an interface off (careful over SSH!)
```

Output Explained: ip route	
`default via 192.168.1.1`	The **default gateway**. Anything not on a known network is sent here. No default route = no internet.
`192.168.1.0/24 dev enp0s3`	Addresses in my own subnet are reached directly through enp0s3, no gateway needed.

18.6 ss — What's Listening and Who's Connected

`ss` (socket statistics) replaces the older `netstat`. The most useful combination is `-tulpn`.

```
ajay@web01:~$ sudo ss -tulpn
Netid State  Recv-Q Send-Q  Local Address:Port  Peer Address:Port Process
udp   UNCONN 0      0       127.0.0.53%lo:53         0.0.0.0:*     users:(("systemd-resolve",pid=520,fd=13))
tcp   LISTEN 0      4096          0.0.0.0:22         0.0.0.0:*     users:(("sshd",pid=901,fd=3))
tcp   LISTEN 0      511           0.0.0.0:80         0.0.0.0:*     users:(("nginx",pid=812,fd=6))
tcp   LISTEN 0      151         127.0.0.1:3306       0.0.0.0:*     users:(("mysqld",pid=1544,fd=21))
```

Output Explained: ss -tulpn	
`-t -u`	Show TCP and UDP sockets
`-l`	Only **listening** sockets (servers waiting for connections)
`-p`	Show the **process** (needs sudo)
`-n`	Show numbers (port 22) instead of names (ssh) — faster and clearer
`0.0.0.0:80`	nginx listens on port 80 on **all** network interfaces: reachable from outside.
`127.0.0.1:3306`	MySQL listens **only on localhost**: other machines cannot reach it. Very important for security (and a common "why can't my app server connect to the DB?" cause).
`127.0.0.53%lo:53`	systemd-resolved's local DNS cache.

```
ajay@web01:~$ ss -tn                          # current TCP connections
ajay@web01:~$ sudo ss -tlnp | grep :8080       # is anything on port 8080?
ajay@web01:~$ ss -s                          # summary counts
ajay@web01:~$ sudo lsof -i :80                # another way: who uses port 80?
```

> **"Address already in use"** when starting an app means another program already holds that port. Find it with `sudo ss -tlnp | grep :PORT`, then stop that program or change your app's port.

18.7 Testing Connectivity

ping — can I reach it?

```
ajay@web01:~$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=21.4 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=20.9 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=22.1 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=117 time=21.0 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 20.9/21.3/22.1/0.47 ms
```

Output Explained: ping	
`-c 4`	Send 4 packets, then stop (without it, ping runs until Ctrl+C).
`icmp_seq=1`	Packet number. Gaps mean lost packets.
`ttl=117`	"Time to live": how many more routers the packet could have passed. Not something to worry about day to day.
`time=21.4 ms`	Round-trip time. Same datacenter: under 1 ms. Same country: 10–50 ms. Other side of the world: 150–300 ms.
`0% packet loss`	Anything above 0% on a wired network is a sign of trouble.

Careful! "ping fails" does **not** always mean the server is down. Many cloud servers and firewalls block ping (ICMP) on purpose. Test the real service port instead: `nc -zv host 443` or `curl`.

traceroute / tracepath — where does it break?

```
ajay@web01:~$ sudo apt install traceroute -y
ajay@web01:~$ traceroute -n google.com
 1  192.168.1.1    1.102 ms  0.981 ms  0.954 ms
 2  10.20.0.1      5.420 ms  5.388 ms  5.301 ms
 3  * * *
 4  142.250.160.92 20.118 ms 20.004 ms 19.887 ms
```

Each line is one router ("hop") on the way. `* * *` means that router didn't answer (often normal). If the trace stops for good at hop 2, the problem is between hop 2 and hop 3 — usually your network provider. `mtr google.com` combines ping and traceroute in a live view.

nc — is this port open?

```
ajay@web01:~$ nc -zv db01 5432                # -z just check, -v tell me
Connection to db01 (192.168.1.20) 5432 port [tcp/postgresql] succeeded!
ajay@web01:~$ nc -zv db01 3306
nc: connect to db01 (192.168.1.20) port 3306 (tcp) failed: Connection refused
ajay@web01:~$ timeout 3 nc -zv 10.0.5.9 22
(no answer after 3 seconds)
```

What the errors mean (very useful!)	
`succeeded`	Network and service both fine.
`Connection refused`	The machine is reachable but **nothing is listening** on that port (service down, or listening on 127.0.0.1 only).
`Timeout (no answer)`	Packets are being **dropped**: a firewall/security group is blocking, or the host is down or unreachable.
`No route to host`	Routing problem, or a firewall actively rejecting.

18.8 DNS Tools: dig and nslookup

```
ajay@web01:~$ dig example.com
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 41022
;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            3600    IN      A       93.184.215.14

;; Query time: 24 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
```

Output Explained: dig	
`status: NOERROR`	The lookup worked. `NXDOMAIN` = the name doesn't exist. `SERVFAIL` = the DNS server had a problem.
`QUESTION`	What we asked: the A record of example.com.
`ANSWER: 3600 IN A 93.184...`	The answer. **3600** is the **TTL**: cache this for 3600 seconds (1 hour).
`SERVER: 127.0.0.53`	Which DNS server answered (here, the local systemd-resolved cache).

```
ajay@web01:~$ dig +short example.com             # just the answer
93.184.215.14
ajay@web01:~$ dig +short MX gmail.com            # mail servers
ajay@web01:~$ dig @8.8.8.8 app.example.com        # ask Google's DNS directly (bypass local cache)
ajay@web01:~$ dig +trace example.com              # show the whole root → .com → domain walk
ajay@web01:~$ dig -x 8.8.8.8 +short               # reverse: IP → name
dns.google.
ajay@web01:~$ nslookup example.com                 # older, simpler tool
```

> **On the Job: "I changed the DNS record but users still see the old server."** That's the TTL. Old answers stay cached until the TTL expires. Compare `dig @8.8.8.8 name` with `dig @1.1.1.1 name` and your local answer. Pro tip: lower the TTL to 300 a day *before* a planned migration.

18.9 Talking HTTP: curl and wget

```
ajay@web01:~$ curl http://localhost                        # print the page
ajay@web01:~$ curl -I https://example.com                  # -I headers only
HTTP/2 200
content-type: text/html; charset=UTF-8
cache-control: max-age=3600
server: ECAcc (nyd/D184)
ajay@web01:~$ curl -s -o /dev/null -w "%{http_code} %{time_total}s\n" https://example.com
200 0.312114s
ajay@web01:~$ curl -L http://example.com                   # -L follow redirects
ajay@web01:~$ curl -v https://example.com 2>&1 | less      # -v show EVERYTHING (TLS, headers)
ajay@web01:~$ curl -X POST -H "Content-Type: application/json" \
     -d '{"name":"ajay"}' http://localhost:5000/api/users   # call an API
ajay@web01:~$ curl -H "Host: app.example.com" http://192.168.1.10   # test a site before DNS points to it
ajay@web01:~$ wget https://example.com/file.tar.gz         # download a file
ajay@web01:~$ curl -fsSLO https://example.com/file.tar.gz     # same with curl
```

Status	Meaning
2xx	Success (200 OK, 201 Created)
3xx	Redirect (301 moved permanently, 302 temporary)
4xx	Client problem: 400 bad request, 401 not logged in, 403 forbidden, 404 not found
5xx	Server problem: 500 app crashed, 502 bad gateway (backend down), 503 unavailable, 504 backend too slow

18.10 Configuring the Network with Netplan

Ubuntu stores network settings in YAML files under `/etc/netplan/`. Netplan reads them and configures the real backend (systemd-

networkd on servers, NetworkManager on desktops).

```
ajay@web01:~$ ls /etc/netplan/
50-cloud-init.yaml
ajay@web01:~$ sudo nano /etc/netplan/01-static.yaml
```

```
# Static IP example
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      dhcp4: false
      addresses:
        - 192.168.1.10/24
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses: [1.1.1.1, 8.8.8.8]
        search: [lab.local]
```

Netplan File Explained	
`enp0s3`	The interface name from `ip a`.
`dhcp4: false`	Don't ask for an automatic address. (Use `true` for DHCP.)
`addresses`	Our fixed IP and subnet.
`routes: to default via`	The default gateway.
`nameservers`	DNS servers to use, and the search domain (so `db01` means `db01.lab.local`).

```
ajay@web01:~$ sudo chmod 600 /etc/netplan/01-static.yaml   # netplan warns if others can read it
ajay@web01:~$ sudo netplan generate                        # check the YAML for mistakes
ajay@web01:~$ sudo netplan try                             # apply, auto-ROLLBACK in 120s unless you press Enter
ajay@web01:~$ sudo netplan apply                           # apply for real
```

Careful! YAML is strict about spaces. Use spaces, never tabs, and keep the indentation exact. Over SSH, always use `netplan try`: if the new settings cut your connection, they roll back automatically after 2 minutes.

Cloud servers: On AWS/Azure/GCP, the file `50-cloud-init.yaml` is generated by cloud-init and the IP comes from the cloud's DHCP. Change IPs in the cloud console, not in Netplan.

18.11 /etc/hosts, /etc/resolv.conf and systemd-resolved

`/etc/hosts` is a tiny local phone book that is checked **before** DNS. Great for labs and testing.

```
ajay@web01:~$ cat /etc/hosts
127.0.0.1       localhost
127.0.1.1       web01
192.168.1.20    db01 db01.lab.local
ajay@web01:~$ getent hosts db01           # resolve the way programs do (hosts file + DNS)
192.168.1.20    db01 db01.lab.local
```

On Ubuntu, `/etc/resolv.conf` points to a local cache at `127.0.0.53` run by **systemd-resolved**. Don't edit resolv.conf directly (it gets overwritten); set DNS servers in Netplan instead.

```
ajay@web01:~$ cat /etc/resolv.conf | grep -v "^#"
nameserver 127.0.0.53
options edns0 trust-ad
search lab.local
ajay@web01:~$ resolvectl status            # the REAL upstream DNS servers per interface
ajay@web01:~$ resolvectl query example.com
ajay@web01:~$ sudo resolvectl flush-caches   # clear the local DNS cache
ajay@web01:~$ hostnamectl set-hostname web01   # change the machine's name
```

18.12 Troubleshooting Method: Bottom to Top

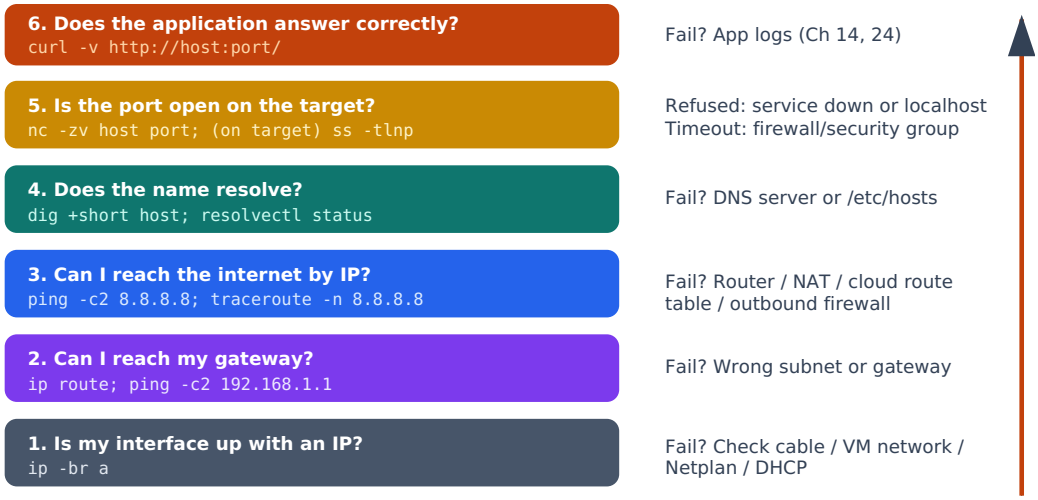


Figure 18.3 — Debug networks from the bottom up. The first step that fails is where your problem is.

Teaching Note: Give the learner a "broken" VM: remove the gateway, or point DNS to a wrong IP, or make a service listen on 127.0.0.1 only. Let them walk Figure 18.3 from step 1 to find the fault. This builds the habit of checking layers in order instead of guessing.

Practice Lab 18

1. On both VMs, find the IP address, subnet and default gateway. Write them down.
2. How many usable hosts are in `10.0.4.0/22`? (Hint: 2^{10}.)
3. Add `db01` to `/etc/hosts` on web01. Ping it by name.
4. Install nginx on db01. From web01, test port 80 with `nc -zv` and `curl -I`. Then stop nginx and test again. What error do you get now?
5. List all listening ports on both VMs with `ss -tulpn`. Which services listen only on localhost?
6. Use `dig` to find the A, MX and NS records of a domain you know. What are the TTLs?
7. Get the HTTP status code and total time of `https://example.com` with one curl command.
8. Give db01 a static IP with Netplan using `netplan try`.

Chapter 18 Summary

- IP = address, port = program, gateway = way out, DNS = names to IPs. Private ranges: 10.x, 172.16–31.x, 192.168.x.
- CIDR /N: addresses = $2^{(32-N)}$. /24 = 256, /16 = 65,536, /32 = one IP, 0.0.0.0/0 = anywhere.
- `ip -br a`, `ip route`, `ss -tulpn`, `ping`, `traceroute`, `nc -zv`, `dig`, `curl -I/-v`.
- "Refused" = nothing listening; "timeout" = firewall or host down.
- Configure with Netplan (`netplan try`); DNS via systemd-resolved (`resolvectl`).

Quick Quiz

1. How many addresses are in a /26?
2. MySQL shows `127.0.0.1:3306` in ss. Can web01 connect to it from another machine?
3. `nc` says "Connection refused". Is it more likely a firewall or a stopped service?
4. What does the 3600 mean in a dig answer?
5. Which HTTP status means "the reverse proxy couldn't reach the backend"?

Answers: 1) 64 2) No, it only listens on localhost 3) Stopped service (or bound to localhost); firewalls usually cause timeouts 4) TTL: cache for 3600 seconds 5) 502 Bad Gateway

CHAPTER 19

Ajay Upadhyay · DevOps Mentor

SSH and Remote Access

Logging in to servers securely, copying files, and reaching machines hidden behind others.

In this chapter you will learn:

- How SSH works and why keys beat passwords
- Creating and installing keys: `ssh-keygen`, `ssh-copy-id`
- The `~/.ssh/config` file that saves you typing
- `ssh-agent` and agent forwarding
- Hardening `sshd_config` safely
- Copying files: `scp`, `sftp`, `rsync`
- Port forwarding (tunnels) and jump hosts
- Fixing common SSH errors

19.1 How SSH Works

SSH (Secure Shell) gives you an encrypted terminal on a remote machine. Everything you type, including passwords, travels scrambled so nobody in between can read it. The server runs `sshd` (the daemon, port 22); you run the `ssh` client.

```
ajay@laptop:~$ ssh ajay@192.168.1.10             # user@host
The authenticity of host '192.168.1.10 (192.168.1.10)' can't be established.
ED25519 key fingerprint is SHA256:Xq3v...p9Q.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.10' (ED25519) to the list of known hosts.
ajay@192.168.1.10's password:
ajay@web01:~$                                    # you're now on the server
ajay@laptop:~$ ssh -p 2222 ajay@web01             # non-standard port
ajay@laptop:~$ ssh ajay@web01 "uptime; df -h /"    # run a command and come back
```

Output Explained: the first-connection question	
`authenticity can't be established`	Your computer has never seen this server before.
`key fingerprint`	The server's unique ID. In a careful setup you'd compare it with what the admin gave you.
`Permanently added ... known hosts`	Saved in `~/.ssh/known_hosts`. Next time, SSH checks the server is still the same machine.

19.2 Keys Instead of Passwords

With key authentication you have a **pair** of keys: a **private key** that never leaves your laptop, and a **public key** you put on every server you want to log in to.

> **Real-Life Example: A Padlock and Its Key**
> The **public key** is a padlock. You can hand out copies to any number of servers: "lock your door with this". The **private key** is the only key that opens those padlocks, and it stays in your pocket. Someone who steals a padlock can't open anything with it. Someone who steals your key can open everything — so protect it with a passphrase.



Figure 19.1 — The private key never travels. Only a signed proof does.

Step 1: Create a key pair (on your laptop)

```
ajay@laptop:~$ ssh-keygen -t ed25519 -C "ajay@laptop"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/ajay/.ssh/id_ed25519): <Enter>
Enter passphrase (empty for no passphrase): ********
Your identification has been saved in /home/ajay/.ssh/id_ed25519
Your public key has been saved in /home/ajay/.ssh/id_ed25519.pub
ajay@laptop:~$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIKx9...q7 ajay@laptop
```

> **Which key type?** Use `ed25519` : small, fast, very secure. Use `-t rsa -b 4096` only for very old systems that don't support ed25519. The `-C` comment just helps you recognise the key later.

Step 2: Install the public key on the server

```
ajay@laptop:~$ ssh-copy-id ajay@192.168.1.10
Number of key(s) added: 1
Now try logging into the machine, with: "ssh 'ajay@192.168.1.10'"
ajay@laptop:~$ ssh ajay@192.168.1.10          # no password asked now (only the key passphrase)
```

Without `ssh-copy-id` (e.g. on Windows), do it by hand on the server:

```
ajay@web01:~$ mkdir -p ~/.ssh && chmod 700 ~/.ssh
ajay@web01:~$ echo "ssh-ed25519 AAAAC3Nz...q7 ajay@laptop" >> ~/.ssh/authorized_keys
ajay@web01:~$ chmod 600 ~/.ssh/authorized_keys
```

> **Careful! Permissions matter.** SSH silently **ignores** keys if `~/.ssh` isn't 700 or `authorized_keys` /the private key isn't 600, or if your home folder is writable by others. This is the #1 reason "my key doesn't work". (Remember Chapter 7!)

ssh-agent: type the passphrase once

```
ajay@laptop:~$ eval "$(ssh-agent -s)"          # start the agent (desktops usually have one already)
ajay@laptop:~$ ssh-add ~/.ssh/id_ed25519       # unlock the key once
ajay@laptop:~$ ssh-add -l                      # list loaded keys
```

19.3 ~/.ssh/config — Stop Typing Long Commands

```
ajay@laptop:~$ nano ~/.ssh/config
Host web01
    HostName 192.168.1.10
    User ajay
    IdentityFile ~/.ssh/id_ed25519

Host db01
    HostName 192.168.1.20
    User ajay
    Port 2222

Host prod-*
    User ubuntu
    IdentityFile ~/.ssh/aws-prod.pem

Host *
    ServerAliveInterval 60
    ServerAliveCountMax 3
ajay@laptop:~$ chmod 600 ~/.ssh/config
ajay@laptop:~$ ssh web01                        # instead of: ssh -i ~/.ssh/id_ed25519 ajay@192.168.1.10
```

ssh config Explained	
`Host web01`	A short nickname you choose.
`HostName`	The real IP or DNS name.
`User / Port / IdentityFile`	Which username, port and key to use.
`Host prod-*`	Wildcard: applies to any nickname starting with prod-.
`Host *`	Applies to every connection. ServerAlive sends a "still here?" every 60 s so idle sessions don't freeze or drop.

Bonus: `scp`, `rsync`, `git` and Ansible all use these nicknames too.

19.4 Hardening the SSH Server

Internet bots try to log in to every public SSH server all day long (you saw them in auth.log in Chapter 14). These settings shut them out.

```
ajay@web01:~$ sudo nano /etc/ssh/sshd_config.d/99-hardening.conf
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
AllowUsers ajay deploy
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no
ClientAliveInterval 300
ClientAliveCountMax 2
```

Hardening Options Explained	
`PermitRootLogin no`	Nobody can log in directly as root. Use your user + sudo.
`PasswordAuthentication no`	**Keys only.** Password-guessing bots become useless.
`KbdInteractiveAuthentication no`	Closes another password-style login path.
`AllowUsers`	Only these users may log in over SSH at all.
`MaxAuthTries 3`	Disconnect after 3 failed attempts.
`LoginGraceTime 30`	30 seconds to finish logging in.
`ClientAlive...`	Drop dead sessions after ~10 minutes of no response.

Apply it the **safe** way:

```
ajay@web01:~$ sudo sshd -t                    # 1. test the config: no output = OK
ajay@web01:~$ sudo systemctl reload ssh       # 2. reload (your current session stays open)
# 3. KEEP this terminal open. In a NEW terminal, test that you can still log in:
ajay@laptop:~$ ssh web01
ajay@laptop:~$ ssh -o PubkeyAuthentication=no web01   # should now be REFUSED (passwords off)
ajay@192.168.1.10: Permission denied (publickey).
```

Careful! The golden rule of SSH changes. Never close your working session until you have confirmed, in a **second** session, that you can still log in. If you disable passwords before your key works, you are locked out and need the console to fix it.

Note for Ubuntu 24.04: SSH may be started by `ssh.socket` (socket activation). In that case the `Port` setting in sshd_config is ignored; to change the port, run `sudo systemctl edit ssh.socket` or disable the socket and enable `ssh.service`. Changing the port reduces log noise, but keys-only login is what actually protects you.

19.5 Copying Files: scp, sftp, rsync

scp — simple copy

```
ajay@laptop:~$ scp report.pdf web01:/tmp/              # laptop → server
ajay@laptop:~$ scp web01:/var/log/nginx/error.log .     # server → laptop (. = here)
ajay@laptop:~$ scp -r ./site web01:/var/www/           # -r = folders
ajay@laptop:~$ scp -P 2222 file db01:~                 # capital -P for port in scp!
```

sftp — interactive file browser

```
ajay@laptop:~$ sftp web01
sftp> ls            # remote files
sftp> lls           # local files
sftp> get app.log   # download
sftp> put conf.yml  # upload
sftp> bye
```

rsync — the professional's choice

`rsync` copies only what **changed**, can resume, keeps permissions, and can mirror folders exactly. It's the standard tool for deployments and backups.

```
ajay@laptop:~$ rsync -avz ./site/ web01:/var/www/site/
sending incremental file list
index.html
css/style.css

sent 2,341 bytes  received 64 bytes  4,810.00 bytes/sec
total size is 18,920  speedup is 7.87
ajay@laptop:~$ rsync -avz ./site/ web01:/var/www/site/     # run again: only changes are sent
sending incremental file list

sent 312 bytes  received 12 bytes  648.00 bytes/sec
```

Option	Meaning
`-a`	Archive: recursive + keep permissions, times, owners, links
`-v` / `-z` / `-h`	Verbose / compress during transfer / human sizes
`--progress` or `-P`	Show progress; `-P` also resumes partial files
`--delete`	Delete files at the destination that no longer exist at the source (exact mirror)
`--exclude=".git"`	Skip files or folders
`-n` / `--dry-run`	Show what WOULD happen, change nothing
`-e "ssh -p 2222"`	Custom SSH options

Careful! The trailing slash. `rsync -a site/ dest/` copies the **contents** of site into dest. `rsync -a site dest/` creates **dest/site/**. And always run `--delete` with `-n` first: a wrong path can delete everything at the destination.

19.6 SSH Tunnels (Port Forwarding)

SSH can carry other traffic through its encrypted connection. This lets you reach services that aren't exposed to the internet.

Local forwarding (-L): bring a remote port to your laptop

PostgreSQL on db01 listens on localhost only (safe!). You want to open it with a GUI tool on your laptop:

```
ajay@laptop:~$ ssh -L 5433:localhost:5432 db01 -N
# Now connect your laptop's DB tool to localhost:5433 — it arrives at db01's port 5432.
# -N = don't open a shell, just keep the tunnel. Ctrl+C closes it.
```



Port 5432 is never exposed to the internet, yet you can use it.

Figure 19.2 — Local port forwarding: `ssh -L 5433:localhost:5432 db01`.

Remote forwarding (-R): share your local port with the server

```
ajay@laptop:~$ ssh -R 9000:localhost:3000 web01 -N
# On web01, localhost:9000 now reaches the app running on your laptop's port 3000.
```

Dynamic forwarding (-D): a SOCKS proxy

```
ajay@laptop:~$ ssh -D 1080 web01 -N
# Set your browser's SOCKS proxy to localhost:1080 — you browse as if you were on web01's network.
```

19.7 Jump Hosts (Bastion)

In production, private servers (databases, app servers) usually have **no public IP**. You reach them through one hardened, public "bastion" server.

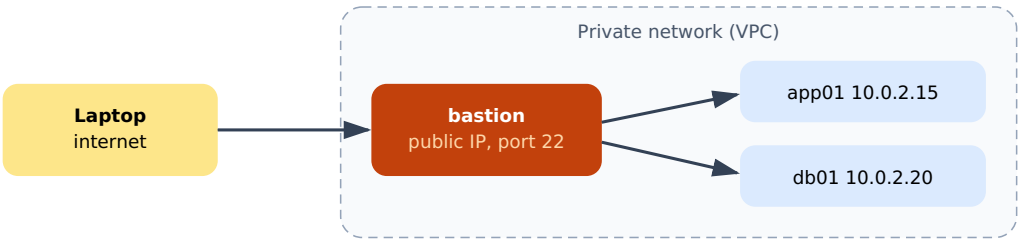


Figure 19.3 — Reaching private servers through a bastion (jump host).

```
ajay@laptop:~$ ssh -J ajay@bastion.example.com ajay@10.0.2.20    # -J = ProxyJump, one command
```

Or permanently in `~/.ssh/config`:

```
Host bastion
    HostName bastion.example.com
    User ajay

Host app01 db01
    User ajay
    ProxyJump bastion

Host app01
    HostName 10.0.2.15
Host db01
    HostName 10.0.2.20
ajay@laptop:~$ ssh db01                          # automatically hops through the bastion
ajay@laptop:~$ rsync -avz backup/ db01:/srv/     # works for rsync/scp too
```

Careful with agent forwarding (`ssh -A`). It lets the remote server use your loaded keys. Anyone with root on that server could use them while you're connected. Prefer `ProxyJump` , which never exposes your agent to the bastion.

19.8 Fixing Common SSH Errors

Error	Usual cause and fix
`Permission denied (publickey)`	Key not on the server, wrong user, wrong key, or bad permissions on `~/.ssh` . Run `ssh -v` to see which keys were tried. Check the server's `/var/log/auth.log` .
`Connection refused`	sshd not running or on another port: `systemctl status ssh` , `ss -tlnp | grep ssh` .
`Connection timed out`	Firewall/security group blocks port 22, or wrong IP.
`REMOTE HOST IDENTIFICATION HAS CHANGED!`	The server's key changed (rebuilt server, or a possible attack). If you're sure it's a rebuild: `ssh-keygen -R hostname` .
`UNPROTECTED PRIVATE KEY FILE!`	`chmod 600 ~/.ssh/id_ed25519` (or your .pem file).
Session freezes when idle	Add `ServerAliveInterval 60` to `~/.ssh/config` .

```
ajay@laptop:~$ ssh -v web01 2>&1 | grep -E "Offering|Authentications|denied"
debug1: Authentications that can continue: publickey
debug1: Offering public key: /home/ajay/.ssh/id_ed25519 ED25519 SHA256:Xq3v...
debug1: Authentications that can continue: publickey
ajay@192.168.1.10: Permission denied (publickey).
```

Reading this: the laptop offered the right key, but the server rejected it → check the server side: is that public key really in `~ajay/.ssh/authorized_keys` , with 700/600 permissions?

Teaching Note: The best SSH lesson is a controlled lockout. Have the learner disable password login, then deliberately break `authorized_keys` permissions (chmod 777), try to log in, fail, and recover through the VM console while reading `auth.log` . They'll never forget the second-session rule.

Practice Lab 19

1. Create an ed25519 key with a passphrase. Copy it to web01 and db01 with `ssh-copy-id` .
2. Write a `~/.ssh/config` so `ssh web01` and `ssh db01` work with no other options.
3. On web01, apply the hardening file from 19.4. Test in a second terminal that keys work and passwords are refused.
4. Use rsync to copy a folder to web01. Change one file and rsync again. How many bytes were sent the second time?
5. Try `rsync --delete -n` . What would it delete?
6. Install PostgreSQL on db01 (listening on localhost only). From your laptop, open a tunnel with `-L` and connect with `psql -h localhost -p 5433` .
7. Pretend web01 is a bastion: set up `ProxyJump` so `ssh db01` goes through web01.
8. Break key permissions on purpose, read the error with `ssh -v` and in `/var/log/auth.log` , then fix it.

Chapter 19 Summary

- Use keys: `ssh-keygen -t ed25519`, `ssh-copy-id`. Private key 600, `~/.ssh` 700.
- `~/.ssh/config` gives nicknames; `ssh-agent` remembers the passphrase.
- Harden: no root login, no passwords, AllowUsers. Test with `sshd -t` and a **second session**.
- `scp` simple, `rsync -avz` smart (watch trailing slashes, dry-run `--delete`).
- Tunnels: `-L` local, `-R` remote, `-D` SOCKS. Bastions: `-J` / `ProxyJump`.

Quick Quiz

1. Which key file do you put on the server: `id_ed25519` or `id_ed25519.pub`?
2. Why keep your current SSH session open while changing sshd settings?
3. What's the difference between `rsync -a src/ dst/` and `rsync -a src dst/`?
4. The DB listens on db01's localhost:3306. Write a tunnel so your laptop's port 3307 reaches it.
5. What does "REMOTE HOST IDENTIFICATION HAS CHANGED" mean?

Answers: 1) The `.pub` file 2) So you can fix things if the new settings lock you out 3) The first copies the contents; the second creates dst/src 4) `ssh -L 3307:localhost:3306 db01 -N` 5) The server's host key changed: a rebuild, or a possible man-in-the-middle attack.

CHAPTER 20

Ajay Upadhyay · DevOps Mentor

Firewall and Security

Closing every door you don't need, watching the ones you do, and keeping the server patched.

In this chapter you will learn:

- How a firewall decides what to allow
- UFW, Ubuntu's easy firewall, in depth
- What iptables and nftables are underneath
- Blocking attackers automatically with fail2ban
- AppArmor, Ubuntu's application sandbox
- Automatic security updates
- A complete server hardening checklist

20.1 How a Firewall Works

A **firewall** checks every network packet against a list of **rules**, top to bottom. The first rule that matches decides: **allow** it, **deny** it (drop silently) or **reject** it (refuse with an answer). If no rule matches, the **default policy** applies.

> **Real-Life Example: The Security Guard**
> A firewall is the guard at a society gate holding a list. "Residents: allow. Delivery boys: allow only to the lobby. Everyone else: not allowed." The guard reads the list from the top and acts on the first line that fits. The best guards use **"deny by default"**: if you're not on the list, you don't get in.

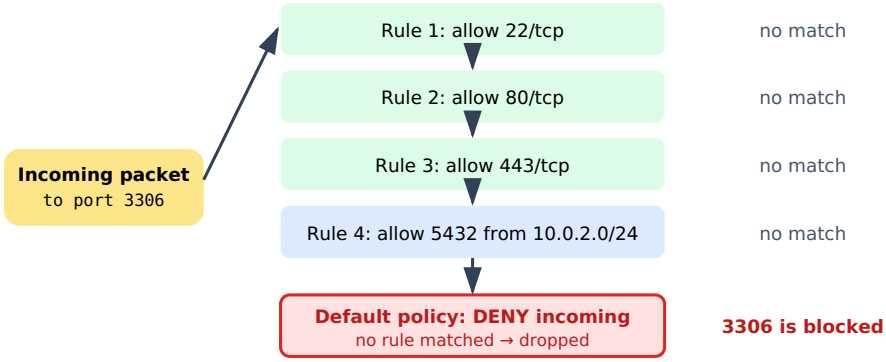


Figure 20.1 — Rules are checked in order; the default policy catches everything else.

The layers of firewalls

Layer	What it is
netfilter	The firewall built into the Linux **kernel**. It does the actual work.
nftables / iptables	Low-level tools that write rules into netfilter. nftables is the modern one; Ubuntu's `iptables` command is now a wrapper that writes nftables rules.
UFW	"Uncomplicated Firewall": an easy front-end on Ubuntu that writes the low-level rules for you.
Cloud security groups	AWS/Azure/GCP firewalls **outside** your VM. Traffic must pass both these and UFW.

20.2 UFW — The Uncomplicated Firewall

Turning it on safely

```
ajay@web01:~$ sudo ufw status
Status: inactive
ajay@web01:~$ sudo ufw default deny incoming      # block everything coming in...
ajay@web01:~$ sudo ufw default allow outgoing     # ...but let the server reach out (updates, DNS)
ajay@web01:~$ sudo ufw allow OpenSSH              # !!! allow SSH BEFORE enabling !!!
ajay@web01:~$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
```

Careful! The most famous lockout. If you `ufw enable` on a remote server without allowing SSH first, you cut your own connection instantly. Allow 22 first. For extra safety, use the `at` trick from Chapter 16: `echo "ufw disable" | sudo at now + 5 minutes`, then cancel it with `atrm` once you're sure.

Adding rules

```
ajay@web01:~$ sudo ufw allow 80/tcp                              # a port
ajay@web01:~$ sudo ufw allow 443/tcp
ajay@web01:~$ sudo ufw allow "Nginx Full"                        # an app profile (80 + 443)
ajay@web01:~$ sudo ufw allow 6000:6010/tcp                       # a port range
ajay@web01:~$ sudo ufw allow from 203.0.113.5                    # everything from one IP
ajay@web01:~$ sudo ufw allow from 10.0.2.0/24 to any port 5432 proto tcp   # DB only from the app subnet
ajay@web01:~$ sudo ufw deny from 45.83.12.9                      # block an attacker
ajay@web01:~$ sudo ufw limit OpenSSH                            # rate-limit: block IPs with 6+ tries in 30s
ajay@web01:~$ sudo ufw app list                                  # available app profiles
```

Viewing and removing rules

```
ajay@web01:~$ sudo ufw status numbered
Status: active

     To                         Action      From
     --                         ------      ----
[ 1] OpenSSH                    LIMIT IN    Anywhere
[ 2] Nginx Full                 ALLOW IN    Anywhere
[ 3] 5432/tcp                   ALLOW IN    10.0.2.0/24
[ 4] Anywhere                   DENY IN     45.83.12.9
[ 5] OpenSSH (v6)               LIMIT IN    Anywhere (v6)
[ 6] Nginx Full (v6)            ALLOW IN    Anywhere (v6)
ajay@web01:~$ sudo ufw delete 4                     # delete by number
ajay@web01:~$ sudo ufw delete allow 80/tcp          # or by the rule itself
ajay@web01:~$ sudo ufw insert 1 deny from 1.2.3.4   # put a rule at the TOP
ajay@web01:~$ sudo ufw status verbose               # also shows default policies and logging
ajay@web01:~$ sudo ufw logging on                   # blocked packets logged to /var/log/ufw.log
ajay@web01:~$ sudo ufw reset                         # remove ALL rules and disable
```

Output Explained: ufw status numbered	
`[ 1]`	Rule number. Order matters: rules are checked from 1 down.
`To`	Port or app profile on this server.
`Action`	ALLOW IN / DENY IN / LIMIT IN (allow but rate-limit) / REJECT.
`From`	Where the traffic comes from. "Anywhere" = 0.0.0.0/0.
`(v6)`	The same rule for IPv6. UFW creates them automatically.

Rule order trick: Because the first match wins, a `deny from 1.2.3.4` added *after* `allow 80` won't stop that IP from reaching port 80. Use `ufw insert 1 deny from ...` to put blocks at the top.

Careful! Docker and UFW. Docker writes its own rules that run **before** UFW's. A container published with `-p 8080:80` is reachable from the internet **even if UFW denies 8080**. Publish internal containers on localhost only (`-p 127.0.0.1:8080:80`) and rely on the cloud security group as an outer wall.

20.3 Under the Hood: iptables and nftables

You'll meet raw iptables rules in old scripts, Docker, and Kubernetes, so you need to read them even if you write UFW.

```
ajay@web01:~$ sudo iptables -L INPUT -n -v --line-numbers | head -6
Chain INPUT (policy DROP 1203 packets, 61248 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1     9810  821K ufw-before-logging-input  all  --  *  *  0.0.0.0/0  0.0.0.0/0
2     9810  821K ufw-before-input          all  --  *  *  0.0.0.0/0  0.0.0.0/0
```

Output Explained: iptables -L	
`Chain INPUT`	Rules for traffic coming **in** to this server. (OUTPUT = going out; FORWARD = passing through, used by routers and Docker.)
`policy DROP`	The default: drop what no rule allowed. The counters show 1203 packets were dropped this way.
`target`	What to do: ACCEPT, DROP, REJECT, or jump to another chain (here, UFW's own chains).
`pkts / bytes`	How many packets matched this rule. Great for checking if a rule is actually used.

Writing rules directly (for understanding; on Ubuntu, prefer UFW):

```
ajay@web01:~$ sudo iptables -A INPUT -p tcp --dport 8080 -s 10.0.2.0/24 -j ACCEPT   # -A append
ajay@web01:~$ sudo iptables -I INPUT 1 -s 45.83.12.9 -j DROP                       # -I insert at top
ajay@web01:~$ sudo iptables -D INPUT -s 45.83.12.9 -j DROP                         # -D delete
ajay@web01:~$ sudo nft list ruleset | less                                          # the real nftables rules
```

iptables flag	Meaning
`-A / -I / -D`	Append / insert / delete a rule
`-p tcp`	Protocol
`--dport 8080`	Destination port
`-s 10.0.2.0/24`	Source address
`-j ACCEPT`	"Jump to" the action

> **Tip:** Raw iptables rules disappear on reboot unless saved (`sudo apt install iptables-persistent` , then `sudo netfilter-persistent save`). UFW saves its rules automatically — another reason to use it.

20.4 fail2ban — Ban Attackers Automatically

fail2ban watches log files for repeated failures (like wrong SSH passwords) and adds a temporary firewall rule to **ban** that IP.

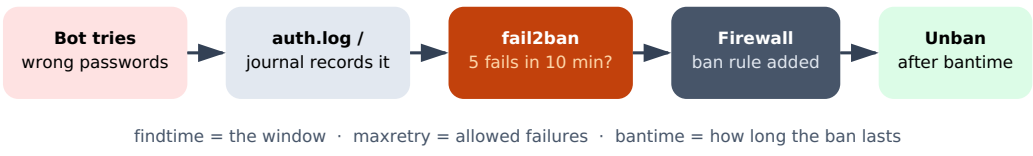


Figure 20.2 — How fail2ban works.

```
ajay@web01:~$ sudo apt install fail2ban -y
ajay@web01:~$ sudo nano /etc/fail2ban/jail.local      # your settings (never edit jail.conf)
[DEFAULT]
bantime  = 1h
findtime = 10m
maxretry = 5
ignoreip = 127.0.0.1/8 203.0.113.5
backend  = systemd

[sshd]
enabled = true

[nginx-http-auth]
enabled = true
ajay@web01:~$ sudo systemctl enable --now fail2ban
ajay@web01:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|  |- Currently failed: 2
|  |- Total failed:     148
|  `- Journal matches:  _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 3
   |- Total banned:     27
   `- Banned IP list:   45.83.12.9 103.21.44.7 91.240.118.5
ajay@web01:~$ sudo fail2ban-client set sshd unbanip 103.21.44.7    # unban (e.g. a colleague)
```

Always add your own IP to `ignoreip`. Otherwise a few typos and you ban yourself from your own server.

20.5 AppArmor — Sandboxing Programs

AppArmor is a Mandatory Access Control system built into Ubuntu. Each protected program gets a **profile** listing the exact files and abilities it may use. Even if an attacker takes over that program, they're stuck inside the profile's limits.

Real-Life Example: Normal permissions (Chapter 7) are like giving someone a key to certain rooms. AppArmor is like also telling the security camera "this cleaner may only ever enter rooms 2 and 5". Even with a stolen master key, the alarm goes off if they try room 9.

```
ajay@web01:~$ sudo aa-status
apparmor module is loaded.
112 profiles are loaded.
48 profiles are in enforce mode.
   /usr/bin/man
   /usr/sbin/mysqld
   ...
3 profiles are in complain mode.
4 processes are in enforce mode.
ajay@web01:~$ sudo apt install apparmor-utils -y
ajay@web01:~$ sudo aa-complain /usr/sbin/mysqld     # log violations but don't block (for debugging)
ajay@web01:~$ sudo aa-enforce  /usr/sbin/mysqld     # block violations again
ajay@web01:~$ sudo journalctl -k | grep -i apparmor | grep DENIED | tail -3
```

Mode	Behaviour
enforce	Violations are blocked and logged.
complain	Violations are only logged. Use when testing a profile.

On the Job: You moved MySQL's data folder to `/data/mysql` and now it won't start, even though permissions are perfect. Check `journalctl -k | grep DENIED`: AppArmor's mysqld profile only allows `/var/lib/mysql`. Fix it by adding the new path to `/etc/apparmor.d/local/usr.sbin.mysqld` and reloading with `sudo apparmor_parser -r /etc/apparmor.d/usr.sbin.mysqld`. (Disabling AppArmor is the wrong fix.)

20.6 Automatic Security Updates

Most real-world break-ins use known bugs that already had a fix. `unattended-upgrades` installs security patches automatically every day.

```
ajay@web01:~$ sudo apt install unattended-upgrades -y
ajay@web01:~$ sudo dpkg-reconfigure -plow unattended-upgrades     # answer "Yes"
ajay@web01:~$ cat /etc/apt/apt.conf.d/20auto-upgrades
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
ajay@web01:~$ sudo unattended-upgrade --dry-run --debug | tail -5   # test it
ajay@web01:~$ cat /var/log/unattended-upgrades/unattended-upgrades.log | tail
```

Fine-tune in `/etc/apt/apt.conf.d/50unattended-upgrades`, for example:

```
Unattended-Upgrade::Automatic-Reboot "true";
Unattended-Upgrade::Automatic-Reboot-Time "03:30";
Unattended-Upgrade::Remove-Unused-Dependencies "true";
Unattended-Upgrade::Package-Blacklist { "docker-ce"; "postgresql-16"; };
```

> **Tip:** By default only the **security** pocket is upgraded automatically, which is safe for most servers. Automatic reboots are a decision for your team: they apply kernel fixes, but restart the server without warning.

20.7 The Server Hardening Checklist

Use this every time you build a new server. It pulls together Levels 1–3.

#	Action	How
1	Patch everything	`apt update && apt upgrade` ; enable unattended-upgrades
2	Personal admin user with sudo; no shared accounts	`adduser` , `usermod -aG sudo` (Ch 6)
3	SSH: keys only, no root login, AllowUsers	Ch 19.4
4	Firewall: deny incoming, allow only needed ports	UFW (20.2) + cloud security group
5	Services listen only where needed	`ss -tulpn` ; databases on 127.0.0.1 or private IP
6	Remove what you don't use	`apt purge` ; `systemctl disable --now`
7	Brute-force protection	fail2ban (20.4)
8	AppArmor enforcing	`aa-status`
9	Correct permissions on secrets	600 for keys, `.env` files; no 777 (Ch 7)
10	Time synced (logs must line up)	`timedatectl` → "System clock synchronized: yes"
11	Logs kept and rotated, ideally shipped off the server	Ch 14
12	Backups that are tested by restoring	Ch 15, 16
13	Audit regularly	`sudo apt install lynis && sudo lynis audit system`

```
ajay@web01:~$ sudo lynis audit system | tail -12
  Lynis security scan details:

  Hardening index : 67 [#############       ]
  Tests performed : 262
  Plugins enabled : 0
  ...
  Suggestions (38):
  - Consider hardening SSH configuration [SSH-7408]
```

Lynis gives a **hardening index** and a list of suggestions with IDs you can look up. Work through them and run it again to watch the score rise.

Teaching Note: Run `lynis` on a fresh VM, note the score, then apply the checklist and run it again. Watching the number go from ~55 to ~75 turns an abstract list into a visible result.

Practice Lab 20

1. On web01, set up UFW: deny incoming, allow outgoing, limit OpenSSH, allow Nginx Full. Use the `at` safety net while enabling it.
2. From db01, check port 80 is open and port 3306 is closed (`nc -zv`). Is the closed port "refused" or "timeout"? Why?
3. Allow port 5432 on db01 only from web01's IP. Test from web01 and from your laptop.
4. Block your laptop's IP with `ufw insert 1 deny from ...` (keep a console open!), test, then delete the rule.
5. Install fail2ban with a jail.local. From another machine, fail SSH login 5 times. Check `fail2ban-client status sshd` , then unban.
6. Run `aa-status` . Which profiles are in enforce mode?
7. Enable unattended-upgrades and run a dry run.
8. Run `lynis audit system` . Note the hardening index, fix three suggestions, and run it again.

Chapter 20 Summary

- Firewalls check rules top-down; use default **deny incoming**.
- UFW: `allow OpenSSH` **before** `enable` ; `status numbered` , `insert` , `delete` , `limit` .
- iptables/nftables are the engine underneath; learn to read `iptables -L -n -v` .
- fail2ban bans repeat offenders; AppArmor sandboxes programs; unattended-upgrades patches daily.
- Use the hardening checklist and `lynis` on every new server.

Quick Quiz

1. What must you do before `ufw enable` on a remote server?
2. Write a UFW rule allowing PostgreSQL only from 10.0.2.0/24.
3. A container published with `-p 8080:80` is reachable even though UFW blocks 8080. Why?
4. What do `maxretry` , `findtime` and `bantime` mean?
5. MySQL fails to read a new data folder despite correct permissions. What should you check?

Answers: 1) `sudo ufw allow OpenSSH` 2) `sudo ufw allow from 10.0.2.0/24 to any port 5432 proto tcp` 3) Docker's rules run before UFW's 4) Allowed failures, the time window, and the ban length 5) AppArmor denials: `journalctl -k | grep DENIED`

CHAPTER 21

Ajay Upadhyay · DevOps Mentor

Bash Shell Scripting

Turning commands you type every day into reliable, reusable automation.

In this chapter you will learn:

- Writing and running your first script (shebang, chmod)
- Variables, quoting, command substitution and arithmetic
- Reading input and script arguments (`$1` , `$@` , `$#`)
- Conditions: `if` , `[ ]` , `[[ ]]` , `case`
- Loops: `for` , `while` , reading a file line by line
- Functions and arrays
- Exit codes, `set -euo pipefail` , `trap` and debugging
- Four real DevOps scripts, explained line by line

21.1 Your First Script

A **script** is just a text file containing commands, run from top to bottom. Anything you can type in the terminal can go in a script.

```
ajay@web01:~$ nano hello.sh
#!/bin/bash
# My first script
echo "Hello from $(hostname)"
echo "You are logged in as $USER"
date
ajay@web01:~$ chmod +x hello.sh         # make it executable (Chapter 7)
ajay@web01:~$ ./hello.sh               # run it (./ = "from this folder")
Hello from web01
You are logged in as ajay
Tue Oct 13 12:40:02 IST 2026
ajay@web01:~$ bash hello.sh            # also works, even without +x
```

Script Explained	
`#!/bin/bash`	The **shebang**. Must be the very first line. Tells Linux which program runs this file.
`# My first script`	A comment: ignored by Bash, but very useful for humans.
`$(hostname)`	**Command substitution**: runs `hostname` and puts its output here.
`$USER`	An environment variable that already exists (Chapter 22).

Why `./` ? For safety, Linux doesn't run programs from the current folder unless you say so. Put scripts you use often into `/usr/local/bin` (for everyone) or `~/bin` (just you) and you can run them by name from anywhere.

21.2 Variables

```
#!/bin/bash
name="Ajay"                     # NO spaces around =
count=5
today=$(date +%F)               # store a command's output
backup_dir="/backups/$today"

echo "Hello, $name"
echo "Backing up to ${backup_dir}/www"   # {} makes the edge of the name clear
echo "Count is $count"
```

Careful! No spaces around =. `name = "Ajay"` fails with "name: command not found", because Bash thinks `name` is a command. Write `name="Ajay"`.

Quoting: the rule that prevents most bugs

Quotes	Behaviour	Example result
`"double"`	Variables and `$( )` **are** expanded; spaces are kept	`"Hi $name"` → Hi Ajay
`'single'`	Everything is literal, nothing expanded	`'Hi $name'` → Hi $name
no quotes	Expanded, then split on spaces and wildcards expanded	Dangerous with file names containing spaces

```
ajay@web01:~$ file="My Report.txt"
ajay@web01:~$ rm $file          # WRONG: tries to delete "My" and "Report.txt"
ajay@web01:~$ rm "$file"        # RIGHT: deletes "My Report.txt"
```

Golden rule: Always write `"$variable"` with double quotes, unless you have a specific reason not to.

Useful variable tricks

```
ajay@web01:~$ f="backup-2026-10-13.tar.gz"
ajay@web01:~$ echo "${f%.tar.gz}"       # remove from the END
backup-2026-10-13
ajay@web01:~$ echo "${f#backup-}"       # remove from the START
2026-10-13.tar.gz
ajay@web01:~$ echo "${#f}"               # length
24
ajay@web01:~$ echo "${f/2026/2027}"      # replace
backup-2027-10-13.tar.gz
ajay@web01:~$ echo "${ENV:-staging}"     # use "staging" if ENV is empty or unset
staging
```

Arithmetic

```
ajay@web01:~$ echo $(( 7 * 6 ))  $(( 17 / 5 ))  $(( 17 % 5 ))
42 3 2                              # Bash only does whole numbers: 17/5 = 3, remainder 2
ajay@web01:~$ count=1; ((count++)); echo $count
2
ajay@web01:~$ echo "scale=2; 17/5" | bc    # decimals need bc or awk
3.40
```

21.3 Input and Arguments

```
ajay@web01:~$ cat greet.sh
#!/bin/bash
echo "Script name: $0"
echo "First arg: $1"
echo "All args: $@"
echo "Number of args: $#"
ajay@web01:~$ ./greet.sh web01 db01
Script name: ./greet.sh
First arg: web01
All args: web01 db01
Number of args: 2
```

Variable	Meaning
`$0`	The script's own name
`$1 $2 ... $9`, `${10}`	The 1st, 2nd... argument
`$@`	All arguments (use as `"$@"` to keep each one intact)
`$#`	How many arguments
`$?`	Exit code of the last command
`$$`	This script's PID (handy for unique temp files)

```
#!/bin/bash
read -p "Which environment? (dev/prod): " env       # -p shows a prompt
read -s -p "DB password: " pass; echo                # -s hides what you type
echo "Deploying to $env"
```

> **Tip:** For automation, prefer **arguments** over `read`. A script that asks questions can't run from cron or a CI pipeline.

21.4 Conditions: if

```
#!/bin/bash
usage=$(df / --output=pcent | tail -1 | tr -dc '0-9')

if [ "$usage" -ge 90 ]; then
  echo "CRITICAL: / is ${usage}% full"
elif [ "$usage" -ge 80 ]; then
  echo "WARNING: / is ${usage}% full"
else
  echo "OK: / is ${usage}% full"
fi
```

```
OK: / is 31% full
```

Script Explained	
`df / --output=pcent`	Print only the "Use%" column for /.
`tail -1 | tr -dc '0-9'`	Take the last line and delete every character that isn't a digit: " 31%" → "31".
`[ "$usage" -ge 90 ]`	A **test**: is usage greater than or equal to 90? Spaces inside the brackets are required.
`then / elif / else / fi`	The branches. `fi` ("if" backwards) closes the block.

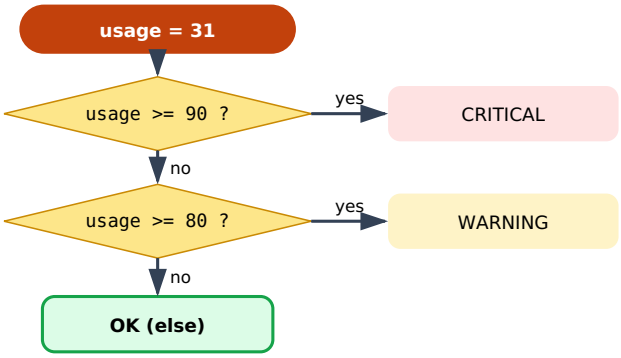


Figure 21.1 — The if / elif / else flow. The first true test wins.

Test operators

Numbers	
`-eq`	equal
`-ne`	not equal
`-gt` / `-ge`	greater / or equal
`-lt` / `-le`	less / or equal

Text (strings)	
`"$a" = "$b"`	equal
`"$a" != "$b"`	not equal
`-z "$a"`	is empty
`-n "$a"`	is not empty

Files	
`-f path`	is a regular file
`-d path`	is a directory
`-e path`	exists (anything)
`-r / -w / -x`	readable / writable / executable
`-s path`	exists and is not empty

Combining	
`[ a ] && [ b ]`	AND
`[ a ] || [ b ]`	OR
`! [ a ]`	NOT

```
#!/bin/bash
if [ $# -ne 1 ]; then
  echo "Usage: $0 <config-file>" >&2      # error messages go to stderr
  exit 1
fi
if [ ! -f "$1" ]; then
  echo "Error: $1 not found" >&2
  exit 2
fi
if systemctl is-active --quiet nginx; then  # any command can be the test!
  echo "nginx is running"
fi
```

> **`[ ]` vs `[[ ]]`:** `[[ ]]` is Bash's improved test. It's safer with empty variables and supports patterns and regex: `[[ $file == *.log ]]`, `[[ $ip =~ ^10\. ]]`. Use `[[ ]]` in Bash scripts; use `[ ]` only if the script must run in plain `sh`.

case — many choices, cleanly

```
#!/bin/bash
case "$1" in
  start)   systemctl start myapp ;;
  stop)    systemctl stop myapp ;;
  restart) systemctl restart myapp ;;
  status)  systemctl status myapp --no-pager ;;
  *)       echo "Usage: $0 {start|stop|restart|status}"; exit 1 ;;
esac
```

21.5 Loops

for — repeat for each item

```
ajay@web01:~$ for s in web01 db01 cache01; do echo "Checking $s"; done
Checking web01
Checking db01
Checking cache01
ajay@web01:~$ for i in {1..3}; do echo "Attempt $i"; done
Attempt 1
Attempt 2
Attempt 3
ajay@web01:~$ for f in /var/log/*.log; do echo "$f: $(wc -l < "$f") lines"; done
ajay@web01:~$ for host in $(cat servers.txt); do ssh "$host" uptime; done
```

while — repeat while a condition is true

```
#!/bin/bash
# Wait until the app answers, up to 30 tries
tries=0
until curl -fs http://localhost:5000/health >/dev/null; do
  ((tries++))
  if [ "$tries" -ge 30 ]; then
    echo "App did not start" >&2; exit 1
  fi
  sleep 2
done
echo "App is healthy after $tries retries"
```

`until` is the opposite of `while`: it loops **until** the command succeeds. This "wait for healthy" pattern is used in almost every deployment script.

Reading a file line by line

```
ajay@web01:~$ cat users.csv
ravi,developers
priya,testers
#!/bin/bash
while IFS=, read -r user group; do
  echo "Creating $user in $group"
  sudo useradd -m -G "$group" "$user"
done < users.csv
```

Loop Explained	
`IFS=,`	"Internal Field Separator": split each line on commas.
`read -r user group`	Put field 1 in `user`, field 2 in `group`. `-r` keeps backslashes as-is.
`done < users.csv`	Feed the file into the loop (Chapter 9 input redirection).

Inside loops, `break` exits the loop and `continue` skips to the next round.

21.6 Functions

A function is a named block of code you can call again and again. It keeps scripts short and readable.

```
#!/bin/bash
log() {
  echo "[$(date +%H:%M:%S)] $*"
}

is_up() {
  systemctl is-active --quiet "$1"      # $1 here = the function's first argument
}

check_service() {
  local svc="$1"                        # local = only exists inside this function
  if is_up "$svc"; then
    log "$svc is running"
    return 0
  else
    log "$svc is DOWN"
    return 1
  fi
}

log "starting checks"
for s in nginx ssh cron; do
  check_service "$s" || failed=1
done
exit "${failed:-0}"
```

```
[12:45:10] starting checks
[12:45:10] nginx is running
[12:45:10] ssh is running
[12:45:10] cron is running
```

> **Tip:** A function's `return` is an exit code (0–255), not a value. To "return" text, `echo` it and capture it: `result=$(my_function)`.

21.7 Arrays

```
ajay@web01:~$ servers=(web01 db01 cache01)
ajay@web01:~$ echo "${servers[0]}"          # first item (counting starts at 0)
web01
ajay@web01:~$ echo "${#servers[@]}"        # how many
3
ajay@web01:~$ servers+=(lb01)                # add one
ajay@web01:~$ for s in "${servers[@]}"; do echo "$s"; done

# Associative array (key → value), Bash 4+
ajay@web01:~$ declare -A port=( [nginx]=80 [postgres]=5432 [redis]=6379 )
ajay@web01:~$ echo "${port[postgres]}"
5432
ajay@web01:~$ for svc in "${!port[@]}"; do echo "$svc -> ${port[$svc]}"; done
```

21.8 Exit Codes and Writing Safe Scripts

By default, Bash **keeps going** after a command fails. In automation, that can be a disaster: imagine `cd /srv/app` failing, and the next line being `rm -rf *`.

Put this line at the top of every serious script:

```
#!/bin/bash
set -euo pipefail
```

Setting	Meaning	Without it
`-e`	**Exit** immediately if any command fails	The script carries on after errors
`-u`	Treat **unset** variables as an error	A typo like `$BAKUP_DIR` silently becomes empty
`-o pipefail`	A pipeline fails if **any** part fails	Only the last command's result counts

```
ajay@web01:~$ bash -c 'false | true; echo "without pipefail: $?"'
without pipefail: 0                      # the failure of "false" was hidden!
ajay@web01:~$ bash -c 'set -o pipefail; false | true; echo "with pipefail: $?"'
with pipefail: 1
ajay@web01:~$ bash -c 'set -u; echo $undefined_var; echo "never printed"'
bash: line 1: undefined_var: unbound variable
```

> **Careful! The classic disaster.** `rm -rf "$DIR/"*` with `DIR` unset becomes `rm -rf /*`. With `set -u` the script stops instead. Even better, use `"${DIR:?DIR is not set}"`, which stops with a clear message if DIR is empty.

trap — clean up no matter what

```
#!/bin/bash
set -euo pipefail
tmpdir=$(mktemp -d)                        # a safe, unique temp folder
trap 'rm -rf "$tmpdir"' EXIT               # runs when the script ends, success OR failure
trap 'echo "Failed at line $LINENO" >&2' ERR

curl -fsSL https://example.com/app.tar.gz -o "$tmpdir/app.tar.gz"
tar -xzf "$tmpdir/app.tar.gz" -C "$tmpdir"
echo "Done"
```

Debugging scripts

```
ajay@web01:~$ bash -n script.sh        # check syntax without running
ajay@web01:~$ bash -x script.sh        # show every command as it runs, with values filled in
+ usage=31
+ '[' 31 -ge 90 ']'
+ '[' 31 -ge 80 ']'
+ echo 'OK: / is 31% full'
OK: / is 31% full
ajay@web01:~$ sudo apt install shellcheck -y && shellcheck script.sh   # finds bugs and bad habits
In script.sh line 7:
rm $file
   ^---^ SC2086: Double quote to prevent globbing and word splitting.
```

> **Use ShellCheck on every script.** It catches unquoted variables, wrong tests, and dozens of other mistakes before they hit production. Most CI pipelines run it automatically.

21.9 Real DevOps Scripts

Script 1: Server health check

```
#!/bin/bash
# healthcheck.sh - exit 0 if healthy, 1 if anything is wrong
set -uo pipefail

DISK_LIMIT=85
MEM_LIMIT=90
SERVICES=(nginx ssh cron)
status=0

warn() { echo "WARN: $*"; status=1; }
ok()   { echo "OK:   $*"; }

# 1. Disk
while read -r pct mnt; do
  use=${pct%\%}
  if [ "$use" -gt "$DISK_LIMIT" ]; then warn "$mnt at $pct"; else ok "$mnt at $pct"; fi
done < <(df -h --output=pcent,target -x tmpfs -x devtmpfs -x squashfs | tail -n +2)

# 2. Memory (percent used, based on "available")
mem=$(free | awk '/^Mem:/ {printf "%d", (1 - $7/$2) * 100}')
if [ "$mem" -gt "$MEM_LIMIT" ]; then warn "memory at ${mem}%"; else ok "memory at ${mem}%"; fi

# 3. Load vs CPU count
load=$(cut -d' ' -f1 /proc/loadavg); cpus=$(nproc)
if awk "BEGIN{exit !($load > $cpus)}"; then warn "load $load > $cpus CPUs"; else ok "load $load ($cpus CPUs)"; fi

# 4. Services
for s in "${SERVICES[@]}"; do
  if systemctl is-active --quiet "$s"; then ok "$s running"; else warn "$s NOT running"; fi
done

exit "$status"
```

New Ideas in This Script	
`< <(command)`	**Process substitution**: feed a command's output into a loop. (Unlike a pipe, variables changed inside the loop, like `status`, keep their value afterwards.)
`df -x tmpfs ...`	Exclude RAM-based and snap filesystems from the check.
`awk '/^Mem:/ ...'`	Read the Mem line of `free`; field 7 is "available", field 2 is "total".
`awk "BEGIN{exit !(...)}"`	A trick to compare decimals (load 1.75 vs 2), which Bash can't do.
`exit "$status"`	0 or 1, so cron, monitoring, or `&&` can react to the result.

Script 2: Backup with rotation and remote copy

```
#!/bin/bash
# backup.sh <source-dir> - archive, keep 7 local copies, send to backup server
set -euo pipefail

SRC="${1:?Usage: $0 <source-dir>}"
DEST=/backups
REMOTE="backup@nas01:/srv/backups/$(hostname)/"
KEEP_DAYS=7
STAMP=$(date +%F_%H%M)
NAME="$(basename "$SRC")-$STAMP.tar.gz"

log() { logger -t backup "$*"; echo "$*"; }
trap 'log "FAILED: $NAME (line $LINENO)"' ERR

mkdir -p "$DEST"
tar -czf "$DEST/$NAME" -C "$(dirname "$SRC")" "$(basename "$SRC")"
log "Created $DEST/$NAME ($(du -h "$DEST/$NAME" | cut -f1))"

rsync -az "$DEST/$NAME" "$REMOTE"
log "Copied to $REMOTE"

find "$DEST" -name "$(basename "$SRC")-*.tar.gz" -mtime +"$KEEP_DAYS" -delete
log "Done"
```

`${1:?Usage...}` stops the script with that message if no argument is given. `-C` with `basename` / `dirname` stores clean relative paths inside the archive.

Script 3: Run a command on many servers

```
#!/bin/bash
# multi.sh "command" - run on every server in servers.txt, in parallel
set -uo pipefail
cmd="${1:?Usage: $0 \"command\"}"

while read -r host; do
  [[ -z "$host" || "$host" == \#* ]] && continue          # skip empty lines and comments
  ( out=$(ssh -o ConnectTimeout=5 -o BatchMode=yes "$host" "$cmd" 2>&1) \
      && echo "[$host] OK: $out" || echo "[$host] FAILED: $out" ) &
done < servers.txt
wait                                                      # wait for all background jobs
ajay@laptop:~$ ./multi.sh "uptime -p"
[web01] OK: up 3 days, 4 hours
[db01] OK: up 12 days, 1 hour
[cache01] FAILED: ssh: connect to host cache01 port 22: Connection timed out
```

Script 4: Zero-surprise deploy

```
#!/bin/bash
# deploy.sh <version> - release folders + symlink switch + health check + auto-rollback
set -euo pipefail
VERSION="${1:?Usage: $0 <version>}"
APP=/srv/myapp
NEW="$APP/releases/$VERSION"
PREV=$(readlink -f "$APP/current" || true)

echo "Deploying $VERSION"
mkdir -p "$NEW"
tar -xzf "/tmp/myapp-$VERSION.tar.gz" -C "$NEW"
ln -sfn "$NEW" "$APP/current"                # atomic switch (Chapter 4 links!)
sudo systemctl restart myapp

for i in {1..15}; do
  if curl -fs http://localhost:5000/health >/dev/null; then
    echo "Healthy. Deployed $VERSION"; exit 0
  fi
  sleep 2
done

echo "Health check failed - rolling back to $PREV" >&2
ln -sfn "$PREV" "$APP/current"
sudo systemctl restart myapp
exit 1
```

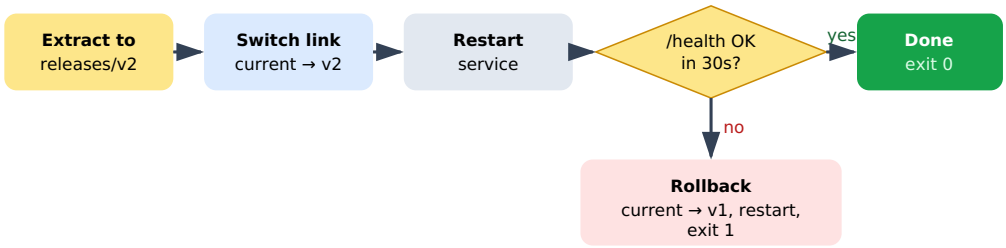


Figure 21.2 — The deploy script's flow: switch, check, and roll back automatically if unhealthy.

Teaching Note: Don't start with an empty file. Give the learner a 5-line working script and grow it together: add an argument check, then a function, then a loop, then `set -euo pipefail`, then ShellCheck. Each step should run successfully before the next one.

Practice Lab 21

1. Write `sysinfo.sh` that prints hostname, IP (`hostname -I`), uptime, disk use of / and memory available.
2. Write `mkuser.sh <username>` that exits with a usage message if no argument is given, and with an error if the user already exists (`id "$1" &>/dev/null`).
3. Write a `case` script `svc.sh {start|stop|status} <service>` .
4. Loop over every `.log` file in `/var/log` and print only those bigger than 1 MB.
5. Read a CSV of `user,group` and create the users (in your lab VM).
6. Write a function `retry N command...` that retries a command up to N times with 2 seconds between tries.
7. Run the health check script; break nginx and run it again. Check `echo $?` both times.
8. Run `shellcheck` on all your scripts and fix every warning.

Chapter 21 Summary

- Start with `#!/bin/bash` and `set -euo pipefail` ; `chmod +x` .
- Variables: `name="x"` (no spaces), use `"$name"` ; `$(cmd)` ; `$(( math ))` ; `${var:-default}` .
- Arguments: `$1 $@ $# $?` . Tests: `-eq -gt -z -f -d` , prefer `[[ ]]` . `case` for menus.
- Loops: `for` , `while` / `until` , `while IFS=, read -r` . Functions with `local` . Arrays `"${arr[@]}"` .
- Exit codes matter; `trap ... EXIT` for clean-up; debug with `bash -x` and `shellcheck` .

Quick Quiz

1. Why does `name = "Ajay"` fail?
2. What is the difference between `'$HOME'` and `"$HOME"`?
3. What do `$#` and `$@` hold?
4. What does `set -o pipefail` change?
5. Which tool finds common mistakes in Bash scripts?

Answers: 1) Spaces around `=` make Bash treat `name` as a command 2) Single quotes print it literally; double quotes expand it to your home path 3) The number of arguments, and all arguments 4) A pipeline fails if any command in it fails 5) `shellcheck`

CHAPTER 22

Ajay Upadhyay · DevOps Mentor

Environment and Shell Customization

How settings reach programs, how PATH finds commands, and making the shell your own.

In this chapter you will learn:

- Shell variables vs environment variables, and `export`
- Important built-in variables like `PATH`, `HOME`, `USER`
- How `PATH` finds commands, and adding to it safely
- Which startup file runs when: `.bashrc`, `.profile`, `/etc/environment`
- Aliases and a better prompt
- Environment variables for services, cron and sudo

22.1 Shell Variables vs Environment Variables

A normal variable you create lives only in your current shell. An **environment variable** is **exported**: it's copied into every program the shell starts.

Real-Life Example: Notes for the Next Shift

A shell variable is a note in your own pocket. Nobody else sees it. `export` pins the note on the notice board, so every worker (child process) who starts their shift after that can read it. But a worker can't change the note in *your* pocket: children never change their parent's variables.

```
ajay@web01:~$ app_env=staging              # shell variable
ajay@web01:~$ bash -c 'echo "child sees: [$app_env]"'
child sees: []                          # not passed to children
ajay@web01:~$ export app_env               # now it's an environment variable
ajay@web01:~$ bash -c 'echo "child sees: [$app_env]"'
child sees: [staging]
ajay@web01:~$ export DB_HOST=db01          # create + export in one step
ajay@web01:~$ DEBUG=1 ./myapp              # set for ONE command only
ajay@web01:~$ unset DB_HOST                # remove it
```

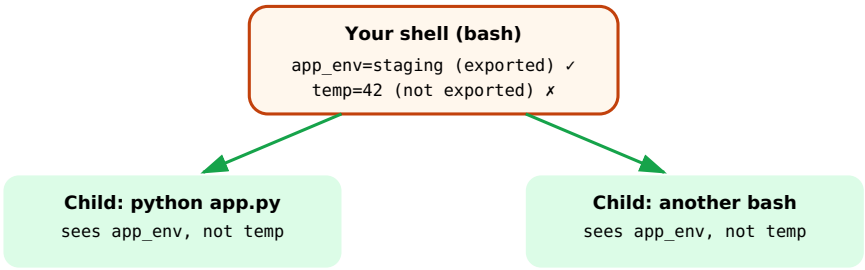


Figure 22.1 — Only exported variables are copied down to child processes.

Seeing variables

```
ajay@web01:~$ env | sort | head -5          # all environment variables
ajay@web01:~$ printenv HOME                # one variable
/home/ajay
ajay@web01:~$ echo "$SHELL $LANG $TERM"
/bin/bash en_US.UTF-8 xterm-256color
ajay@web01:~$ sudo cat /proc/812/environ | tr '\0' '\n'   # env of a running process (PID 812)
```

Variable	Holds
`PATH`	Folders searched for commands (22.2)
`HOME`	Your home folder (what `~` means)
`USER`	Your username
`SHELL`	Your login shell
`PWD` / `OLDPWD`	Current / previous folder (what `cd -` uses)
`LANG`	Language and character encoding
`EDITOR`	Default editor for `crontab -e`, `git commit`, `visudo`
`PS1`	The look of your prompt (22.5)
`HISTSIZE`	How many commands `history` keeps

22.2 PATH — How Commands Are Found

When you type `nginx`, Bash looks through each folder listed in `PATH`, left to right, and runs the **first** match.

```
ajay@web01:~$ echo "$PATH" | tr ':' '\n'
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin
/sbin
/bin
/snap/bin
```



First match wins. That's how /usr/local/bin can override a system command.

Figure 22.2 — PATH search, left to right.

```
ajay@web01:~$ export PATH="$HOME/bin:$PATH"          # ADD your folder to the front (keep the old $PATH!)
ajay@web01:~$ export PATH="$PATH:/opt/tools/bin"      # or to the end
ajay@web01:~$ type -a python3                          # every match, in order
ajay@web01:~$ hash -r                                  # forget remembered locations after moving programs
```

> **Careful!** `export PATH=/opt/tools/bin` (forgetting `$PATH`) **replaces** the whole list. Suddenly `ls`, `sudo` and everything else is "command not found". Fix it in that session with `export PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin`, then correct the file you edited. Also never put `.` (current folder) in PATH: it's a security risk.

22.3 Startup Files: Which One Runs When?

When Bash starts, it reads certain files to set up your environment. Which ones depends on the **type** of shell:

Shell type	When it happens	Files read (Ubuntu)
Login shell	SSH login, console login, `su - user`, `bash -l`	`/etc/profile` → `~/.profile` (which also loads `~/.bashrc`)
Interactive non-login	Opening a new terminal window on a desktop, typing `bash`	`/etc/bash.bashrc` → `~/.bashrc`
Non-interactive	Scripts, cron jobs, `ssh host "cmd"`	Usually **none** of the above

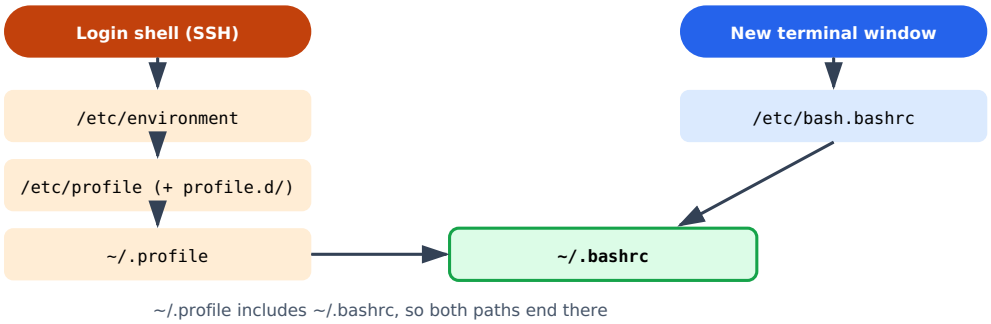


Figure 22.3 — Startup file order on Ubuntu.

Where should I put my setting?

Setting	Best place
Aliases, functions, prompt, history settings	`~/.bashrc`
Your own `PATH` additions and env vars	`~/.profile` (or `~/.bashrc` if you want them in every terminal)
A variable for **all users**	`/etc/environment` (simple `KEY=value` lines, no `export`, no `$VAR`)
A script for all users at login	A new file in `/etc/profile.d/myteam.sh`
Variables for a **service**	The systemd unit: `Environment=` or `EnvironmentFile=` (22.6)

```
ajay@web01:~$ echo 'export EDITOR=vim' >> ~/.bashrc
ajay@web01:~$ source ~/.bashrc          # reload now (or: . ~/.bashrc), no need to log out
ajay@web01:~$ cat /etc/environment
PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin"
HTTP_PROXY="http://proxy.corp.local:3128"
```

What does `source` do? It runs a file **inside your current shell** instead of in a child. That's why changes to variables and aliases take effect right away. `./file.sh` would run in a child, and its changes would vanish when it ends.

22.4 Aliases

An **alias** is a short nickname for a longer command.

```
ajay@web01:~$ alias                             # list current aliases
alias ll='ls -alF'
alias la='ls -A'
alias grep='grep --color=auto'
ajay@web01:~$ nano ~/.bashrc      # add your own at the end:
alias ..='cd ..'
alias ...='cd ../..'
alias ports='sudo ss -tulpn'
alias logs='sudo journalctl -f'
alias dfh='df -hT -x tmpfs -x squashfs'
alias myip='curl -s ifconfig.me; echo'
alias update='sudo apt update && sudo apt upgrade -y'
alias k='kubectl'
alias rm='rm -i'
ajay@web01:~$ source ~/.bashrc
ajay@web01:~$ \rm file.txt          # a backslash skips the alias for one run
ajay@web01:~$ unalias rm            # remove an alias in this session
```

Alias or function? Aliases can't take arguments in the middle. For anything smarter, write a function in `.bashrc`:
`mkcd() { mkdir -p "$1" && cd "$1"; }` → `mkcd ~/projects/new`

22.5 A Better Prompt and History

On servers it helps to see, at a glance, **which machine** you're on. Many teams colour production prompts red.

```
# in ~/.bashrc
PS1='\[\e[1;31m\][PROD]\[\e[0m\] \u@\h:\w\$ '
# result:
[PROD] ajay@web01:/var/log$
```

Code	Shows	Code	Shows
`\u`	Username	`\h`	Hostname
`\w`	Full current folder	`\W`	Last part of folder
`\$`	`$` , or `#` for root	`\t`	Time
`\e[1;31m`	Start bold red	`\e[0m`	Reset colour

```
# Better history (also in ~/.bashrc)
HISTSIZE=10000
HISTFILESIZE=20000
HISTTIMEFORMAT='%F %T  '          # show date and time in "history"
HISTCONTROL=ignoreboth            # skip duplicates and commands starting with a space
shopt -s histappend               # don't overwrite history from other terminals
```

> **Tip:** With `ignoreboth` , a command typed with a **leading space** is not saved in history. Handy when a command contains a secret — though it's better to avoid secrets on the command line entirely.

22.6 Environment Variables for Services, cron and sudo

Services, cron jobs and sudo **don't** read your `.bashrc` . This surprises everyone once.

Where	How to give it variables
systemd service	`Environment="PORT=5000"` or `EnvironmentFile=/etc/myapp/env` in the unit (Ch 13). Check with `systemctl show myapp -p Environment` .
cron	Set them at the top of the crontab (`PATH=...`), or inside the script.
sudo	sudo resets the environment. Use `sudo -E` to keep yours, or `sudo VAR=x command` .
Docker	`docker run -e KEY=value` or `--env-file .env`

```
ajay@web01:~$ sudo nano /etc/myapp/env
DB_HOST=db01
DB_USER=myapp
DB_PASSWORD=S3cr3t!
ajay@web01:~$ sudo chown root:myapp /etc/myapp/env && sudo chmod 640 /etc/myapp/env   # secrets: not world-readable!
```

> **On the Job: "It works in my terminal, but not as a service / in cron."** Nine times out of ten it's a missing environment variable or PATH. Compare: `env` in your terminal vs `systemctl show myapp -p Environment` or a cron line like `* * * * * env > /tmp/cron-env.txt` .

Practice Lab 22

1. Create a variable without export and show a child bash can't see it. Export it and test again.
2. Print your PATH one folder per line. Create `~/bin/hello` (executable), add `~/bin` to PATH in `~/.profile` , log out and in, and run `hello` from anywhere.
3. Add five useful aliases and the `mkcd` function to `~/.bashrc` . Reload with `source` .
4. Give your lab "production" VM a red `[PROD]` prompt.
5. Turn on history timestamps and check `history | tail` .
6. Add a cron line that dumps `env` to a file. Compare it with your terminal's `env` . What's missing?
7. Move your app service's settings into an `EnvironmentFile` with 640 permissions.

Chapter 22 Summary

- `export` turns a variable into an environment variable passed to child processes.
- `PATH` is searched left to right; always extend it with `$PATH`, never replace it.
- Login shells read `/etc/profile` & `~/.profile`; terminals read `~/.bashrc`. Apply with `source`.
- Aliases and functions in `.bashrc`; system-wide vars in `/etc/environment`.
- Services, cron and sudo don't read `.bashrc`: use `EnvironmentFile`, crontab variables, `sudo -E`.

Quick Quiz

1. What does `export` do?
2. What's wrong with `export PATH=/opt/bin`?
3. You added an alias to `.bashrc`. How do you use it without logging out?
4. Where should a systemd service get its DB password from?
5. Your script uses `$JAVA_HOME` from your `.bashrc` but fails in cron. Why?

Answers: 1) Makes a variable visible to child processes 2) It replaces the whole PATH, so normal commands disappear 3) `source ~/.bashrc` 4) An `EnvironmentFile` with restricted permissions 5) cron doesn't read `.bashrc`; set the variable in the crontab or the script.

CHAPTER 23

Ajay Upadhyay · DevOps Mentor

Web Servers and Common Services

Serving websites, putting nginx in front of apps, free SSL certificates, and running a database.

In this chapter you will learn:

- How a web request travels, and what a web server does
- nginx: files, server blocks, enabling sites, testing config
- Reverse proxy to an app, with the right headers
- Load balancing across several app servers
- Free HTTPS with Let's Encrypt (certbot)
- Apache basics, and nginx vs Apache
- MySQL and PostgreSQL: install, secure, create users, back up
- Fixing 403, 404, 502 and 504 errors

23.1 How a Web Request Travels

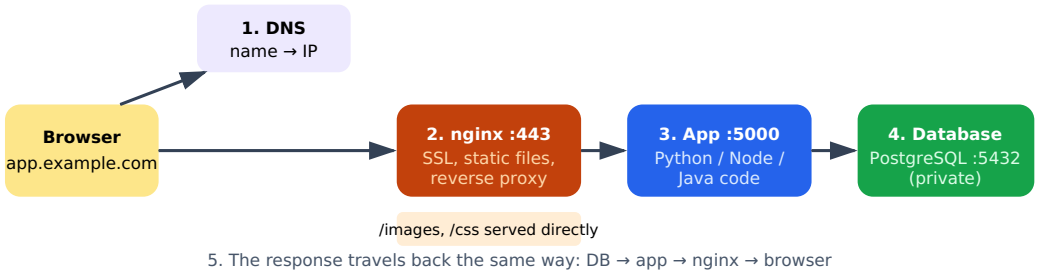


Figure 23.1 — The classic three-tier setup: web server, application, database.

Real-Life Example: A Restaurant

nginx is the waiter at the door: greets every customer (request), hands out ready items like water and menus (static files) himself, and passes real food orders to the **kitchen** (your app). The kitchen takes ingredients from the **store room** (database). Customers never walk into the kitchen or the store room.

23.2 nginx: Files and Layout

```
ajay@web01:~$ sudo apt install nginx -y && systemctl is-active nginx
active
ajay@web01:~$ curl -I localhost
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
```

Path	What it is
`/etc/nginx/nginx.conf`	Main config (workers, logging, includes the folders below)
`/etc/nginx/sites-available/`	One file per website (the "library")
`/etc/nginx/sites-enabled/`	**Symlinks** to the sites that are switched on
`/etc/nginx/conf.d/*.conf`	Extra config, all loaded automatically
`/etc/nginx/snippets/`	Reusable pieces (SSL settings etc.)
`/var/www/html/`	Default website files
`/var/log/nginx/access.log` , `error.log`	Logs (Ch 14)

23.3 Your First Website (Server Block)

```
ajay@web01:~$ sudo mkdir -p /var/www/site1
ajay@web01:~$ echo "<h1>Site 1 by Ajay</h1>" | sudo tee /var/www/site1/index.html
ajay@web01:~$ sudo nano /etc/nginx/sites-available/site1
server {
    listen 80;
    server_name site1.example.com www.site1.example.com;

    root /var/www/site1;
    index index.html;

    access_log /var/log/nginx/site1.access.log;
    error_log  /var/log/nginx/site1.error.log;

    location / {
        try_files $uri $uri/ =404;
    }

    location /downloads/ {
        autoindex on;
    }
}
ajay@web01:~$ sudo ln -s /etc/nginx/sites-available/site1 /etc/nginx/sites-enabled/
ajay@web01:~$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
ajay@web01:~$ sudo systemctl reload nginx
ajay@web01:~$ curl -H "Host: site1.example.com" http://localhost
<h1>Site 1 by Ajay</h1>
```

Server Block Explained	
`listen 80`	Accept HTTP on port 80.
`server_name`	Which domain names this block answers. One nginx can host many sites; it picks the block by the request's `Host` header.
`root`	Folder where the files are.
`index`	File to show for a folder, e.g. `/` → `index.html`.
`location / { }`	Rules for URLs starting with `/`. More specific locations (`/downloads/`) win.
`try_files $uri $uri/ =404`	Try the exact file, then a folder, else return 404.
`autoindex on`	Show a file listing for that folder.

> **The nginx golden routine:** edit → `sudo nginx -t` → `sudo systemctl reload nginx`. Never skip the test. Remove the default site with `sudo rm /etc/nginx/sites-enabled/default` once you have your own.

23.4 Reverse Proxy to an Application

Apps (Node, Python, Java) usually listen on a high port like 5000 or 3000. nginx sits in front on 80/443 and forwards requests to them. This is a **reverse proxy**.

```
ajay@web01:~$ sudo nano /etc/nginx/sites-available/myapp
server {
    listen 80;
    server_name app.example.com;

    location /static/ {
        alias /srv/myapp/current/static/;      # nginx serves these itself (fast)
        expires 7d;
    }

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_http_version 1.1;
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;
    }

    client_max_body_size 20M;                  # allow uploads up to 20 MB
}
```

Reverse Proxy Explained	
`proxy_pass`	Send the request to the app at 127.0.0.1:5000. The app only needs to listen on localhost: safer.
`Host`	Pass the original domain name to the app.
`X-Real-IP / X-Forwarded-For`	The visitor's real IP. Without these, the app thinks every visitor is 127.0.0.1.
`X-Forwarded-Proto`	Tells the app whether the visitor used http or https (needed for correct redirects).
`proxy_read_timeout`	How long to wait for the app. Too short → 504 errors on slow pages.
`client_max_body_size`	Max upload size. Default is 1 MB; bigger uploads get "413 Request Entity Too Large".

WebSocket apps (chat, live dashboards) also need: `proxy_set_header Upgrade $http_upgrade;` and `proxy_set_header Connection "upgrade";`

23.5 Load Balancing

When one app server isn't enough, run several and let nginx spread the traffic.

```
upstream app_servers {
    least_conn;                            # send to the server with fewest active connections
    server 10.0.2.15:5000 max_fails=3 fail_timeout=30s;
    server 10.0.2.16:5000 max_fails=3 fail_timeout=30s;
    server 10.0.2.17:5000 backup;          # only used if the others are down
}

server {
    listen 80;
    server_name app.example.com;
    location / {
        proxy_pass http://app_servers;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

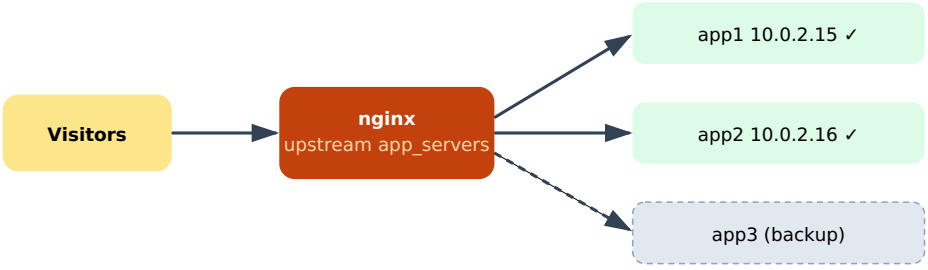


Figure 23.2 — nginx as a load balancer. A failed server is skipped automatically.

Method	How it picks a server
(default) round robin	Takes turns: 1, 2, 1, 2...
`least_conn`	The one with fewest active connections (good for uneven request times)
`ip_hash`	The same visitor IP always goes to the same server ("sticky sessions")
`weight=3`	Give a bigger server more traffic

23.6 Free HTTPS with Let's Encrypt

HTTPS encrypts traffic between browser and server using a **TLS certificate**. Let's Encrypt gives certificates for free, valid for 90 days, renewed automatically by `certbot`.

Before you start: the domain's DNS A record must point to this server, and ports 80 and 443 must be open in UFW and the cloud security group.

```
ajay@web01:~$ sudo apt install certbot python3-certbot-nginx -y
ajay@web01:~$ sudo ufw allow "Nginx Full"
ajay@web01:~$ sudo certbot --nginx -d app.example.com -d www.app.example.com
Requesting a certificate for app.example.com and www.app.example.com
Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/app.example.com/fullchain.pem
Key is saved at:         /etc/letsencrypt/live/app.example.com/privkey.pem
This certificate expires on 2027-01-11.
Deploying certificate
Successfully deployed certificate for app.example.com to /etc/nginx/sites-enabled/myapp
Congratulations! You have successfully enabled HTTPS on https://app.example.com
```

What certbot just did	
`Proved ownership`	Let's Encrypt asked for a special file over http://app.example.com/.well-known/... — only the real owner of the domain could serve it.
`fullchain.pem`	The certificate (public, sent to browsers).
`privkey.pem`	The private key. **Never share it.** Readable only by root.
`Deploying certificate`	Added `listen 443 ssl` and certificate lines to your nginx config, plus an HTTP→HTTPS redirect.

```
ajay@web01:~$ systemctl list-timers | grep certbot          # auto-renewal timer (Chapter 13!)
ajay@web01:~$ sudo certbot renew --dry-run                  # test renewal
ajay@web01:~$ sudo certbot certificates                     # list certs and expiry dates
ajay@web01:~$ echo | openssl s_client -connect app.example.com:443 -servername app.example.com 2>/dev/null | openssl x509
-noout -dates
notBefore=Oct 13 05:12:44 2026 GMT
notAfter=Jan 11 05:12:43 2027 GMT
```

> **Careful!** Let's Encrypt has rate limits. Practise with `--staging` (test certificates) while experimenting, and only request real ones when your setup works.

23.7 Apache in Brief

Apache (`apache2`) is the other big web server. You'll meet it on older systems and with PHP apps like WordPress.

```
ajay@web01:~$ sudo apt install apache2 -y
ajay@web01:~$ sudo nano /etc/apache2/sites-available/site2.conf
<VirtualHost *:80>
    ServerName site2.example.com
    DocumentRoot /var/www/site2
    ErrorLog ${APACHE_LOG_DIR}/site2-error.log
    CustomLog ${APACHE_LOG_DIR}/site2-access.log combined
</VirtualHost>
ajay@web01:~$ sudo a2ensite site2          # enable site (creates the symlink for you)
ajay@web01:~$ sudo a2enmod rewrite proxy_http   # enable modules
ajay@web01:~$ sudo apache2ctl configtest     # test (like nginx -t)
ajay@web01:~$ sudo systemctl reload apache2
```

	nginx	Apache
Design	Event-driven: few processes, thousands of connections	Process/thread per connection (event MPM is better)
Best at	Static files, reverse proxy, load balancing	PHP apps, per-folder `.htaccess` rules
Config test	`nginx -t`	`apache2ctl configtest`
Enable site	`ln -s` into sites-enabled	`a2ensite`

> **Careful!** nginx and Apache both want port 80. If you install both, one will fail with "Address already in use". Run only one on a port, or put Apache on 8080 behind nginx.

23.8 Databases: MySQL and PostgreSQL

MySQL / MariaDB

```
ajay@db01:~$ sudo apt install mysql-server -y
ajay@db01:~$ sudo mysql_secure_installation          # remove test DB, anonymous users, etc.
ajay@db01:~$ sudo mysql                                # on Ubuntu, root logs in via the system root user
mysql> CREATE DATABASE shopdb;
mysql> CREATE USER 'shop'@'10.0.2.%' IDENTIFIED BY 'Str0ng#Pass';
mysql> GRANT ALL PRIVILEGES ON shopdb.* TO 'shop'@'10.0.2.%';
mysql> FLUSH PRIVILEGES;
mysql> SHOW DATABASES;
mysql> EXIT;
```

By default MySQL listens on `127.0.0.1` . To let the app server connect, change `bind-address` in `/etc/mysql/mysql.conf.d/mysqld.cnf` to the private IP, restart, and allow port 3306 **only** from the app subnet in UFW.

```
ajay@db01:~$ mysqldump -u shop -p shopdb | gzip > shopdb-$(date +%F).sql.gz      # backup
ajay@db01:~$ gunzip < shopdb-2026-10-13.sql.gz | mysql -u shop -p shopdb         # restore
```

PostgreSQL

```
ajay@db01:~$ sudo apt install postgresql -y
ajay@db01:~$ sudo -u postgres psql                  # log in as the postgres admin user
postgres=# CREATE USER shop WITH PASSWORD 'Str0ng#Pass';
postgres=# CREATE DATABASE shopdb OWNER shop;
postgres=# \l             # list databases
postgres=# \du            # list users
postgres=# \q             # quit
```

Remote access needs two files in `/etc/postgresql/16/main/` :

```
# postgresql.conf
listen_addresses = 'localhost,10.0.2.20'
# pg_hba.conf  (who may connect, from where, how)
# TYPE  DATABASE  USER  ADDRESS        METHOD
host    shopdb    shop  10.0.2.0/24    scram-sha-256
ajay@db01:~$ sudo systemctl restart postgresql
ajay@web01:~$ psql -h 10.0.2.20 -U shop -d shopdb -c "SELECT version();"
ajay@db01:~$ sudo -u postgres pg_dump shopdb | gzip > shopdb-$(date +%F).sql.gz   # backup
```

Database security in one line: listen only on private IPs, allow the port only from the app servers, give each app its own user with access to only its database, and test your restores.

23.9 Fixing Common Web Errors

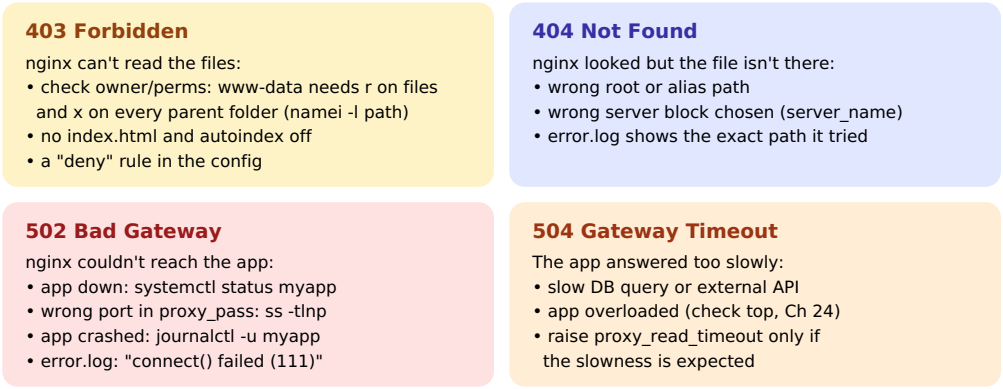


Figure 23.3 — The four web errors you'll see most, and where to look.

```
ajay@web01:~$ sudo tail -f /var/log/nginx/error.log
2026/10/13 13:02:11 [error] 813#813: *42 connect() failed (111: Connection refused) while connecting to upstream, client:
192.168.1.50, server: app.example.com, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:5000/"
```

Output Explained: an nginx error line	
`[error] 813#813`	Level, and the worker PID that hit it.
`connect() failed (111: Connection refused)`	Nothing listens on the upstream port → the app is down or on another port. This produces a **502**.
`client / server / request`	Who asked, which server block, and what URL.
`upstream`	Exactly where nginx tried to forward the request.

Practice Lab 23

1. Install nginx on web01. Create two sites (site1, site2) with different `server_name`s and test both with `curl -H "Host: ..."`.
2. Run the demo app from Level 2's project (Python on port 5000). Put nginx in front as a reverse proxy.
3. Stop the app and load the page. Which error do you get? Find the matching line in error.log.
4. Run two copies of the app (ports 5000 and 5001) and load-balance between them. Stop one and confirm the site still works.
5. Set wrong permissions (chmod 600) on index.html and reproduce a 403. Fix it.
6. If you own a domain, get a Let's Encrypt certificate (use `--staging` first) and test renewal.
7. Install PostgreSQL on db01. Create a user and database, allow access only from web01, and connect from web01 with `psql`.
8. Back up the database with `pg_dump` and restore it into a new database.

Chapter 23 Summary

- nginx sites: `sites-available` + symlink in `sites-enabled`; always `nginx -t` then reload.
- Reverse proxy: `proxy_pass http://127.0.0.1:PORT` + Host / X-Forwarded headers.
- Load balancing: `upstream` with round robin, `least_conn`, `ip_hash`, `backup`.
- HTTPS: `certbot --nginx -d domain`; renewal is automatic (check with `--dry-run`).
- Databases listen privately, per-app users, port allowed only from app servers; back up with mysqldump/pg_dump.
- 403 permissions, 404 path, 502 app down, 504 app slow — read error.log.

Quick Quiz

1. What two commands should follow every nginx config change?
2. Why should the app listen on 127.0.0.1 rather than 0.0.0.0?
3. Without `X-Forwarded-For`, what IP does the app see for every visitor?
4. What must be true before certbot can issue a certificate?
5. The site shows 502. What's the first thing you check?

Answers: 1) `sudo nginx -t` and `sudo systemctl reload nginx` 2) So it's only reachable through nginx, not directly from the internet 3) 127.0.0.1 (nginx's address) 4) DNS points to the server and port 80 is reachable 5) Is the app running and listening on the proxy_pass port? (`systemctl status`, `ss -tlnp`)

CHAPTER 24

Ajay Upadhyay · DevOps Mentor

Performance Monitoring and Troubleshooting

A calm, repeatable method for finding out why a server is slow or broken.

In this chapter you will learn:

- The USE method: Utilization, Saturation, Errors
- The first 60 seconds on a slow server
- Load average explained properly
- CPU, memory, disk and network tools: `vmstat`, `mpstat`, `iostat`, `sar`, `iotop`, `nload`, `iftop`
- Digging into one process: `lsof`, `strace`
- Watching packets with `tcpdump`
- Five real incidents, solved step by step

24.1 A Method, Not Guesswork

Under pressure, people restart random services and hope. Professionals follow a method. The simplest good one is the **USE method**: for every resource (CPU, memory, disk, network), check three things.

Check	Question	Example
Utilization	How busy is it?	CPU 95% busy; disk 100% busy
Saturation	Is work **waiting** in a queue?	Load 8 on 2 CPUs; long disk queue; swapping
Errors	Are there failures?	Disk I/O errors in dmesg; dropped packets; OOM kills

Real-Life Example: A Doctor's Check-up
A good doctor doesn't guess "it's probably your heart". They check pulse, blood pressure, temperature and breathing in order, every time. Your server's vital signs are CPU, memory, disk and network — and for each one you ask: how busy, how much waiting, any errors?

24.2 The First 60 Seconds

Run these ten commands, in order, the moment you log in to a slow server. Together they give you the whole picture.

#	Command	Look for
1	`uptime`	Load average trend (rising or falling?)
2	`dmesg -T | tail`	OOM kills, disk errors, network errors
3	`vmstat 1 5`	Run queue (r), swapping (si/so), CPU split, I/O wait
4	`mpstat -P ALL 1 3`	One CPU at 100% while others idle (single-threaded bottleneck)
5	`pidstat 1 3`	Which processes use the CPU
6	`iostat -xz 1 3`	Disk %util, waiting time (await)
7	`free -h`	available memory, swap used
8	`sar -n DEV 1 3`	Network throughput per interface
9	`sar -n TCP,ETCP 1 3`	New connections per second, retransmits
10	`top` / `htop`	Confirm the guilty process

```
ajay@web01:~$ sudo apt install sysstat iotop nload iftop -y   # mpstat, pidstat, iostat, sar live in "sysstat"
ajay@web01:~$ sudo systemctl enable --now sysstat               # start collecting history for sar
```

24.3 Load Average, Properly

Load average = the average number of processes that are **running** or **waiting** (for CPU, or for disk in "D" state) over 1, 5 and 15 minutes.

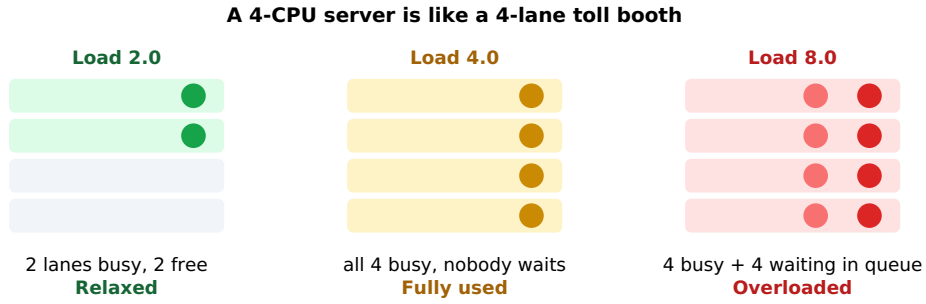


Figure 24.1 — Compare load average with the number of CPUs (`nproc`).

```
ajay@web01:~$ uptime; nproc
 13:10:22 up 3 days,  4:01,  1 user,  load average: 7.82, 4.10, 1.95
4
```

Reading it: the 15-minute value is 1.95, the 5-minute is 4.10, the 1-minute is 7.82. Load is **rising fast** and is now about double the 4 CPUs. Something started 5–10 minutes ago. If the numbers were 1.95, 4.10, 7.82 the other way round, the problem would be calming down.

> **High load but low CPU use?** Remember load also counts processes waiting for **disk** (state D). High load + high `wa` + low `us` = a storage problem, not a CPU problem.

24.4 CPU and Memory: vmstat, mpstat, pidstat

```
ajay@web01:~$ vmstat 1 5
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 9  0      0 512004  61220 1502112    0    0     4    28 2210 4120 88 10  2  0  0
 8  0      0 511876  61220 1502140    0    0     0    12 2190 4054 90  9  1  0  0
 9  1      0 511620  61220 1502168    0    0     0   140 2240 4230 89 10  1  0  0
```

Output Explained: vmstat (ignore the first line; it's an average since boot)	
`r`	**Runnable** processes (running + waiting for CPU). Consistently above the CPU count = CPU saturated. Here 8–9 on 4 CPUs.
`b`	Processes **blocked** on I/O (disk). Should be near 0.
`swpd / free / buff / cache`	Memory in KB. Same meaning as `free`.
`si / so`	**Swap in / out** per second. Any steady non-zero value = memory pressure. 0 here: memory is fine.
`bi / bo`	Blocks read from / written to disk per second.
`in / cs`	Interrupts and context switches per second.
`us sy id wa st`	CPU % for user, system, idle, I/O wait, stolen. Here 88–90% user, ~1% idle: **CPU-bound** by an application.

```
ajay@web01:~$ mpstat -P ALL 1 1 | tail -6      # per-CPU view
Average:     CPU    %usr   %nice    %sys %iowait    %irq   %soft  %steal  %idle
Average:     all   89.25    0.00    9.50    0.00    0.00    0.25    0.00   1.00
Average:       0   90.00    0.00    9.00    0.00    0.00    0.00    0.00   1.00
Average:       1   88.00    0.00   11.00    0.00    0.00    1.00    0.00   0.00
Average:       2   89.00    0.00   10.00    0.00    0.00    0.00    0.00   1.00
Average:       3   90.00    0.00    8.00    0.00    0.00    0.00    0.00   2.00
ajay@web01:~$ pidstat 1 1 | sort -k8 -nr | head -3    # top CPU processes (column 8 = %CPU)
Average:     1001      2741  352.00   4.00    0.00    0.00  356.00     -  python3
```

All four CPUs are evenly busy, and one Python process (PID 2741) is using 356% CPU — that's about 3.5 cores. Now you know exactly where to look next: that app's logs and recent deployments.

Memory leak or just cache?

```
ajay@web01:~$ ps -o pid,rss,cmd -p 2741 ; sleep 60 ; ps -o pid,rss,cmd -p 2741
    PID    RSS CMD
   2741 812344 python3 app.py
    PID    RSS CMD
   2741 905120 python3 app.py
```

Real memory (RSS) grew ~90 MB in one minute and keeps growing = likely a **memory leak** in the app. A stable RSS with shrinking "free" but healthy "available" is just the page cache doing its job.

24.5 Disk: iostat and iotop

```
ajay@db01:~$ iostat -xz 1 2
Device   r/s     w/s     rkB/s    wkB/s  r_await w_await aqu-sz  %util
sda     12.00  845.00   192.00  52480.00    1.20   38.50  32.40  99.80
```

Output Explained: iostat -x	
`r/s, w/s`	Read and write operations per second. Here: very write-heavy.
`rkB/s, wkB/s`	Throughput. ~52 MB/s being written.
`r_await / w_await`	Average wait in ms per request. SSDs should be ~1–5 ms. 38.5 ms writes = the disk can't keep up.
`aqu-sz`	Average queue length (saturation). 32 requests waiting.
`%util`	% of time the disk was busy. Near 100% = fully used. (On fast SSD/NVMe arrays, %util can be misleading; trust await and aqu-sz.)

```
ajay@db01:~$ sudo iotop -oPa          # -o only active, -P processes, -a accumulated totals
Total DISK READ: 0.00 B/s | Total DISK WRITE: 51.24 M/s
    PID  PRIO  USER     DISK READ  DISK WRITE  SWAPIN     IO>    COMMAND
   1544 be/4 mysql        0.00 B     2.91 G     0.00 %  86.12 %  mysqld
   4410 be/4 root         0.00 B   312.00 M     0.00 %   9.40 %  tar -czf /backups/...
```

MySQL is doing most of the writing, but a backup `tar` started during business hours is adding to the load. Fix: move the backup to night time, and run it with `nice -n 19 ionice -c3` (Chapter 12).

24.6 Network: nload, iftop, ss

```
ajay@web01:~$ nload enp0s3                      # live in/out bandwidth graph
ajay@web01:~$ sudo iftop -i enp0s3 -n            # bandwidth per connection (who is talking to whom)
ajay@web01:~$ ss -s                              # connection counts
Total: 2140
TCP:   1988 (estab 412, closed 1501, orphaned 0, timewait 1498)
ajay@web01:~$ ss -tan | awk 'NR>1 {print $1}' | sort | uniq -c | sort -rn   # connections by state
ajay@web01:~$ sar -n DEV 1 3                     # rxkB/s, txkB/s per interface
ajay@web01:~$ ip -s link show enp0s3             # errors and dropped packets counters
```

> **Tip:** Thousands of connections in `TIME-WAIT` are usually normal for a busy web server (closed connections waiting a short time before being cleaned up). A growing number of `CLOSE-WAIT`, on the other hand, means **your app** isn't closing connections properly — a bug.

24.7 Looking Inside One Process: lsof and strace

lsof — list open files

Remember "everything is a file": open files, network sockets and pipes all show up in `lsof`.

```
ajay@web01:~$ sudo lsof -p 2741 | wc -l             # how many files does PID 2741 have open?
ajay@web01:~$ sudo lsof -p 2741 -a -i               # only its network connections
ajay@web01:~$ sudo lsof /var/log/myapp/app.log      # who has this file open?
ajay@web01:~$ sudo lsof +L1                         # DELETED files still held open (eating disk space!)
COMMAND  PID   USER  FD  TYPE DEVICE   SIZE/OFF NLINK  NODE NAME
python3 2741 webapp   3w  REG  252,0 8589934592     0 13371 /var/log/myapp/debug.log (deleted)
```

Output Explained: a deleted-but-open file	
`FD 3w`	File descriptor 3, opened for **w**riting.
`SIZE 8589934592`	8 GB still in use on disk.
`NLINK 0, (deleted)`	Someone ran `rm` on the file, but the app still holds it open, so the space is **not** freed. This is why "df says full but du can't find the files". Fix: restart the app (or truncate via `> /proc/2741/fd/3`).

Too many open files? Check the limit: `cat /proc/2741/limits | grep "open files"`. Raise it for a service with `LimitNOFILE=65535` in its unit file (Chapter 13).

strace — watch a program's system calls

Every time a program opens a file, connects to the network or sleeps, it asks the kernel through a **system call**. `strace` shows those calls live: the ultimate "what is this program actually doing?" tool.

```
ajay@web01:~$ strace -f -e trace=openat,connect ./myscript.sh 2>&1 | tail -4
openat(AT_FDCWD, "/etc/myapp/config.yml", O_RDONLY) = -1 ENOENT (No such file or directory)
connect(3, {sa_family=AF_INET, sin_port=htons(5432), sin_addr=inet_addr("10.0.2.20")}, 16) = -1 ETIMEDOUT (Connection
timed out)
ajay@web01:~$ sudo strace -p 2741 -f -tt -e trace=network   # attach to a running process
ajay@web01:~$ sudo strace -c -p 2741                         # summary: which calls take the most time (Ctrl+C to stop)
```

Output Explained: strace	
`openat(... config.yml) = -1 ENOENT`	The program tried to open its config and **it doesn't exist**. A silent failure revealed.
`connect(... 10.0.2.20:5432) = -1 ETIMEDOUT`	It tried to reach the database and timed out → a firewall or network problem (Chapter 18/20), not an app bug.
`-f / -tt / -e trace=`	Follow child processes / show timestamps / only certain kinds of calls.

> **Careful!** strace slows the traced program down a lot. Use it briefly on production, with `-e` filters, and detach (Ctrl+C) as soon as you have what you need.

24.8 tcpdump — See the Actual Packets

When two systems disagree about what happened ("I sent it!" — "I never got it!"), `tcpdump` shows the truth on the wire.

```
ajay@db01:~$ sudo tcpdump -i enp0s3 -nn port 5432 -c 6
13:20:01.112 IP 10.0.2.15.51234 > 10.0.2.20.5432: Flags [S], seq 1840... length 0
13:20:02.120 IP 10.0.2.15.51234 > 10.0.2.20.5432: Flags [S], seq 1840... length 0
13:20:04.136 IP 10.0.2.15.51234 > 10.0.2.20.5432: Flags [S], seq 1840... length 0
```

Output Explained: tcpdump	
`-i enp0s3 -nn`	Listen on this interface; show numbers, not names.
`port 5432 -c 6`	Only PostgreSQL traffic; stop after 6 packets.
`10.0.2.15.51234 > 10.0.2.20.5432`	From the app server (random port 51234) to the DB on port 5432.
`Flags [S]`	SYN = "can we talk?" (start of a TCP connection). A healthy reply would be `[S.]` (SYN-ACK).
`Repeated [S], no reply`	The packets **arrive** at db01, but db01 never answers → db01's own firewall (UFW) is dropping them. If they didn't appear at all, the block would be earlier (cloud security group, routing).

Filter	Captures
`host 10.0.2.15`	Traffic to/from one IP
`port 443` / `tcp` / `udp port 53`	By port or protocol (DNS = udp 53)
`src 10.0.2.15 and dst port 5432`	Combine with and / or / not
`-w capture.pcap`	Save to a file; open later in Wireshark
`-A`	Show packet contents as text (plain HTTP only)

24.9 History: What Happened at 3 AM? (sar)

Most tools show **now**. With sysstat enabled, `sar` records stats every 10 minutes, so you can look at the past.

```
ajay@web01:~$ sar -u -s 02:30:00 -e 04:00:00          # CPU between 2:30 and 4:00 today
02:30:01     CPU     %user     %nice   %system   %iowait    %steal     %idle
02:40:01     all      4.12      0.00      1.20      0.31      0.00     94.37
02:50:01     all     12.40      0.00      3.10     41.80      0.00     42.70
03:00:01     all     15.22      0.00      4.02     62.15      0.00     18.61
ajay@web01:~$ sar -r -f /var/log/sysstat/sa12        # memory, from yesterday's file (day 12)
ajay@web01:~$ sar -q                                  # run queue and load average history
```

At 02:50 iowait jumped to 42% and then 62%. Compare with Chapter 16: what cron jobs run at 02:30? (`grep CRON /var/log/syslog` .) That's how you connect "the site was slow at 3 AM" to "the backup started at 2:30".

24.10 The Troubleshooting Flowchart

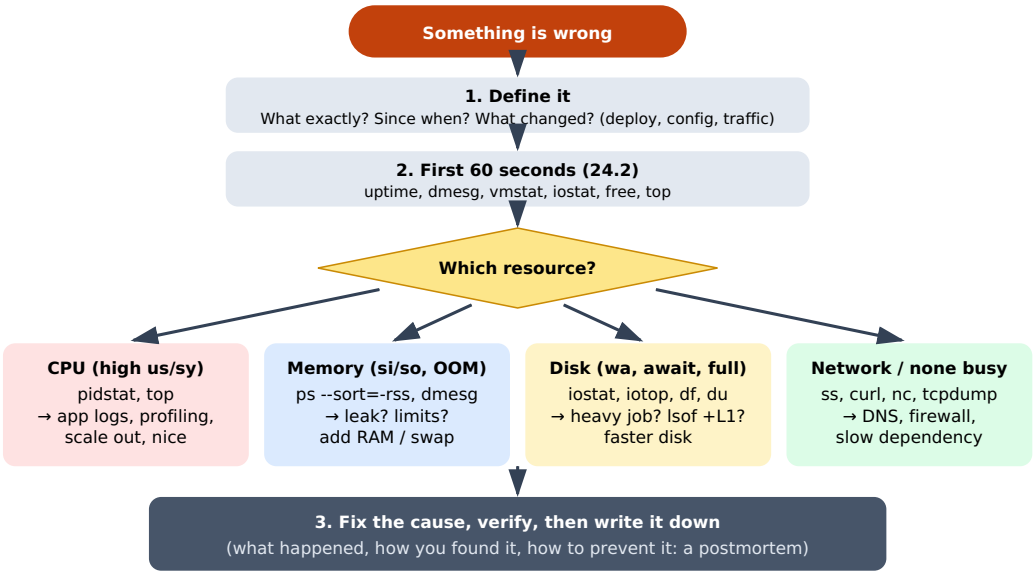


Figure 24.2 — The complete troubleshooting flow. "What changed?" solves more incidents than any tool.

24.11 Five Real Incidents, Solved

Incident 1: "The website is very slow."
`uptime` : load 7.8 on 4 CPUs, rising. `vmstat` : r=9, us=89%, wa=0. `pidstat` : one python3 at 356%. App log shows a new report endpoint deployed 10 minutes ago, called in a loop. **Fix:** roll back the deploy (Chapter 21's script), then fix the endpoint.

Incident 2: "Disk full, but du can't find the big files."
`df -h` : / at 100%. `du -sh /*` adds up to only 14 GB of 24 GB. `sudo lsof +L1` : an 8 GB deleted debug.log held by the app. **Fix:** restart the app to release it; add logrotate with `copytruncate` (Chapter 14).

Incident 3: "The database died at night."
`journalctl -u mysql -b` : nothing useful. `dmesg -T | grep -i oom` : "Killed process 1544 (mysqld)" at 03:12. `sar -r` : memory climbed from 02:30. Cron: a report job loading a huge table into memory on the same server. **Fix:** move the report to a replica, tune MySQL's buffer size, add swap as a safety net.

Incident 4: "The app can't connect to the database after the migration."
`nc -zv db01 5432` : timeout (not refused). `tcpdump port 5432` on db01: SYN packets arrive, no reply. `ufw status` on db01: rule allows the **old** app subnet only. **Fix:** `ufw allow from 10.0.3.0/24 to any port 5432 proto tcp` .

Incident 5: "Random 502 errors under load."
nginx error.log: "connect() failed (11: Resource temporarily unavailable)" and "Too many open files". `cat /proc/$(pidof -s python3)/limits` : open files 1024. **Fix:** `LimitNOFILE=65535` in the app's systemd unit, `daemon-reload` , restart; add a second app server behind the load balancer (Chapter 23).

Teaching Note: Practise with deliberate "chaos". Use `stress-ng` (`sudo apt install stress-ng`) to create CPU, memory or disk load: e.g. `stress-ng --cpu 4 --timeout 120s` , `--vm 2 --vm-bytes 80%` , or `--hdd 2` . Without telling the learner which one you started, have them find it using only the first-60-seconds commands.

Practice Lab 24

1. Install sysstat and enable it. Run all ten "first 60 seconds" commands and write one line about what each shows.
2. Run `stress-ng --cpu $(nproc) --timeout 90s` and watch it in `uptime` , `vmstat 1` and `mpstat -P ALL 1` .
3. Run `stress-ng --hdd 2 --timeout 90s` . Which columns change in `vmstat` and `iostat -xz 1` ? Find it in `iotop` .
4. Run `stress-ng --vm 2 --vm-bytes 90% --timeout 60s` . Watch `si/so` and `free -h` . Check `dmesg` for OOM messages.
5. Reproduce Incident 2: write a big file with a background `tail -f` holding it, delete it, then find it with `lsof +L1` .
6. Use `strace -e trace=openat` on `cat /etc/hostname` . Which files does it open?
7. Reproduce Incident 4 between web01 and db01 using UFW, and diagnose it with `nc` and `tcpdump` .
8. The next day, use `sar` to find the exact times of your stress tests.

Chapter 24 Summary

- Use a method: USE (Utilization, Saturation, Errors) and the first 60 seconds. Ask "what changed?".
- Load average vs `nproc` ; rising 1-min vs 15-min shows the trend. High load + high wa = disk.
- `vmstat` (r, si/so, us/wa), `mpstat` , `pidstat` , `iostat -xz` (await, %util), `iotop` , `nload` / `iftop` , `ss` .
- `lsof +L1` finds deleted-but-open files; `strace` shows what a process really does; `tcpdump` shows packets.
- `sar` answers "what happened at 3 AM?". Fix the cause, then write a postmortem.

Quick Quiz

1. Load average is 6.0 on a 2-CPU server, but CPU is 70% idle with 60% iowait. What's the bottleneck?
2. In vmstat, what does a steady non-zero `so` column mean?
3. df says / is full, but du finds much less. Which command helps?
4. tcpdump on the DB shows SYN packets arriving but no SYN-ACK replies. Where is the problem?
5. Which tool shows what the CPU was doing at 3 AM yesterday?

Answers: 1) Disk I/O (processes waiting on disk count toward load) 2) The system is swapping out: memory pressure 3) `sudo lsof +L1` 4) The DB server itself is dropping them: its local firewall (or the service isn't listening, which would usually send a reset instead) 5) `sar` (from sysstat)

LEVEL 3 FINALE

Ajay Upadhyay · DevOps Mentor

Final Project: Secure Two-Tier Production Setup

Build, secure, automate and break-test a real web + database setup using Chapters 18–24.

Scenario: You have two fresh Ubuntu 24.04 servers on a private network: `web01` (10.0.2.15, also has a public IP) and `db01` (10.0.2.20, private only). Build a secure, monitored app setup. Try each step yourself before reading the solution.

#	Requirement	Chapter
1	Key-only SSH on both; from your laptop, reach db01 **only** through web01 (ProxyJump).	19
2	Hostnames and `/etc/hosts` so the servers find each other by name.	18
3	UFW: web01 allows 22, 80, 443; db01 allows 22 from web01 only and 5432 from web01 only. fail2ban on both.	20
4	PostgreSQL on db01 with an `app` user/database, listening on the private IP.	23
5	A demo app on web01 as a systemd service with an `EnvironmentFile` holding the DB settings.	13, 22
6	nginx reverse proxy in front of the app (HTTPS with certbot if you have a domain).	23
7	A health-check script with exit codes, run every 5 minutes; failures logged to the journal.	21, 16
8	Nightly `pg_dump` backup from db01, copied to web01 with rsync.	21, 19
9	Automatic security updates and sysstat on both servers. Run lynis and record the score.	20, 24
10	Break-test: stop PostgreSQL, then block 5432 in UFW. Diagnose each failure from web01 using only the tools in this book.	18, 24

Solution

```
# ---------- 1. SSH (on your laptop) ----------
ajay@laptop:~$ ssh-keygen -t ed25519 -C "ajay@laptop"
ajay@laptop:~$ ssh-copy-id ajay@<web01-public-ip>
ajay@laptop:~$ cat >> ~/.ssh/config <<'EOF'
Host web01
    HostName <web01-public-ip>
    User ajay
Host db01
    HostName 10.0.2.20
    User ajay
    ProxyJump web01
EOF
ajay@laptop:~$ ssh-copy-id db01          # works through the jump host
# On BOTH servers: keys only, no root (keep a session open while testing!)
ajay@web01:~$ printf "PermitRootLogin no\nPasswordAuthentication no\nKbdInteractiveAuthentication no\nAllowUsers ajay\n" \
                | sudo tee /etc/ssh/sshd_config.d/99-hardening.conf
ajay@web01:~$ sudo sshd -t && sudo systemctl reload ssh

# ---------- 2. Names ----------
ajay@web01:~$ sudo hostnamectl set-hostname web01
ajay@web01:~$ echo "10.0.2.20 db01" | sudo tee -a /etc/hosts
ajay@db01:~$  sudo hostnamectl set-hostname db01
ajay@db01:~$  echo "10.0.2.15 web01" | sudo tee -a /etc/hosts

# ---------- 3. Firewalls + fail2ban ----------
ajay@web01:~$ sudo ufw default deny incoming && sudo ufw default allow outgoing
ajay@web01:~$ sudo ufw limit OpenSSH && sudo ufw allow "Nginx Full" && sudo ufw enable
ajay@db01:~$  sudo ufw default deny incoming && sudo ufw default allow outgoing
ajay@db01:~$  sudo ufw allow from 10.0.2.15 to any port 22 proto tcp
ajay@db01:~$  sudo ufw allow from 10.0.2.15 to any port 5432 proto tcp
ajay@db01:~$  sudo ufw enable
ajay@both:~$  sudo apt install -y fail2ban && printf "[sshd]\nenabled = true\n" | sudo tee /etc/fail2ban/jail.local \
                 && sudo systemctl enable --now fail2ban

# ---------- 4. PostgreSQL on db01 ----------
ajay@db01:~$ sudo apt install -y postgresql
```

```
ajay@db01:~$ sudo -u postgres psql -c "CREATE USER app WITH PASSWORD 'Ch4nge-Me!';"
ajay@db01:~$ sudo -u postgres psql -c "CREATE DATABASE appdb OWNER app;"
ajay@db01:~$ sudo sed -i "s/^#listen_addresses.*/listen_addresses = 'localhost,10.0.2.20'/"
/etc/postgresql/16/main/postgresql.conf
ajay@db01:~$ echo "host appdb app 10.0.2.15/32 scram-sha-256" | sudo tee -a /etc/postgresql/16/main/pg_hba.conf
ajay@db01:~$ sudo systemctl restart postgresql && sudo ss -tlnp | grep 5432

# ---------- 5. App service on web01 ----------
ajay@web01:~$ sudo apt install -y postgresql-client nginx
ajay@web01:~$ sudo useradd --system --shell /usr/sbin/nologin webapp
ajay@web01:~$ sudo mkdir -p /srv/app /etc/webapp
ajay@web01:~$ sudo tee /srv/app/app.py <<'EOF'
import os, subprocess
from http.server import BaseHTTPRequestHandler, HTTPServer
class H(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/health":
            ok = subprocess.run(["pg_isready", "-h", os.environ["DB_HOST"], "-p", "5432"]).returncode == 0
            self.send_response(200 if ok else 503); self.end_headers()
            self.wfile.write(b"db ok\n" if ok else b"db down\n"); return
        self.send_response(200); self.end_headers()
        self.wfile.write(b"Hello from Ajay's production app\n")
HTTPServer(("127.0.0.1", int(os.environ.get("PORT", 5000))), H).serve_forever()
EOF
ajay@web01:~$ printf "PORT=5000\nDB_HOST=db01\nDB_USER=app\nDB_PASSWORD=Ch4nge-Me!\n" | sudo tee /etc/webapp/env
ajay@web01:~$ sudo chown root:webapp /etc/webapp/env && sudo chmod 640 /etc/webapp/env
ajay@web01:~$ sudo tee /etc/systemd/system/webapp.service <<'EOF'
[Unit]
Description=Demo production app
After=network-online.target
Wants=network-online.target
[Service]
User=webapp
EnvironmentFile=/etc/webapp/env
ExecStart=/usr/bin/python3 /srv/app/app.py
Restart=on-failure
LimitNOFILE=65535
[Install]
WantedBy=multi-user.target
EOF
ajay@web01:~$ sudo systemctl daemon-reload && sudo systemctl enable --now webapp
ajay@web01:~$ curl -s localhost:5000/health

# ---------- 6. nginx reverse proxy ----------
ajay@web01:~$ sudo tee /etc/nginx/sites-available/webapp <<'EOF'
server {
    listen 80;
    server_name _;
    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
EOF
ajay@web01:~$ sudo rm -f /etc/nginx/sites-enabled/default
ajay@web01:~$ sudo ln -s /etc/nginx/sites-available/webapp /etc/nginx/sites-enabled/
ajay@web01:~$ sudo nginx -t && sudo systemctl reload nginx && curl -s localhost/health
# With a domain:  sudo certbot --nginx -d app.yourdomain.com

# ---------- 7. Health check every 5 minutes ----------
ajay@web01:~$ sudo tee /usr/local/bin/app-health <<'EOF'
#!/bin/bash
set -uo pipefail
fail=0
check() { if "$@" >/dev/null 2>&1; then :; else logger -t app-health -p user.err "FAILED: $*"; fail=1; fi; }
check systemctl is-active --quiet nginx
check systemctl is-active --quiet webapp
check curl -fs --max-time 5 http://localhost/health
use=$(df / --output=pcent | tail -1 | tr -dc '0-9')
[ "$use" -gt 85 ] && { logger -t app-health -p user.warning "disk / at ${use}%"; fail=1; }
```

```
[ "$fail" -eq 0 ] && logger -t app-health "all checks passed"
exit "$fail"
EOF
ajay@web01:~$ sudo chmod 755 /usr/local/bin/app-health
ajay@web01:~$ echo "*/5 * * * * root /usr/local/bin/app-health" | sudo tee /etc/cron.d/app-health
ajay@web01:~$ sudo /usr/local/bin/app-health; echo "exit=$?"; journalctl -t app-health -n 3

# ---------- 8. Nightly DB backup (on db01) ----------
ajay@db01:~$ sudo tee /usr/local/bin/db-backup <<'EOF'
#!/bin/bash
set -euo pipefail
DEST=/var/backups/pg; STAMP=$(date +%F)
mkdir -p "$DEST"
sudo -u postgres pg_dump appdb | gzip > "$DEST/appdb-$STAMP.sql.gz"
rsync -az "$DEST/appdb-$STAMP.sql.gz" ajay@web01:/home/ajay/db-backups/
find "$DEST" -name "appdb-*.sql.gz" -mtime +7 -delete
logger -t db-backup "OK appdb-$STAMP"
EOF
ajay@db01:~$ sudo chmod 755 /usr/local/bin/db-backup
# give db01's root an SSH key that web01 accepts (for rsync), then:
ajay@db01:~$ echo "15 1 * * * root /usr/local/bin/db-backup >> /var/log/db-backup.log 2>&1" | sudo tee /etc/cron.d/db-
backup

# ---------- 9. Updates, sysstat, audit (both) ----------
ajay@both:~$ sudo apt install -y unattended-upgrades sysstat lynis
ajay@both:~$ sudo dpkg-reconfigure -plow unattended-upgrades && sudo systemctl enable --now sysstat
ajay@both:~$ sudo lynis audit system | grep "Hardening index"

# ---------- 10. Break-test ----------
ajay@db01:~$  sudo systemctl stop postgresql
ajay@web01:~$ curl -s localhost/health; nc -zv db01 5432          # "db down" + Connection REFUSED = service down
ajay@db01:~$  sudo systemctl start postgresql && sudo ufw delete allow from 10.0.2.15 to any port 5432 proto tcp
ajay@web01:~$ timeout 5 nc -zv db01 5432                           # TIMEOUT = firewall
ajay@db01:~$  sudo tcpdump -i any -nn port 5432 -c 3               # SYNs arrive, no replies → local firewall
ajay@db01:~$  sudo ufw allow from 10.0.2.15 to any port 5432 proto tcp   # fix, and confirm /health is "db ok"
```

Where to go from here: Everything in this project is done by hand on purpose, so you understand each piece. In Level 4 you'll automate the same setup with Ansible and cloud-init, run the app in Docker, and add proper monitoring with Prometheus and Grafana.

Level 3 Cheat Sheet

Print this page and keep it next to your keyboard. — *Ajay Upadhyay*

Networking (Ch 18)

`ip -br a` / `ip route`	IPs / gateway
`sudo ss -tulpn`	Listening ports
`ping -c4` / `traceroute -n`	Reachability / path
`nc -zv host port`	Port open? refused/timeout
`dig +short name` / `@8.8.8.8`	DNS lookup
`curl -I` / `-v` / `-w "%{http_code}"`	HTTP checks
`sudo netplan try`	Apply network safely
`resolvectl status`	Real DNS servers
/24=256 · /16=65,536 · /32=1	CIDR sizes

SSH (Ch 19)

`ssh-keygen -t ed25519`	Make keys
`ssh-copy-id user@host`	Install key
`~/.ssh/config`	Nicknames
Host/User/ProxyJump	
`sshd -t` + 2nd session	Safe hardening
`rsync -avz [--delete -n] src/ h:dst/`	Smart copy
`ssh -L 5433:localhost:5432 h -N`	Local tunnel
`ssh -J bastion host`	Jump host
`ssh -v` / `ssh-keygen -R h`	Debug / reset host key

Firewall & Security (Ch 20)

`ufw allow OpenSSH` then `enable`	Never lock out
`ufw allow from CIDR to any port N proto tcp`	Restricted rule
`ufw status numbered` / `delete N` / `insert 1`	Manage rules
`iptables -L -n -v` / `nft list ruleset`	Read low-level
`fail2ban-client status sshd`	Bans
`aa-status` / `journalctl -k | grep DENIED`	AppArmor
`lynis audit system`	Hardening score

Bash (Ch 21)

`#!/bin/bash` + `set -euo pipefail`	Safe start
`v="x"` · `"$v"` · `$(cmd)` · `$((1+2))`	Variables
`$1 $@ $# $? ${v:-def} ${1:?msg}`	Args & defaults
`if [[ ... ]]; then ... elif ... else ... fi`	Conditions
`-eq -gt -z -n -f -d`	Tests
`for x in ...; do ...; done`	Loop
`while IFS=, read -r a b; do ...; done < f`	Read lines
`f() { local x="$1"; }`	Function
`arr=(a b); "${arr[@]}"`	Arrays
`trap '...' EXIT ERR`	Clean-up
`bash -x` / `shellcheck`	Debug / lint

Environment (Ch 22)

`export VAR=x` / `env`	Env vars
`export PATH="$HOME/bin:$PATH"`	Extend PATH
`~/.bashrc` / `~/.profile` / `source`	Startup files
`EnvironmentFile=` (640)	Service vars

Web & DB (Ch 23)

`nginx -t && systemctl reload nginx`	Golden routine
`proxy_pass http://127.0.0.1:5000;`	Reverse proxy
`upstream { least_conn; server ...; }`	Load balance
`certbot --nginx -d domain`	Free HTTPS
`sudo -u postgres psql` / `pg_dump`	PostgreSQL
403 perms · 404 path · 502 app down · 504 slow	Errors

Troubleshooting (Ch 24)

`uptime; dmesg -T | tail; vmstat 1 5`	First look
`mpstat -P ALL 1` / `pidstat 1`	CPU
`iostat -xz 1` / `iotop -oPa`	Disk
`lsof +L1` / `lsof -i :80`	Open files
`strace -p PID -e trace=network`	Syscalls
`tcpdump -i any -nn port N`	Packets
`sar -u -s 02:00:00`	History

You finished Level 3!

You can now design and debug networks, secure SSH and firewalls, automate with Bash, manage environments, run nginx, SSL and databases, and troubleshoot production calmly with a method. Together with Levels 1 and 2, that's a complete Linux administrator's foundation. **Next up, Level 4 (Expert / DevOps):** Linux internals and kernel tuning, containers from the inside (namespaces and cgroups), Docker, Ansible and cloud-init, Git, backup and recovery, and running Linux in the cloud with Prometheus and Grafana.

Questions or feedback? Reach me at **ajayupadhyaydevops@gmail.com** — Ajay Upadhyay, DevOps Mentor.